TeacherSupporter

The Complete Stack

From Spring Boot to Kubernetes and CI/CD —
a self-contained field guide, taught from one real project

Vuong Dang

written with Claude

August 2026

How to Read This Book

This book teaches every technology inside the TeacherSupporter project — Spring Boot and Spring Cloud, Postgres and MongoDB, Kafka, Docker, Kubernetes, and the Jenkins pipeline that ties them together — using the project's own files as the worked examples. Nothing is invented for the page: every configuration you will read is deployed, or about to be deployed, on a real two-core home server called `ts-server`.

The rules the book follows

Concepts before configs. Each chapter first teaches the idea in vendor-neutral terms, then reads the project's real file line by line, then shows one or two examples from *outside* the project so you can tell what generalizes from what is local habit.

Decisions are first-class. Wherever the project chose between alternatives (Jib over `docker build`, Kustomize over Helm, a second Postgres server over a second database) the reasoning is in a DECISION box. These are the "why did you…" questions interviews are made of.

Failure is shown symptom-first. PITFALL boxes start with what you would actually see (`CrashLoopBackOff`, a TLS error, a silent 404) and work back to the cause, because that is the direction real debugging runs in.

The box legend

DECISION — EXAMPLE

Why the project chose one design over another, and what would have to change for the other choice to win.

PITFALL — EXAMPLE

A symptom you will eventually meet, followed by its cause and fix.

HANDS-ON — EXAMPLE

A command to run against the real stack, with the output you should expect. The book is meant to be read next to a terminal.

ON THIS HARDWARE — EXAMPLE

A choice that is only correct on this machine (2 cores, 19.4 GB RAM, SATA SSD, Wi-Fi). Do not copy these into a data center.

CHECK YOURSELF (answers: Appendix A)

Every chapter ends with a few questions in this box. Answers are in Appendix A. Write your answer down before checking — recognition is not recall.

Notation

Inline code and file names look like `application.yml`. Annotated listings carry circled markers — ①, ② — explained directly beneath the listing. Shell commands are shown with a `$` prompt for the server and `PS>` for Windows PowerShell; do not type the prompt.

What is assumed, and what is not

Assumed: you can read Java, and you have used Maven and git. Not assumed: any prior Kubernetes, Jenkins, Kafka, Docker internals, or frontend knowledge. Where the project's own documentation already covers one-time machine setup (`docs/RUNBOOK-SERVER-SETUP.md`), this book summarizes and moves on.

The Project Map

TeacherSupporter is a course-management platform built as Spring Cloud microservices: teachers manage courses, students, assignments and submissions; a dictionary service serves vocabulary lookups; registration and login (password, Google OAuth2, and TOTP two-factor) live in a dedicated auth service; e-mail notifications flow through Kafka. It is deliberately built with the "classic" Spring Cloud stack — Eureka, Config Server, Spring Cloud Gateway — because that stack still runs in a great many enterprises, and understanding it makes the cloud-native replacements (Chapter 26) meaningful rather than magical.

The cast

Service	Port	Backing store	Job
config-server	8888	— (classpath)	serves every service's config
discovery-server	8761	—	Eureka registry
api-gateway	8080	—	single entry point, JWT check
auth-service	8081	Postgres `ts_auth`	accounts, JWT, OAuth2, 2FA
course-service	8082	Postgres `ts_course`, MinIO	courses, students, files
dictionary-service	8083	MongoDB	vocabulary lookups
notification-service	8084	— (Kafka in)	events → e-mail
frontend	3000	—	web UI (React)

Table 1: The eight deployables and what they own.

One request, end to end

Every chapter illuminates one link of this chain. Part I is the boxes on the right; Part II the data stores; Part III the Kafka path (not drawn — it carries events, not requests); Parts IV–VI are how all of it gets packaged, run, and shipped.

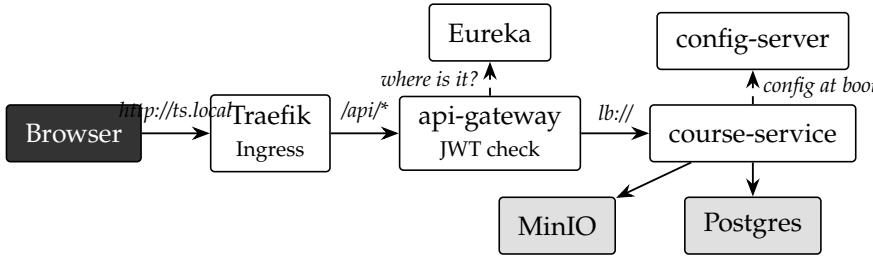

Figure 1: The request path this book keeps returning to.

Where things live in the repository

```
TeacherSupporter/
  pom.xml                  parent: modules, shared plugin config (Jib!)
  common/                   shared DTOs and events
  config-server/           + src/main/resources/configurations/*.yml
  discovery-server/         api-gateway/        auth-service/
  course-service/          dictionary-service/ notification-service/
  frontend/
  docker-compose.yml        local dev stack
  Jenkinsfile              the CI/CD pipeline (Chapter 23)
  deploy/
    jenkins/               Jenkins controller image + run script
    k8s/
      create-secrets.sh     secrets are made, not committed
      jenkins-rbac.yaml     what Jenkins may touch in the cluster
      base/                every Kubernetes manifest (Part V)
  docs/                    design docs, runbooks, and this book
```

The machine underneath

Everything deploys to ts-server: a repurposed laptop running Ubuntu 24.04 with 2 physical cores, 19.4 GB RAM and a SATA SSD, reachable from Windows over Tailscale. It runs k3s (a single-node Kubernetes), a Docker-hosted Jenkins, and a local image registry. The constraint that shapes almost every operational decision in this book is *CPU, not memory*: two cores means builds and demos must not overlap.

Contents

Part I

The Application

Spring Boot: The Machine Under Every Service

Seven of the eight deployables in this project are Spring Boot applications. Before any YAML file can make sense, you need three ideas: what a *starter* pulls in, what *auto-configuration* decides on your behalf, and — most important for everything in Parts V and VI — exactly *how Spring resolves a configuration property* when the same key is defined in several places at once.

1.1 The problem Spring Boot solves

Classic Spring asked you to wire everything: a servlet container to deploy into, XML or Java config declaring every bean, and a long list of version-compatible dependencies. Spring Boot inverts this. An application is a plain `main()` method:

```
@SpringBootApplication
public class CourseServiceApplication {
  public static void main(String[] args) {
    SpringApplication.run(CourseServiceApplication.class, args);
  }
}
```

`@SpringBootApplication` is three annotations in a trench coat: `@Configuration` (this class may define beans), `@ComponentScan` (find `@Service`, `@RestController`, … in this package and below), and `@EnableAutoConfiguration` — the interesting one.

1.1.1 Starters

A starter such as `spring-boot-starter-web` is an empty jar whose only job is a curated dependency list: Spring MVC, Jackson, and an *embedded* Tomcat. "Embedded" matters operationally: the server is inside the jar, so `java -jar course-service.jar` is a complete web server. That single fact is what makes the container images of Chapter 13 so simple — there is no Tomcat to install, only a JVM to provide.

1.1.2 Auto-configuration

At startup, Spring Boot walks a list of candidate configurations and applies each one whose conditions hold. The conditions read like:

- *is this class on the classpath?* (`@ConditionalOnClass` — you added the Postgres driver, so a `DataSource` is configured)
- *has the user defined their own bean already?* (`@ConditionalOnMissingBean` — your explicit bean always wins)
- *is a property set?* (`@ConditionalOnProperty`)

Auto-configuration is why `course-service` contains no code creating a connection pool, a Kafka producer, or an S3 client wiring — adding the dependency plus a few properties *is* the configuration. When behavior seems to appear from nowhere, run the app with `–debug` once: the "condition evaluation report" lists every auto-configuration applied and skipped, with reasons.

1.2 Externalized configuration: the property resolution order

This is the single most load-bearing mechanism in the whole book. Every trick the Kubernetes manifests play in Part V works because of it.

A Spring property such as `spring.datasource.url` can be defined in many *property sources*. Spring merges them all into one `Environment`, and when two sources define the same key, the more specific source wins. Simplified to the sources this project actually uses, from weakest to strongest:

1. defaults inside `application.yml` packaged in the jar
2. config fetched from config-server (Chapter 2)
3. OS environment variables
4. command-line arguments

DECISION — WHY THE CLUSTER OVERRIDES WITH ENV VARS

The same image must run unchanged on a laptop (Compose) and in k3s. Baking environment differences into the jar would mean one image per environment — exactly what containers are meant to avoid. Environment variables outrank both the packaged YAML and the config-server, so the Kubernetes Deployment can repoint a service at `postgres-course:5432` without touching a single file the developers own. One artifact, many environments, differences injected at the edge.

1.2.1 Relaxed binding

Environment variable names cannot contain dots. Spring therefore maps SPRING_DATASOURCE_URL → spring.datasource.url: uppercase, dots become underscores. Read any Deployment in deploy/k8s/base/ and you can now decode every env: entry as "this overrides that YAML key." For example SPRING_DATA_MONGODB_URI in dictionary-service.yaml overrides spring.data.mongodb.uri from the config-server.

1.2.2 Placeholders with defaults

The project's config files use one more idiom:

```
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIs...}
```

${NAME:default} means "use the environment variable if present, else the default." This is why a developer can run auth-service with zero setup, while production injects the real secret — and why in Chapter 16 the Kubernetes Secret only has to provide JWT_SECRET to take control of this value.

1.3 The project's file, annotated

Every service carries only a tiny application.yml; the interesting config lives on the config-server. Here is course-service's, complete:

```
spring:
  application:
    name: course-service              ①
  config:
    import: optional:configserver:http://localhost:8888   ②
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/          ③
```

① The service's identity. The config-server uses it to pick which file to serve (configurations/course-service.yml); Eureka uses it as the registration name; the gateway routes to it as lb://course-service. One string ties all three systems together.

② Bootstrap-time import of remote config. optional: means "start anyway if the config-server is down" — kind in dev, and debatable in production (a

service that boots with half its config may fail in stranger ways than one that refuses to boot). The localhost:8888 default is overridden by SPRING_CONFIG_IMPORT in Compose and Kubernetes alike.

③ Where Eureka lives — again a localhost default, again overridden per environment by EUREKA_CLIENT_SERVICEURL_DEFAULTZONE.

1.4 Profiles

A Spring *profile* is a named variant of configuration. spring.profiles.active=native on the config-server (Chapter 2) is the project's only profile use so far, but the mechanism generalizes: application-prod.yml is loaded on top of application.yml when the prod profile is active, letting one jar carry several personalities. Kubernetes reduces the need — the environment injects differences instead — which is why this project leans on env vars, not profile files, for per-environment change.

1.5 Actuator

spring-boot-starter-actuator adds operational endpoints under /actuator: /health (aggregated readiness of DB, disk, custom checks), /info, /metrics, /prometheus. Two facts matter now:

- The starter must be on the classpath. Exposure configuration without spring-boot-starter-actuator is inert — the project shipped exactly that mistake in four services and found it through a failing Kubernetes probe (Chapter 17).
- Endpoints must be *exposed* (management.endpoints.web.exposure.include) — the project exposes health,info,metrics,prometheus.
- Exposure is not *authorization*: if Spring Security protects /actuator/**, the endpoint exists but answers 401. This exact interaction decides which Kubernetes probes each service can use in Chapter 17 — dictionary-service permits actuator and probes /actuator/health; auth-service does not, and must be probed via /v3/api-docs instead.

1.6 Outside the project

Two contrasting examples of the same property machinery:

A Heroku-style twelve-factor app configures everything by environment variable — DATABASE_URL, PORT — and carries an application.yml of nothing but ${...} placeholders. Spring's resolution order makes that style native.

A bank running one artifact through five environments (dev/test/uat/pre-prod/prod) typically pairs a config-server backed by a git repository with profiles per environment: `course-service-uat.yml` lives in an audited repo, and promotion is a git merge. The project's classpath-native config-server (Chapter 2) is the single-developer version of the same shape.

PITFALL — MY OVERRIDE IS BEING IGNORED

Symptom: you set an environment variable but the service still uses the old value. Usual causes, in order of frequency: the variable name does not map to the key you think (check the relaxed-binding spelling — `SERVICEURL` in the Eureka key is one word, an irregular name that must be copied exactly); the value is also set by a *stronger* source (a command-line arg); or the service reads the key at build time rather than from the Environment. Diagnose with the actuator `/actuator/env` endpoint (when exposed), which lists every property source and which one won.

CHECK YOURSELF (answers: Appendix A)

1. A key is defined in the jar's `application.yml`, in the config-server's file for that service, and as an env var in the Deployment. Which value wins, and why is that ordering the right one for containers?
2. What three annotations does `@SpringBootApplication` combine, and which of them explains why deleting an unused dependency can change runtime behavior?
3. `${GOOGLE_CLIENT_ID:placeholder}` — what does this resolve to when the variable is unset, and what operational property does that give auth-service at startup?
4. Why does exposing an actuator endpoint not guarantee that a Kubernetes probe can use it?

Centralized Configuration: Config Server

2.1 The problem

Seven services each need database URLs, Kafka addresses, JWT settings, mail hosts. Copy that into seven application.yml files and every change becomes seven edits, seven rebuilds, and the guarantee that two copies will eventually disagree. The classic Spring Cloud answer: one small HTTP service whose only job is handing each application its configuration at startup.

2.2 How Spring Cloud Config works

The server exposes configuration over a plain HTTP contract:

```
GET /{application}/{profile}
GET /course-service/default      -> the config for course-service
```

The client side is the spring.config.import: configserver:... line you saw in Chapter 1. During startup — *before* most beans are created — the client fetches its document and merges it into the Environment as a property source that outranks the packaged YAML but loses to environment variables. The resolution order of Section 1.2 is doing all the work; the config-server is just one more source in the stack.

2.3 The project's server, annotated

```
server:
  port: 8888                    ①
spring:
  application:
    name: config-server
  profiles:
    active: native              ②
  cloud:
    config:
```

```
    server:
      native:
        search-locations: classpath:/configurations    ③
management:
  endpoints:
    web:
      exposure:
        include: health              ④
```

① 8888 is the conventional config-server port, the way 8761 is Eureka's. Convention saves documentation.
② The `native` profile means "serve files from a filesystem/classpath location" instead of the default git backend.
③ The served files are *inside the config-server's own jar*: `configurations/course-service.yml`, `auth-service.yml`, and so on — selected by matching file name to the requesting service's `spring.application.name`.
④ Only `health` is exposed — and this is the endpoint the Compose healthcheck and the Kubernetes probes rely on.

DECISION — NATIVE CLASSPATH VS. GIT BACKEND

The enterprise-standard backend is a git repository: config changes are commits — audited, reviewed, revertible without rebuilding anything. The cost is a second repository and credentials management. For a single-developer project whose config changes ship with code changes anyway, classpath-native keeps everything in one repo and one pipeline. The trade traded away: changing a value now requires rebuilding the config-server image. Know which side of that trade your employer sits on.

PITFALL — A SERVICE STARTS WITH DEFAULTS AND FAILS ODDLY

Symptom: auth-service boots but every request fails with connection errors to `localhost:5432`. Cause: the config-server was unreachable at startup, and `optional:configserver:` let the service boot from its packaged defaults — which point at localhost. Because the failure appears minutes later and far from the cause, it reads as a database problem. Check: the startup log prints whether remote config was fetched. The Compose file prevents this ordering problem with `depends_on: condition: service_healthy`; Kubernetes has no startup ordering (Chapter 15), so services must tolerate a late config-server by crashing and restarting until it appears.

2.4 Startup order and the bootstrap dance

The config-server creates a hidden sequencing constraint across the whole system: every other service wants it up first. You will meet the consequences three times in this book:

- Compose encodes the order explicitly with `depends_on` and a `curl`-based healthcheck (Chapter 12).
- Kubernetes refuses to encode startup order at all; the system converges by retry (Chapter 15).
- The Jenkins pipeline deploys all manifests in one `kubectl apply -k` and lets rollouts settle concurrently (Chapter 23).

2.5 Refreshing config without restart

Out of the box, a client reads its config once at startup. Spring Cloud offers `/actuator/refresh` (re-fetch on demand, per instance, applies to `@RefreshScope` beans) and Spring Cloud Bus (broadcast refresh over a message broker). This project uses neither: on Kubernetes a config change ships as a new image or env change, and the Deployment's rolling update (Chapter 15) restarts pods anyway. That is the modern answer to a question the config-server was originally designed around.

2.6 Outside the project

A typical git-backed enterprise setup, for contrast:

```
spring:
  cloud:
    config:
      server:
        git:
          uri: https://git.company.com/platform/config-repo
          default-label: main
          search-paths: '{application}'
```

Config lives per-service in folders of a dedicated repo; the label is a branch, so `default-label: release-42` pins an environment to a release. The second widespread pattern skips the config-server entirely: Kubernetes ConfigMaps and Secrets mounted as env vars — which is exactly where this project is heading (Chapters 16 and 26); the config-server survives here mostly as a working example of the classic stack.

CHECK YOURSELF (answers: Appendix A)

1. In which order do these sources win for `spring.datasource.url`: packaged YAML, config-server document, Deployment env var?
2. What does `profiles.active: native` change about where the server finds its documents, and what does the git backend buy that native cannot?
3. Why is `optional:` in `spring.config.import` a double-edged sword? Describe the failure mode it permits.
4. The config-server's own config cannot come from a config-server. Where must it live, and what general bootstrapping principle is that?

Service Discovery: Eureka

3.1 The problem

The gateway must forward `/api/courses` to course-service. To which address? In Compose it could hard-code `course-service:8082` — but then every new replica, every moved service, every environment is a config change. Service discovery replaces addresses with *names resolved at runtime*: services register themselves in a registry; clients ask the registry.

3.2 How Eureka works

Eureka (from Netflix, the ancestral Spring Cloud stack) is a REST service holding a registry of instances:

- **Registration.** At startup each client POSTs its identity: app name (`spring.application.name`), host, port, health URL.
- **Heartbeats.** Every 30 s (default) the client renews its lease. Miss a few renewals and the instance is evicted.
- **Client-side caching.** Consumers fetch the registry and cache it (default refresh 30 s), then load-balance *locally* across the instances they know. The registry being briefly down does not stop traffic — callers keep using their cache.
- **Self-preservation.** If too many heartbeats vanish at once, Eureka assumes a network problem rather than mass death and stops evicting. Fewer surprises during network blips; staler data during real outages.

The consumer side in this project is the gateway's `lb://course-service` URI (Chapter 4): "look this name up in the registry, pick an instance."

3.3 The server, and the one k8s-critical setting

The discovery-server module is Eureka with almost no configuration — a `@EnableEurekaServer` annotation and a port. The interesting configuration is on every *client*, and one line of it earns its own section because it decides whether the whole system works on Kubernetes:

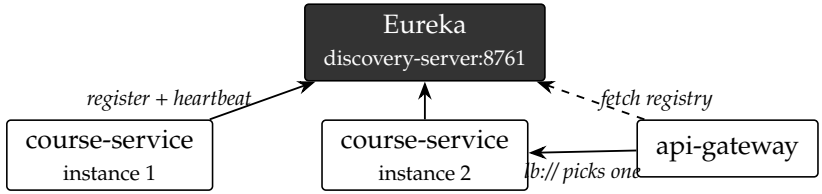

Figure 3.1: Registration, heartbeat, and client-side load balancing.

```
- name: EUREKA_INSTANCE_PREFER_IP_ADDRESS
  value: "true"
```

By default a client registers its *hostname*. On your laptop that is fine. Inside a Kubernetes pod, the hostname is the *pod name* — `course-service-7d9f6c-x2vlq` — which is not resolvable by other pods. Every registration would be an address no one can dial. Pod *IPs*, by contrast, are routable across the whole cluster. So every Deployment in `deploy/k8s/base/` sets this flag: register the IP, not the name.

PITFALL — EVERYTHING REGISTERS, NOTHING CAN CALL ANYTHING

Symptom: the Eureka dashboard shows all services UP, but every gateway-routed request fails with `UnknownHostException`. Cause: the registered hostnames are pod names (see above). This is the single most common Eureka-on-Kubernetes failure, and it is invisible until the first cross-service call.

DECISION — WHY EUREKA AT ALL, WHEN KUBERNETES HAS SERVICES

Kubernetes already gives every Service a DNS name (`course-service:8082`) with built-in load balancing — discovery is native to the platform. Eureka here is deliberate résumé-driven redundancy: the classic stack is still widespread in industry, and migrating *off* it (Chapter 26: replace `lb://` with plain Service DNS, delete two deployables) is itself the kind of modernization work enterprises pay for. Building both stacks teaches the migration.

3.4 Outside the project

Consul (HashiCorp) is the main non-JVM alternative: agent-based, language-agnostic, with health checks executed by the agent rather than self-reported heartbeats, plus a key-value store that can replace the config-server too. *Kubernetes-native discovery* is the other pole: no registry process at all — the

platform's controllers watch pod readiness and update Service endpoints; "registration" is simply a pod becoming Ready (Chapter 17 shows the machinery). Notice the pattern: the more the platform provides, the less the application must carry.

ON THIS HARDWARE

Eureka's conservative defaults (30 s heartbeat, 30 s client cache, eviction after 90 s) mean a crashed instance can receive traffic for up to a minute. On one replica per service that hardly matters — the caller would fail anyway — but it explains why rolling deployments in Chapter 23 wait on Kubernetes readiness, not on Eureka status.

CHECK YOURSELF (answers: Appendix A)

1. Trace what happens between course-service starting and the gateway successfully routing to it: which registrations, fetches, and caches are involved, and what is the worst-case delay?
2. Why does `prefer-ip-address` matter on Kubernetes but not on Compose? (What does each platform make resolvable?)
3. Self-preservation mode trades which property for which? Give a scenario where it helps and one where it hurts.
4. If Eureka were removed and the gateway pointed at `http://course-service:8082` directly, what Kubernetes machinery replaces each Eureka feature?

The API Gateway

4.1 The problem

Without a gateway, the frontend must know six service addresses, each service must implement its own authentication, and cross-cutting concerns (CORS, rate limits, tracing headers) are done six times or not at all. A gateway gives external clients *one* address and centralizes the edge concerns; behind it, services can trust their caller.

4.2 Spring Cloud Gateway's model

Three concepts compose every route:

Route a destination URI plus the conditions and rewrites below.
Predicates conditions deciding whether a request matches: path, method, header, host. First matching route wins.
Filters transformations applied on the way through: strip or add path segments, add headers, rate-limit, retry.

The gateway is built on the *reactive* stack (Project Reactor + Netty, `web-application-type: reactive` in its `application.yml`) rather than servlet Tomcat. A proxy spends its life waiting on other servers; an event-loop model holds thousands of in-flight requests without one thread each. Operationally this is why the gateway gets a smaller memory request than the MVC services in its Deployment (384 Mi vs. 512 Mi) and boots faster — which Chapter 17 exploits with a shorter startup probe.

4.3 The project's routes, annotated

From `configurations/api-gateway.yml` (abridged — the course-service block repeats for students, assignments, enrollments, submissions):

```
spring:
  cloud:
    gateway:
      routes:
```

```
        - id: auth-service
          uri: lb://auth-service          ①
          predicates:
            - Path=/api/auth/**           ②
          filters:
            - StripPrefix=2               ③
        - id: auth-oauth2
          uri: lb://auth-service
          predicates:
            - Path=/oauth2/**             ④
        - id: course-service-courses
          uri: lb://course-service
          predicates:
            - Path=/api/courses/**
          filters:
            - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecret...}     ⑤
```

① lb:// = "resolve this name via the discovery client and load-balance." The name is the target's spring.application.name. Swap lb:// for http:// and you have a static proxy with no Eureka involved.

② Path predicate. ** matches any depth.

③ StripPrefix=2 removes two segments: /api/auth/login arrives at auth-service as /login. This is why auth-service's SecurityConfig (Chapter 5) permits /login, not /api/auth/login — the service is unaware of its public prefix.

④ No StripPrefix here: Google's OAuth2 redirect URI is registered with the full path, so it must arrive at auth-service unchanged.

⑤ The gateway holds the JWT *validation* key. Same secret as auth-service, which *signs* — one shared secret, two roles, and in Kubernetes one Secret object mounted into both (Chapter 16).

PITFALL — 404 FROM THE SERVICE, 200 FROM CURL-ON-THE-POD

Symptom: calling /api/courses/list through the gateway returns 404, but the same endpoint answers when called directly on the service. Cause: prefix mismatch — either StripPrefix removed segments the controller expects, or the controller mapping includes a prefix the gateway already stripped. Debug by logging the *rewritten* path: enable spring.cloud.gateway debug logging and compare with the controller's @RequestMapping.

4.4 Two routers: the gateway and the Ingress

On Kubernetes this project has *two* L7 routers in series: Traefik (the Ingress controller, Chapter 18) at the cluster edge, then this gateway. The division of labor is deliberate:

- Traefik owns the *host*: TLS later, one public entry, path-prefix fan-out. It sends /api, /oauth2, /login/oauth2 to the gateway and will send / to the frontend.
- The gateway owns the *application edge*: Eureka resolution, JWT validation, per-route rewrites.

DECISION — NO SIDE DOORS

Milestone 3 had an Ingress sending /courses straight to course-service — useful scaffolding, but it bypassed JWT validation: an unauthenticated path to protected data. It was deleted the day the gateway was deployed. The rule it teaches: *every* external route must pass the component that enforces authentication, or the authentication is decorative.

4.5 Advanced corners

Worth knowing, not yet used here: RequestRateLimiter filter (token bucket backed by Redis), Retry with backoff for idempotent GETs, CircuitBreaker filter (Resilience4j) returning fallbacks when a service is down, and default filters applied to all routes. Each is a few YAML lines — the gateway is where such policies belong, because it is the one place every external request passes.

4.6 Outside the project

The same three-route idea in a Kubernetes-native gateway (no Eureka, Service DNS instead) — an nginx-ingress annotation style:

```
nginx.ingress.kubernetes.io/rewrite-target: /$2
# path: /api/courses(/|$)(.*)  ->  service: course-service
```

And the cloud-managed version: an AWS API Gateway or GCP API Gateway route table doing predicates/filters as vendor config, with JWT validation delegated to the provider (Cognito, IAP). The concepts transfer 1:1; only the syntax and the billing change.

CHECK YOURSELF (answers: Appendix A)

1. Follow `POST /api/auth/login` from browser to controller method on Kubernetes: name every hop and every path rewrite.
2. Why must the OAuth2 routes not strip their prefix while the API routes must?
3. The gateway validates JWTs; auth-service signs them. What breaks, and with what symptom, if the two ever hold different secrets?
4. Why is a reactive stack the right choice for a proxy but not automatically for course-service?

Security: JWT, OAuth2, and the Trusted Edge

Security is where this project is most instructive, because it uses three mechanisms that industry uses constantly — stateless JWTs, OAuth2 login, TOTP two-factor — and wires them through a gateway in the standard "trusted edge" pattern.

5.1 Spring Security in one page

Spring Security is a chain of servlet filters in front of your controllers. Each request passes through the chain; filters authenticate (who are you?) and the authorization rules decide (may you?). The project's auth-service chain, from its SecurityConfig:

```
http
  .csrf(AbstractHttpConfigurer::disable)                    ①
  .sessionManagement(s ->
      s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) ②
  .authorizeHttpRequests(auth -> auth
      .requestMatchers("/register", "/login", "/activate",
          "/refresh", "/verify-2fa", "/change-password",
          "/oauth2/**", "/login/oauth2/**",
          "/v3/api-docs/**", "/swagger-ui/**").permitAll()  ③
      .anyRequest().authenticated())
  .oauth2Login(o -> o.successHandler(oAuth2LoginSuccessHandler))
  .addFilterBefore(gatewayHeaderAuthFilter,
      UsernamePasswordAuthenticationFilter.class);          ④
```

① CSRF protection defends browser cookie sessions; a stateless JWT API has no cookie session to ride, so it is disabled. Disable it for that reason — not because a tutorial did.

② STATELESS: no HTTP session is created. Every request must carry its own proof of identity. This is the property that lets any replica answer any request — the scaling prerequisite.

③ The public paths. Note they are the *stripped* paths (/login, not /api/auth/login) — Chapter 4 explains why. Note also what is *absent*: /actuator/**. That absence

decides this service's Kubernetes probe strategy in Chapter 17.

④ The custom filter that trusts identity headers from the gateway — the subject of Section 5.3.

5.2 JWT: stateless identity

A JSON Web Token is three base64url parts, header.payload.signature. The payload carries *claims*: subject (user id), role, expiry. The signature is HMAC-SHA256 over header and payload with a shared secret — the 256-bit-minimum string both auth-service and the gateway hold.

The properties that matter:

- **Self-contained.** Validation needs no database call: check signature, check expiry, read claims. This is what lets the *gateway* authenticate without owning user data.
- **Readable.** Base64 is encoding, not encryption — anyone can decode the payload. Never put secrets in claims.
- **Irrevocable until expiry.** A signed token cannot be un-signed. Hence the two-token pattern: a short-lived *access token* (here 15 minutes) paired with a long-lived *refresh token* (7 days) that *is* checked against the database at /refresh — revocation happens there.

5.3 The trusted-edge pattern

The gateway's JwtAuthenticationFilter validates the token, then forwards the request with identity as plain headers:

```
ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
    .header(JwtConstants.HEADER_USER_ID, userId)
    .header(JwtConstants.HEADER_USER_ROLE, role)
    .build();
```

Downstream, each service's GatewayHeaderAuthFilter reads those headers and builds the Spring Security context from them — no re-validation. Services stay simple; cryptography happens once.

The pattern's obligation: those headers are only trustworthy if *every* path to a service goes through the gateway. A client who can reach course-service directly could forge X-User-Id. This is the deep reason the direct /courses Ingress was deleted (Chapter 4), and why in-cluster traffic is the remaining soft spot — any pod can still call course-service:8082 with invented headers. NetworkPolicies

or mTLS (service mesh) are the industrial hardening; Chapter 26 discusses when they earn their complexity.

5.4 OAuth2 login with Google

OAuth2's *authorization code* flow, as it runs here:

1. Browser hits `/oauth2/authorization/google` (via the gateway); Spring redirects to Google with the app's client-id and a `redirect_uri`.
2. The user consents at Google; Google redirects the browser to the `redirect_uri` — `/login/oauth2/code/google` — carrying a one-time *code*.
3. Auth-service exchanges code + client-secret for tokens server-to-server, reads the user's e-mail/profile, then the project's `OAuth2LoginSuccessHandler` takes over: find or create the local account and issue *our own* JWTs. Google authenticates; the application still authorizes with its own tokens.

PITFALL — REDIRECT_URI_MISMATCH

Symptom: Google shows "Error 400: redirect_uri_mismatch" instead of a consent screen. Cause: the URI the app sent is not byte-identical to one registered in the Google Cloud console. It changes per environment (`localhost:8080` in dev, `ts.local` in the cluster — which is why the Deployment overrides `...GOOGLE_REDIRECT_URI`), and every variant must be registered. The URI must reach auth-service *unstripped*, which is why the OAuth2 gateway routes carry no StripPrefix.

5.5 TOTP two-factor

Time-based One-Time Passwords (RFC 6238): at enrollment the server generates a secret, shows it as a QR code, and the authenticator app derives a 6-digit code from `HMAC(secret, time/30s)`. Login becomes two steps — password check issues a short-lived pending state, then `/verify-2fa` checks the code and issues the real tokens. Server and phone never communicate; they only share the secret and a clock.

5.6 Outside the project

The main industrial alternative moves token issuing out of your code entirely: an identity provider (Keycloak self-hosted, or Auth0/Cognito managed) issues to-

kens signed with an *asymmetric* keypair (RS256). Services validate with the *public* key fetched from a JWKS URL — no shared secret to distribute, key rotation built in:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: https://idp.company.com/realms/app/protocol/openid-connect/certs
```

Understanding the hand-rolled HS256 version — what this project builds — is precisely what makes that configuration meaningful: same claims, same validation, different key ceremony.

CHECK YOURSELF (answers: Appendix A)

1. Why can CSRF protection be disabled for a stateless JWT API but not for a cookie-session webapp?
2. An attacker steals an access token. How long is it useful, what cannot the server do about it meanwhile, and which endpoint finally stops the session?
3. In the trusted-edge pattern, state the exact invariant that must hold for `X-User-Id` to be trustworthy, and name two deployment mistakes that would break it.
4. In the Google flow, which secret never leaves the server, which value transits the browser, and why is the browser-visible one safe?
5. HS256 vs RS256: who can validate, who can sign, and why do identity providers choose RS256?

The Business Services and the Common Module

The previous chapters covered the plumbing services. This one maps the three that do the actual work, plus the shared `common` module — and, through them, the two ways microservices talk to each other.

6.1 course-service: the workhorse

Courses, students, assignments, enrollments, submissions — classic CRUD over Postgres (Chapter 7), plus file attachments stored in MinIO (Chapter 9). It *publishes* `ts.assignment.created` events and *listens* for `ts.user.activated` — the pattern to notice is that it reacts to another service's business event without ever calling that service.

6.2 auth-service: identity

Everything Chapter 5 described: registration with e-mail activation, password and Google login, TOTP 2FA, JWT issuing, refresh tokens. It publishes three events: `ts.user.registered`, `ts.user.activated`, `ts.user.admin-provisioned`.

6.3 dictionary-service: the small one

Vocabulary lookups over MongoDB (Chapter 8). Deliberately simple — it exists partly to prove the architecture accommodates a document store and partly as the low-stakes service to try things on first.

6.4 notification-service: the pure consumer

No REST API of consequence, no database. It subscribes to three topics and turns events into e-mails (Maildev in dev, a real SMTP relay later):

Topic	Producer	Mail sent
ts.user.registered	auth-service	activation link
ts.user.admin-provisioned	auth-service	credentials notice
ts.assignment.created	course-service	new-assignment notice

Because its *only* input is Kafka, it is the one service that cannot usefully run before a broker exists — which is why its Kubernetes manifest shipped with the Kafka milestone (Chapter 25) rather than with the other services.

6.5 Synchronous vs. asynchronous, in this codebase

The project uses both interaction styles, each where it belongs:

Synchronous (HTTP through the gateway) when the caller needs the answer to proceed: the frontend fetching a course list. Coupled in time — both sides must be up.

Asynchronous (events through Kafka) when the caller only needs the fact recorded: a registration happened; whoever cares can act. auth-service does not know notification-service exists. Decoupled in time — mail goes out late, not never, if the consumer is down.

The rule of thumb the codebase follows: *commands* that need answers are HTTP; *facts* that others may react to are events, named in the past tense (UserRegistered, AssignmentCreated).

6.6 The common module

```
com/ts/common/
  dto/        CourseDto, StudentDto, UserDto, ...
              UserRegisteredEvent, UserActivatedEvent,
              AssignmentCreatedEvent, AdminProvisionedUserEvent
  exception/  ApiException, ErrorResponse
  security/   JwtConstants     <- header names, claim names
```

Three kinds of sharing, with three different risk profiles:

- **Event records** are the *contract* between producer and consumer. Sharing the class guarantees both sides agree — at the price of lockstep deployment when the shape changes. (The industrial alternative — schema registry — appears in Chapter 11.)

- `JwtConstants` shares header and claim *names* between gateway and services. A typo'd header name would fail silently — requests simply arrive anonymous — so centralizing it is cheap insurance.
- **DTOs and exceptions** are convenience sharing: pleasant in a monorepo, and the first thing to duplicate-instead-of-share if services ever split into separate repositories, because shared convenience classes are how "independent" services end up released together.

DECISION — A SHARED MODULE AT ALL?

Microservice purists forbid shared code between services. The purist alternative — each service defines its own copy of every event — makes drift possible precisely where agreement is mandatory. This project draws the line at: share *contracts* (events, header names), think twice about sharing anything with behavior. `jib.skip=true` in common's POM marks it as the one module that is a library, not a deployable.

PITFALL — THE LOCKSTEP TRAP

Symptom: after adding a field to `UserRegisteredEvent` and deploying only auth-service, notification-service starts throwing deserialization errors on every message — and because it shares a consumer group, it keeps retrying the same broken message. Cause: the contract changed on one side only, and the topic now holds events in two shapes. Rules that prevent it: only *add* optional fields, never rename or remove; deploy consumers before producers; and configure the consumer's error handling (Chapter 11) so one poison message cannot stall the topic.

CHECK YOURSELF (answers: Appendix A)

1. For each pair, say whether HTTP or an event fits and why: (a) frontend needs a student list; (b) a submission was graded and the student should be told; (c) course-service must verify a JWT.
2. Why is notification-service the only service whose Kubernetes manifest waits for the Kafka milestone, when auth-service also uses Kafka?
3. Name one thing that belongs in `common` and one that should never be there, with the reasoning.
4. What deployment-order rule makes an event schema change safe, and what property must the change itself have?

Part II

Data

Relational Data: Postgres, JPA, Flyway

Two of the services own a PostgreSQL database each: auth-service owns `ts_auth`, course-service owns `ts_course`. This chapter covers the three layers between a `@Service` method and bytes on disk — JPA/Hibernate, the connection pool, and Flyway — and the architectural rule that there are *two* databases at all.

7.1 Database-per-service

DECISION — TWO POSTGRES SERVERS, NOT TWO SCHEMAS IN ONE

Each service owns its data exclusively; the only way to another service's data is through its API or its events. This forbids the classic failure mode of shared databases — service A quietly joining service B's tables, coupling their schemas forever. The project takes it further than required: not just separate databases but separate server processes, so even accidental cross-connection is impossible and each can be tuned, backed up, or broken independently. Cost on this box: about 256 MiB per extra server. In a cloud account the same isolation is usually two databases on one managed RDS instance — isolation by credentials rather than by process — because managed instances are billed per server.

7.2 JPA and Hibernate

JPA is the specification (annotations like `@Entity`, `@OneToMany`, the `EntityManager`); Hibernate is the implementation Spring Boot auto-configures. Spring Data JPA sits on top, deriving queries from repository method names (`findByCourseIdAndStudentId`) and generating the boilerplate.

The configuration from `configurations/course-service.yml`:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course   ①
    username: postgres
    password: root                                    ②
  jpa:
```

```
  hibernate:
    ddl-auto: none          ③
  show-sql: true            ④
flyway:
  enabled: true             ⑤
```

① The dev default. Compose overrides host to postgres-course; Kubernetes overrides identically via SPRING_DATASOURCE_URL. Note the k8s Service was deliberately *named* postgres-course so the override is the same string in both environments.
② Plaintext credentials are acceptable only because this file is the *dev default*; in the cluster, SPRING_DATASOURCE_USERNAME/PASSWORD arrive from a Kubernetes Secret (Chapter 16) and outrank these.
③ The most important line in the block — Section 7.3.
④ Every SQL statement is logged — a teaching tool and an N+1-query detector in dev; turn it off where log volume costs money.
⑤ Schema management belongs to Flyway, not Hibernate.

7.3 Why ddl-auto: none

Hibernate can generate schema from entities (ddl-auto: update), and every tutorial uses it. Production systems do not, because update is a one-way ratchet with no history: it adds columns but never drops or renames safely; two developers' databases diverge silently; there is no record of what changed when, and no way to review a schema change before it hits production.

7.4 Flyway: migrations as code

Flyway treats schema as an ordered series of immutable SQL scripts, applied exactly once each, tracked in a table (flyway_schema_history) inside the target database:

```
auth-service/src/main/resources/db/migration/
  V1__create_users.sql
  V2__create_refresh_tokens.sql
  V3__add_admin_user_invitations.sql
  V4__seed_default_admin.sql
```

At startup, Flyway compares the history table with the classpath scripts and applies the missing ones, in version order, before Hibernate ever initializes. The

rules that make it work: never edit an applied migration (each row stores a checksum — edits are detected and refused); only add new versions; and write migrations to be compatible with the *previous* application version when you deploy rolling (Chapter 15): add-then-migrate-then-remove, never rename in one step.

PITFALL — WORKS LOCALLY, CRASHLOOPBACKOFF IN THE CLUSTER

Symptom: a service boots on your machine but its pod restarts endlessly; logs show a Flyway checksum mismatch or a failed migration. Cause: the local database had been migrated by an older (or edited) script — your laptop's history table and the cluster's disagree. This is also why the Kubernetes startup probe (Chapter 17) targets an endpoint that only answers after the Spring context is up: "migrations applied" is part of what *ready* means. Fix by adding a corrective migration, never by editing history.

7.5 The connection pool

Opening a Postgres connection costs a TCP handshake, authentication, and a server process fork — far too much per request. HikariCP (Spring Boot's default pool) keeps a small set of open connections that requests borrow and return. Default maximum: 10.

ON THIS HARDWARE

Pool sizing folklore says "more connections, more throughput"; the reality on a 2-core server is the opposite — Postgres does work *per connection*, and the practical formula ($\approx$ cores $\times$ 2) suggests the default 10 is already generous for two services sharing two cores. If Postgres ever shows `too many clients`, lower the pools before raising `max_connections`.

7.6 Outside the project

A managed-cloud equivalent of this chapter's setup — AWS RDS Postgres with the same Flyway flow — changes only the URL and the trust model:

```
spring:
  datasource:
    url: jdbc:postgresql://app-db.xxxxx.eu-central-1.rds.amazonaws.com:5432/ts_course
    username: ${DB_USER}        # from AWS Secrets Manager
    password: ${DB_PASSWORD}
```

Backups, replication and failover become the provider's problem — which is why Chapter 19's warning about self-hosted state matters. The second industrial variant worth knowing: running Flyway as a separate CI step (`flyway migrate` against the target) instead of at application startup, so schema changes deploy independently of code and a slow migration cannot eat the startup probe's budget.

CHECK YOURSELF (answers: Appendix A)

1. Give two concrete failure modes of `ddl-auto: update` in a team setting that `none` + Flyway prevents.
2. Why must an applied migration never be edited, and what does Flyway use to detect it?
3. You need to rename a column without downtime under rolling deployments. Sketch the multi-release sequence.
4. Defend and attack the two-servers decision: one argument each way, and the setting in which each wins.

Document Data: MongoDB

8.1 The model

MongoDB stores *documents* — JSON-like structures (BSON) — in *collections*, without a fixed schema. Where Postgres normalizes a dictionary entry into rows across words, definitions, examples tables joined by keys, Mongo stores the whole entry as one nested document:

```
{ "word": "ubiquitous",
  "phonetic": "yoo-BIK-wi-tuhs",
  "meanings": [
    { "partOfSpeech": "adjective",
      "definitions": [
        { "definition": "present everywhere",
          "example": "smartphones are ubiquitous" } ] } ] }
```

DECISION — WHY THE DICTIONARY IS THE MONGO SERVICE

A dictionary entry is read as a unit, written as a unit, and naturally nested — the document model's best case. There are no cross-entry transactions and no joins to lose. Courses and enrollments, by contrast, are genuinely relational (a submission references a student *and* an assignment; consistency between them matters), so they stay in Postgres. "Polyglot persistence" is this: pick per service, by data shape — not one blessed database for everything, and not novelty for its own sake.

8.2 Spring Data MongoDB

The programming model mirrors JPA closely enough that the dictionary service is a good Rosetta stone: @Document instead of @Entity, MongoRepository instead of JpaRepository, derived query methods identical in spirit. What disappears: Flyway and schema DDL (collections spring into being on first write), the dialect, and the pool tuning (the driver manages its own connection pool). What appears: the single connection URI.

8.3 The configuration — all of it

Dictionary-service's entire data config is one line, plus its cluster override:

```
# configurations/dictionary-service.yml  (dev default)
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/ts_dictionary

# deploy/k8s/base/dictionary-service.yaml  (the override)
- name: SPRING_DATA_MONGODB_URI
  value: "mongodb://mongodb:27017/ts_dictionary"
```

The URI carries host, port, database, and (when enabled) credentials and options — one string to override per environment, which is exactly what the relaxed-binding machinery of Chapter 1 is for.

8.4 Schemaless cuts both ways

No migrations does not mean no schema — it means the schema lives *implicitly in the code*. Two documents written by two versions of the application can have different shapes in the same collection, and nothing but your reading code will notice. Disciplines that replace Flyway: keep document classes backward-readable (new fields optional), write data-fixing scripts for real shape changes, and validate at the boundary. Mongo can also enforce JSON Schema per collection — opt-in, and unused here.

PITFALL — THE COLLECTION THAT WAS NEVER CREATED

Symptom: the service starts, queries run, everything returns empty — no error anywhere. Cause: the URI pointed at a fresh database (a typo, or a new PVC in the cluster); Mongo silently created an empty `ts_dictionary` on first touch. Postgres would have failed loudly (`database does not exist`); Mongo's create-on-write turns a wrong address into empty results. When a Mongo-backed service "loses" data, check *which database it is talking to* before anything else: `kubectl exec` into the pod's node and run `mongosh --eval 'show dbs'`.

8.5 Operations in this cluster

The manifest (`deploy/k8s/base/mongodb.yaml`) follows the same StatefulSet pattern as Postgres (Chapter 15 explains the kind; 19 the storage), with two Mongo-

specific notes, both from its comments: the readiness probe execs `mongosh –eval "db.adminCommand('ping')"` — "accepting commands," not merely "port open" — and it runs without authentication, acceptable only because it is a ClusterIP service with no Ingress: unreachable from outside the cluster. The moment that assumption changes (an exposed NodePort, multi-tenancy), auth comes first.

8.6 Outside the project

The managed version is MongoDB Atlas; the URI grows credentials and TLS and nothing else changes:

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://app:${MONGO_PWD}@cluster0.abc.mongodb.net/ts_dictionary?
      ↪ retryWrites=true
```

For contrast, DynamoDB shows a document store with the opposite philosophy: schemaless like Mongo, but access patterns must be designed into the table (partition keys, no ad-hoc queries) — a reminder that "NoSQL" is a family of trade-offs, not one thing.

CHECK YOURSELF (answers: Appendix A)

1. Which properties of the dictionary data make it a good document case, and which properties of enrollments make them a bad one?
2. "Schemaless" moves schema enforcement from the database to where? Name two disciplines that replace migrations.
3. Why does a wrong database name fail loudly in Postgres but silently in Mongo, and which failure do you prefer in production?
4. The Mongo pod has no password while Postgres does, in the same cluster. Reconstruct the reasoning and state the condition that invalidates it.

Object Storage: MinIO and the S3 Model

9.1 The problem

Assignment submissions are files — PDFs, images, archives. Files in a relational database bloat backups and stream badly; files on a pod's local disk vanish with the pod. The industry's answer is *object storage*: an HTTP service storing immutable blobs in named *buckets* under string *keys*, metadata included, no directories (the / in a key is naming convention, not structure). Amazon S3 defined the API; MinIO is a self-hostable server speaking the same API — so course-service uses the standard AWS SDK and cannot tell the difference.

9.2 The configuration, annotated

From configurations/course-service.yml:

```
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000}          ①
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000} ②
    access-key: ${APP_S3_ACCESS_KEY:minioadmin}                 ③
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PRESIGNED_EXPIRY_MIN:15} ④
```

① Where *the service* talks to MinIO. In the cluster: http://minio:9000.
② Where *the browser* talks to MinIO — the subject of the pitfall below.
③ Credentials with dev defaults; the cluster injects real ones from the minio-app Secret. app.s3.* is custom (@ConfigurationProperties) config, not Spring-managed — the same override machinery applies regardless.
④ How long a presigned link stays valid.

9.3 Presigned URLs: the pattern worth knowing

Naive file download routes bytes *through* the service: browser → service → MinIO and back, tying up an application thread per transfer. The S3-native alternative: the service uses its credentials to *sign a URL* — bucket, key, expiry,

HMAC signature — and hands the URL to the browser, which downloads *directly* from storage. The service spends microseconds; storage does the streaming; the link self-expires. The same trick works for uploads.

PITFALL — THE TWO-ENDPOINTS PROBLEM

Symptom: uploads work, but clicking a download link fails with `ERR_NAME_NOT_RESOLVED` on `minio:9000`. Cause: a presigned URL embeds the endpoint *it was signed against*. The service signs against `minio:9000` — a name only resolvable inside the cluster network — and the browser lives outside it. Hence two configured endpoints: sign download links against `public-endpoint`, talk to storage over `endpoint`. Every containerized S3 setup meets this bug once; some meet it again with the signature itself, because the signature covers the host — you cannot sign against one host and rewrite the URL to another afterwards.

ON THIS HARDWARE

MinIO here is a single container with a single PVC — fine for a homelab's submissions, nothing like production MinIO (multi-node erasure coding) or S3's eleven nines of durability. The interesting part is that course-service cannot tell: the same SDK calls would hit real S3 with only the endpoint and credentials changed. That is the point of speaking a standard API.

9.4 Outside the project

The same service on AWS proper: set `endpoint` and `public-endpoint` to the real S3 URL (or leave the SDK's default), replace static keys with an IAM role assumed by the pod (IRSA on EKS — no long-lived credentials at all), and turn on bucket versioning and lifecycle rules (auto-expire old submissions to cheaper storage). The concepts — buckets, keys, presigning, the identity that signs — transfer unchanged; what changes is who manages durability and how identity is proven.

CHECK YOURSELF (answers: Appendix A)

1. Why do presigned URLs scale better than proxying downloads through the service, and what new failure mode do they introduce?
2. Explain why `endpoint` and `public-endpoint` must differ inside the cluster, and why post-signing URL rewriting is not a fix.
3. A presigned URL leaks in a chat log. What limits the damage, by design?

4. What changes — and what does not — when this code moves from MinIO to real S3?

Part III

Messaging

Kafka: The Core Model

Kafka is the technology in this stack most worth learning deeply: it appears in a large fraction of backend job descriptions, and its model is genuinely different from a message queue's. This chapter is platform-independent Kafka; Chapter 11 is the Spring side.

10.1 A log, not a queue

Kafka's core abstraction is an *append-only log*. A *topic* (e.g. `ts.user.registered`) is split into *partitions*; each partition is an ordered, immutable sequence of *records*, each at a numbered *offset*. Records are not deleted when read — they expire by retention policy (this project: 168 hours). Reading is not destructive; a consumer just remembers *how far it got*.

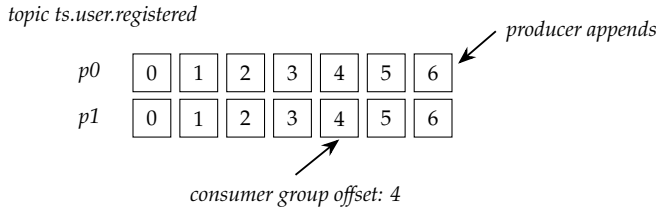

Figure 10.1: Two partitions; producers append at the head, consumer groups remember an offset per partition.

The consequences, each of which a queue lacks:

- **Replayable.** A new consumer can start from offset 0 and see history. Deploy a new analytics service next year; it can process last week's registrations.
- **Multi-reader.** Notification-service and course-service both read auth events independently, at their own pace, without stealing from each other — each *consumer group* keeps its own offsets.
- **Ordered per partition.** Records with the same *key* hash to the same partition and stay in order. This is why the publishers in this project key by `userId`: all events about one user arrive in sequence. Across partitions there is no order — design around per-key order only.

10.2 Consumer groups

A consumer group is a named team splitting work: each partition is assigned to exactly one consumer *within the group*. Two consumers in group `notification-group` on a 2-partition topic get one partition each; a third sits idle — *partitions are the unit of parallelism*. When a consumer joins or dies, the group *rebalances*: partitions are reassigned and members briefly pause. Different groups are invisible to each other, which is how the same event fans out to both `notification-group` and `course-group`.

10.3 Brokers, replication, KRaft

A production cluster runs several *brokers*; each partition has a leader on one broker and *replicas* on others (`replication-factor`, RF). Producers write to the leader; `acks=all` waits until in-sync replicas have it. Metadata coordination historically required ZooKeeper, a second distributed system; modern Kafka replaces it with *KRaft* — brokers running an internal Raft quorum. This project runs KRaft with one node playing both `broker` and `controller` roles, RF = 1: fine for a homelab, zero fault tolerance — one disk failure loses the log.

10.4 The advertised-listeners maze

The single most-Googled Kafka error. From `docker-compose.yml`:

```
KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092
    ↪ ①
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://
    ↪ localhost:9092 ②
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093                ③
```

① *Listeners*: the sockets the broker opens. Three: one for clients inside the Docker network, one for the KRaft controller, one published to the host.

② *Advertised* listeners: the addresses the broker *tells clients to use*. This is the trap: a Kafka client's first request is "where are the partition leaders?", and the broker answers with these strings. Containers must be told `kafka:19092` (resolvable on the Docker network); tools on your laptop must be told `localhost:9092`. One broker, two audiences, two advertised addresses.

③ KRaft quorum of one: node 1 is the whole election.

PITFALL — CONNECTS, THEN TIMES OUT

Symptom: a client connects to the bootstrap address successfully, then every send/poll fails with `Connection to node -1 could not be established` or hangs. Cause: bootstrap worked, but the *metadata answer* advertised an address the client cannot reach (e.g. your laptop was told `kafka:19092`). The bootstrap address only starts the conversation; the advertised listeners carry all subsequent traffic. Diagnose by asking what the broker advertises — e.g. `kcat -L -b localhost:9092` — and checking those exact strings resolve from where the client runs.

10.5 Delivery semantics

Kafka's default is *at-least-once*: a producer retry after a lost ack, or a consumer crash after processing but before committing its offset, both cause duplicates — not loss. Design consumers to be *idempotent* (processing the same event twice is harmless): notification-service sending one duplicate mail is acceptable; a payment service would instead track processed event IDs. Exactly-once exists (idempotent producers + transactions) but costs complexity; know that at-least-once + idempotent consumer is the industry's bread and butter.

10.6 Outside the project

Managed Kafka (AWS MSK, Confluent Cloud) removes broker operations and sells the same protocol — your clients change a bootstrap string and gain SASL/TLS. The other names in this space solve different problems: RabbitMQ is a smart-routing *queue* (work distribution, per-message acks, no replay); Redpanda reimplements the Kafka protocol without the JVM. When Chapter 25 moves this broker into the cluster under Strimzi, nothing in this chapter changes — Strimzi is *operations* for the same Kafka.

CHECK YOURSELF (answers: Appendix A)

1. Registration events for one user must be processed in order, but scaling matters. What do you key by, how many partitions, and what is the parallelism limit?
2. Two groups, `notification-group` and `course-group`, read `ts.user.activated`. Explain how both get every event.
3. Distinguish `LISTENERS` from `ADVERTISED_LISTENERS`, and explain

why the bug they cause appears *after* a successful connection.

4. Why does at-least-once delivery push idempotency onto the consumer, and what would notification-service's idempotency strategy reasonably be?

Spring Kafka: Producers, Listeners, Poison Pills

11.1 Producing

Publishing an event in this project is two moving parts: a KafkaTemplate and configuration choosing serializers. From auth-service:

```
kafkaTemplate.send("ts.user.registered", event.userId(), event);
//                 topic                key              value
```

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

The key (userId) is a string; the value is the event record serialized to JSON. Keying by user id is the ordering decision from Chapter 10 made concrete. send() is asynchronous — it returns a future; the message sits in a client-side buffer until the broker acks. Two operational consequences: a service can *start* without a broker (sends fail later, per call — which is why auth-service could deploy to the cluster before Kafka), and a service that crashes right after send() returns may lose the buffered message — the outbox pattern below exists for when that matters.

11.2 Consuming

```
@KafkaListener(topics = "ts.user.registered",
               groupId = "notification-group")
public void on(UserRegisteredEvent event) {
    mailService.sendActivation(event);
}
```

Spring runs a polling loop per listener container, deserializes each record, invokes the method, and commits offsets after successful processing — at-least-

once out of the box. The consumer half of the YAML holds this chapter's most production-relevant lines:

```
spring:
  kafka:
    consumer:
      group-id: course-group
      auto-offset-reset: earliest                    (1)
      key-deserializer: ...ErrorHandlingDeserializer (2)
      value-deserializer: ...ErrorHandlingDeserializer
      properties:
        spring.deserializer.value.delegate.class: ...JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto  (3)
```

(1) Where a *brand-new* group starts when it has no committed offset: `earliest` = from the beginning of retention; the default `latest` = only new messages. `earliest` means a consumer deployed late still processes events published before it existed — the right choice when events are facts to act on. Irrelevant once the group has offsets.
(2) The poison-pill defense — next section.
(3) JSON deserialization instantiates classes named in message headers; trusting only `com.ts.common.dto` stops a malicious or buggy producer from making the consumer instantiate arbitrary classes. A security boundary, not boilerplate.

11.3 The poison pill

A *poison pill* is a message that fails deserialization forever — a producer bug, a renamed field, a corrupted payload. Without defenses the listener throws *before* the message can be skipped, the offset never advances, and the container retries the same record endlessly: the partition is stalled and the logs fill at full speed.

`ErrorHandlingDeserializer` wraps the real deserializer: on failure it emits a typed error instead of throwing inside the poll loop, letting Spring's error handler skip past the record (optionally after retries, optionally publishing it to a dead-letter topic). The composition — error-handler outside, JSON delegate inside — is the standard production pattern, and it is why the config names *two* deserializer classes per side.

PITFALL — ONE BAD MESSAGE, TOTAL SILENCE

Symptom: notification mails stop entirely; the consumer logs the same deserialization stack trace thousands of times. Cause: a poison pill on one partition

with no error-handling deserializer — the loop can neither process nor pass it. This failure hides in systems that never tested a malformed message. Verify your defense once, deliberately: publish garbage to a dev topic and watch it get skipped.

11.4 Advanced corners

Dead-letter topics. DefaultErrorHandler + DeadLetterPublishingRecordRecoverer: after N failed attempts, the record lands on <topic>.DLT for inspection and replay — failure handling with an audit trail.

The outbox pattern. "Save the user *and* publish the event" is two systems and cannot be atomic directly. The outbox: write the event into an outbox table in the *same database transaction* as the user row; a relay (or Debezium reading the WAL) publishes from that table. If auth-service ever must not lose a registration event, this is the answer.

Schema registry. Chapter 6's shared event classes solve producer/consumer agreement inside one repo. At organization scale the contract moves into a schema registry (Avro/Protobuf schemas, checked for compatibility at publish time) — the same idea, enforced by infrastructure instead of a shared jar.

CHECK YOURSELF (answers: Appendix A)

1. Why can auth-service start when Kafka is down, and at what moment does a Kafka outage become visible to it?
2. A new consumer group is deployed today. Explain what auto-offset-reset: earliest makes it do, and when this setting stops having any effect for that group.
3. Walk through the poison-pill failure without ErrorHandlingDeserializer, then with it. Where does the bad record end up in a full production setup?
4. Why is spring.json.trusted.packages a security control?
5. When is the outbox pattern necessary, and what pair of writes does it make effectively atomic?

Part IV

Containers

Docker: Images, Containers, Compose

12.1 What a container actually is

A container is not a small virtual machine. It is an ordinary Linux process, isolated by two kernel features: *namespaces* (its own view of processes, network, filesystem mounts, hostname) and *cgroups* (limits on CPU and memory). No guest OS, no hypervisor — which is why a container starts in milliseconds and why the JVM inside one must be told about its memory limit (Chapter 17's `MaxRAMPercentage`) — from inside, it can see the *host's* RAM.

An *image* is the packaged filesystem plus metadata (entrypoint, env, exposed ports), built as a stack of immutable *layers*. Layers are content-addressed and shared: fifty images `FROM eclipse-temurin:25-jre-alpine` store the JRE layer once. A *registry* (Docker Hub, GHCR, the project's `registry:2`) stores images; `docker push`/`pull` move only layers the other side lacks.

12.2 The project's Dockerfile, annotated

Every service carries the same five lines:

```
FROM eclipse-temurin:25-jre-alpine    ①
WORKDIR /app
COPY target/*.jar app.jar             ②
EXPOSE 8082                           ③
ENTRYPOINT ["java", "-jar", "app.jar"]④
```

① Base image: a JRE (not JDK — no compiler needed at runtime) on Alpine Linux. Smaller base = smaller attack surface and faster pulls.

② Copies the Maven-built fat jar. Subtlety: this Dockerfile requires `mvn package` to have run *first* — the build happens outside. The consequence: one `COPY` of a ~60 MB jar makes one big layer, so *every* build re-pushes everything even for a one-line change. Jib (Chapter 13) exists to fix exactly this.

③ Documentation, not action — it opens nothing. Publishing ports happens at run time.

④ Exec-form entrypoint: java is PID 1, receives signals directly, so `docker stop` triggers a clean Spring shutdown.

12.3 Networking and Compose

Containers on a Docker *bridge network* reach each other by *service name* — Docker runs a DNS server answering `postgres-course`, `kafka`, `minio`. `localhost`, inside a container, is *that container* — the number-one beginner trap, and the reason every dev-default URL in the configs needed a per-environment override. Ports are only reachable from the host when *published* (`-p 5434:5432`).

Compose turns a fleet of `docker run` commands into one declarative file — the project's `docker-compose.yml` is the whole dev stack: four datastores, Kafka, Zipkin, Maildev, and the seven services, with three idioms worth naming:

```
discovery-server:
  depends_on:
    config-server:
      condition: service_healthy     ①
config-server:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"] ②
postgres-course:
  ports:
    - "5434:5432"                    ③
  volumes:
    - postgres-course-data:/var/lib/postgresql/data
```

①+② encode startup order — and not merely "started" but "answering its health endpoint," which is what makes the ordering real. ③ maps distinct host ports (5433, 5434) onto the two Postgres containers' identical internal 5432 — names are per-network, host ports are global. The named volume keeps data across container recreation; Chapter 19 is the Kubernetes version of the same need.

DECISION — WHY COMPOSE SURVIVES ALONGSIDE KUBERNETES

The cluster is the deployment target, but Compose remains the inner loop: one command, laptop-local, debug ports mapped (every service opens a JDWP port in Compose — attach the IDE debugger to 5082 and step through course-service). Kubernetes buys self-healing and rolling deploys at the cost of ceremony; a developer iterating on business logic wants neither cost. Two environments, one image contract between them.

PITFALL — WORKS IN COMPOSE, DIES IN KUBERNETES

Symptom: the stack is green locally; the same image crash-loops in the cluster. The usual suspects, all environmental: a URL still pointing at a Compose service name that has a different Service name in k8s; a `depends_on` ordering the cluster does not honor (Kubernetes has no startup ordering — Chapter 15); or memory limits present in the cluster but absent locally, turning a fat JVM into an OOMKill. The image is identical — diff the *environment*, not the code: `docker inspect` vs. `kubectl describe pod`.

CHECK YOURSELF (answers: Appendix A)

1. A container and a VM both isolate. Name two mechanisms the container uses, and two consequences of not having a guest kernel.
2. Why does `localhost:5432` fail inside a container that works on your laptop, and what two different fixes do Compose and Kubernetes offer?
3. Both Postgres containers listen on 5432 internally without conflict — explain. Where do conflicts become possible again?
4. What makes `condition: service_healthy` stronger than plain `depends_on`, and which project file supplies the health signal it waits for?

Images Without a Daemon: Jib and the Local Registry

13.1 Why not just docker build in CI?

The Dockerfiles work. Three reasons the pipeline builds images differently:

1. **CI needs no Docker daemon.** docker build requires a running daemon; giving Jenkins one means docker-in-docker or mounting the host's Docker socket — which is root on the host, handed to every build. Jib builds and pushes images in pure Java: no daemon, no privilege.
2. **Layering that matches how code changes.** The Dockerfile copies one fat jar: one huge layer, fully re-pushed per build. Jib splits the application into layers by volatility — dependencies / resources / classes — so a one-line code change re-pushes only the small classes layer. On two cores and Wi-Fi this is the difference between seconds and minutes per service.
3. **OpenShift-ready by accident.** Jib images run as a non-root user with group-0 file permissions — exactly what OpenShift's arbitrary-UID security model demands. Portability for free.

13.2 The configuration, annotated

Jib is configured once in the parent pom.xml's pluginManagement; each runnable module opts in with a bare <plugin> reference, and common opts *out* with jib.skip=true:

```
<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>jib-maven-plugin</artifactId>
  <version>3.4.4</version>
  <configuration>
    <from><image>eclipse-temurin:25-jre-alpine</image></from>  ①
    <to><image>${docker.registry}/${project.artifactId}:${project.version}</image></to>
      ↪②
    <allowInsecureRegistries>true</allowInsecureRegistries>    ③
    <container>
      <mainClass>${start-class}</mainClass>                    ④
      <creationTime>USE_CURRENT_TIMESTAMP</creationTime>       ⑤
```

```
    </container>
  </configuration>
</plugin>
```

① Same base as the Dockerfiles — runtime behavior unchanged, only the packaging path differs.
② Resolves to localhost:5000/course-service:1.0.0-SNAPSHOT. The name is written *from the cluster's point of view* — Section 13.3.
③ The local registry is plain HTTP; Jib refuses insecure pushes unless told. Every tool in this chain (Jib, containerd) makes the same secure-by-default choice and needs the same explicit opt-out.
④ Set explicitly because Jib's bundled bytecode scanner does not yet parse Java 25 class files — a real example of bleeding-edge Java taxing the toolchain.
⑤ Jib defaults to epoch-0 timestamps for reproducible builds; this trades that away so docker images shows honest build times.

13.3 The registry, and why localhost:5000

```
docker run -d --restart=unless-stopped --name registry \
  -p 5000:5000 -v registry-data:/var/lib/registry registry:2
```

The non-obvious fact that forces this architecture: **k3s does not see Docker's images.** k3s runs its own containerd with its own image store; docker build on the host puts images somewhere k3s never looks. A registry is the honest interface between builder and cluster — which the pipeline wants anyway.

An image name is not just a name — it is *where to pull from*: localhost:5000/course-service means "pull from localhost:5000." Since the puller is the k3s node itself, and pushing Jenkins shares the host's network namespace (Chapter 22), the same string is valid for both — and localhost can never drift the way a LAN IP or hostname can. Two configuration hooks make it work:

```
# /etc/rancher/k3s/registries.yaml -- containerd assumes HTTPS otherwise
mirrors:
  "localhost:5000":
    endpoint: ["http://localhost:5000"]
```

PITFALL — HTTP: SERVER GAVE HTTP RESPONSE TO HTTPS CLIENT

The signature error of this chapter. Both Jib (pushing) and containerd (pulling) default to TLS; `registry:2` speaks plain HTTP. If a pod sticks in `ImagePullBackOff` with this text in `kubectl describe pod`, the `registries.yaml` entry is missing — and note containerd reads it *only at startup*: `systemctl restart k3s` after editing.

DECISION — LOCAL REGISTRY VS. GHCR

GitHub Container Registry would make images pullable anywhere and add vulnerability scanning — for the price of pushing hundreds of megabytes up residential Wi-Fi on every build. A LAN-local registry keeps the build loop at disk speed. The decision flips the moment a second machine must pull images; nothing in the manifests would change except the image prefix.

13.4 Tags, digests, and the SNAPSHOT trap

A *tag* (`:1.0.0-SNAPSHOT`, `:a1b2c3d`) is a movable pointer; a *digest* (`@sha256:...`) is the immutable content hash. The pipeline pushes every build twice-labelled: the SNAPSHOT tag (which the manifests reference) and the git-SHA tag (which deploys pin — Chapter 23). Kubernetes caches images by tag; a re-pushed SNAPSHOT would be silently ignored on the node — which is why every Deployment sets `imagePullPolicy: Always`: re-check the registry each start; if the digest matches, the check costs one manifest fetch.

CHECK YOURSELF (answers: Appendix A)

1. Name the three CI problems Jib solves over daemon-based builds, and which one is a security issue.
2. Why must the image name be written from the *cluster's* viewpoint, and what breaks if Jenkins and k3s disagree about what `localhost:5000` means?
3. A colleague suggests `docker save | k3s ctr images import` instead of a registry. What does that cost per deploy, and which pipeline stage stops working naturally?
4. Explain how a re-pushed `:1.0.0-SNAPSHOT` plus default pull policy produces "I deployed but nothing changed," and both fixes.

Part V

Kubernetes

Kubernetes: Cluster Anatomy and the Reconciliation Loop

14.1 The one idea everything else hangs on

You do not tell Kubernetes to *do* things; you tell it what should *be true*. You write *desired state* ("a Deployment named course-service with 1 replica of image X") into its database via the API server; *controllers* run reconciliation loops forever comparing desired with actual and acting on the difference. Delete a pod: its controller notices actual ≠ desired and makes another. Nobody replays your commands — the state converges.

This explains behaviors that otherwise seem strange:

- `kubectl apply` is a merge into desired state, not a script run — apply the same file twice, nothing happens the second time (idempotence — the property the pipeline leans on in Chapter 23).
- There is no "startup order." Everything starts at once; services that need absent dependencies crash and are restarted until the world is ready. Crash-looping *during* convergence is normal operation, not failure.
- Deleting from a manifest file does not delete from the cluster — apply adds and updates, it does not prune. Removing the old course-service Ingress took an explicit `kubectl delete`.

14.2 The moving parts

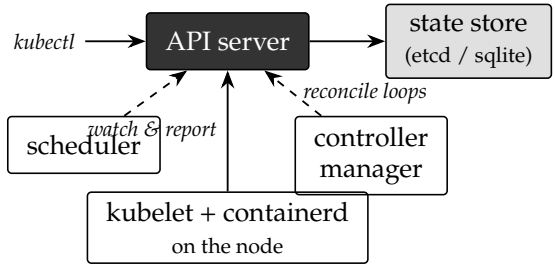

Figure 14.1: Everything talks only to the API server.

API server the single front door; kubectl is a REST client for it. Authentication, authorization (Chapter 22's RBAC), and validation happen here.

State store the desired-state database (etcd in standard clusters; k3s embeds sqlite on a single node).

Scheduler assigns unscheduled pods to nodes, honoring the resource *requests* of Chapter 17.

Controller manager the reconciliation loops: Deployment → ReplicaSet → pods, endpoints, and dozens more.

kubelet the node agent: watches for pods assigned to its node, drives containerd to run them, executes the probes, reports status.

k3s packages all of this into one binary — a CNCF-certified Kubernetes, minus a VM's overhead, plus batteries: Traefik as Ingress controller, local-path storage, containerd. Every manifest in this book would apply unchanged to EKS or OpenShift; that certification is why learning on k3s transfers.

14.3 Objects, labels, namespaces

Every object is a YAML document with the same skeleton — apiVersion, kind, metadata (name, namespace, labels), spec (desired), status (observed, written by controllers). Two organizing mechanisms:

Namespaces partition the cluster; this project puts everything in ts, keeping it separable from kube-system and making RBAC scopeable — Jenkins' permissions stop at the namespace edge (Chapter 22).

Labels arbitrary key/value pairs; *selectors* query them. Every wiring in Part V is a label selector: a Service finds pods by label, a Deployment owns pods by label. The kustomization stamps app.kubernetes.io/part-of: teacher-supporter onto everything so one selector can list — or delete — the whole application.

14.4 Getting kubectl to the cluster

kubectl reads ~/.kube/config: cluster URL, CA certificate, and a credential. From Windows, this project reaches https://100.85.66.43:6443 over Tailscale — which required installing k3s with –tls-san 100.85.66.43, because the API server's certificate is only valid for the names baked in at install time.

HANDS-ON — FEEL THE LOOP

```
kubectl -n ts get pods
kubectl -n ts delete pod -l app=course-service
```

```
kubectl -n ts get pods -w
```

Delete the pod, then watch: a replacement appears within seconds, created not by your command but by the ReplicaSet controller noticing a missing replica. You never ordered a restart — you made actual state diverge from desired, and the loop closed the gap.

PITFALL — CERTIFICATE IS VALID FOR... NOT 100.85.66.43

Symptom: `kubectl` from Windows fails TLS validation while working fine on the server. Cause: the API server certificate lacks the tailnet address as a SAN — it was installed without `–tls-san`. Fix at install time is one flag; fix afterwards means deleting the server TLS directory and restarting k3s. A concrete instance of a general rule: certificates bind to *names*, so decide every name a server will be called *before* issuing.

CHECK YOURSELF (answers: Appendix A)

1. Explain "declarative": what do you write, who acts on it, and why does applying the same manifest twice change nothing?
2. Why is there deliberately no `depends_on` in Kubernetes, and what replaces startup ordering during convergence?
3. Trace `kubectl -n ts delete pod ...` through the components: who receives it, who notices, who acts, on which machine?
4. Why did removing the old Ingress from git not remove it from the cluster, and what two commands would each have removed it?

Workloads: Pods, Deployments, StatefulSets

15.1 Pods

The pod is the unit Kubernetes schedules: one or more containers sharing a network namespace (one IP, `localhost` between them) and volumes. Almost every pod here is a single container; the multi-container patterns (sidecars for proxies or log shippers) matter when you meet a service mesh. Pods are *mortal by design* — they are never healed, only replaced, and their IP dies with them. Everything else in Part V — controllers, Services, PVCs — exists to build durable behavior out of disposable pods.

15.2 Deployments and the rollout

You never create bare pods. A *Deployment* declares "N replicas of this pod template"; it manages a *ReplicaSet* (the counter), which manages the pods. The three-layer indirection is what makes updates elegant: change the template (a new image tag), and the Deployment creates a *new* ReplicaSet, scaling it up while the old scales down:

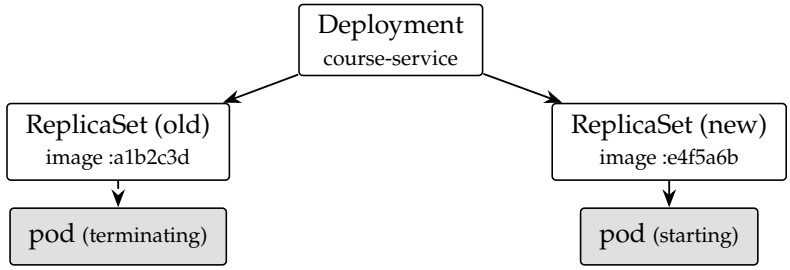

Figure 15.1: A rolling update mid-flight: two ReplicaSets, traffic shifting.

The rollout only proceeds as new pods become *Ready* (Chapter 17's probes — this is where they earn their keep), so a bad image stops the rollout instead of taking the service down; and `kubectl rollout undo` points back at the previous

ReplicaSet, which is kept around precisely for this. The pipeline's kubectl rollout status (Chapter 23) is CI waiting on this machinery to conclude.

With one replica — this cluster — a rolling update still means start-new-then-stop-old, a brief two-pod overlap costing a moment of double memory. maxUnavailable/maxSurge tune that dance at scale.

15.3 StatefulSets: when identity matters

For the databases, interchangeability is exactly wrong: whichever pod comes up must be *the same Postgres* with *the same data*. A StatefulSet provides what a Deployment refuses to: stable names (postgres-course-0, not a random suffix), per-replica PVCs via volumeClaimTemplates (Chapter 19) that survive the pod and re-attach to its successor, and ordered, one-at-a-time replacement. From postgres-course.yaml:

```
kind: StatefulSet
spec:
  serviceName: postgres-course
  replicas: 1
  volumeClaimTemplates:          # a PVC per replica, not per manifest
    - metadata: { name: data }
      spec:
        accessModes: [ReadWriteOnce]
        resources: { requests: { storage: 5Gi } }
```

The decision rule: does the *process* hold state the next process must inherit? Databases yes; Spring services no — their state lives in those databases, which is what made them Deployments, and what makes them scalable and rollable at all. (The JWT statelessness of Chapter 5 is this same property one layer up.)

PITFALL — THE DEPLOYMENT-DATABASE DATA LOSS

Symptom: someone runs a database as a Deployment with a plain volume; it works for months; then a node problem creates a *second* replica pointing at the same files, and the database corrupts. Cause: a Deployment's contract is "interchangeable replicas" and its rolling update briefly runs old and new *simultaneously* — fatal when both open the same data directory. StatefulSet ordering + per-replica claims exist to make this impossible. The lesson generalizes: choose the controller by its *contract*, not by which YAML you saw first.

15.4 Other workload kinds

For completeness, the remaining controllers, none yet used here: **DaemonSet** (one pod per node — log collectors, node metrics), **Job** (run to completion — Chapter 7's Flyway-as-CI-step would be one), **CronJob** (Jobs on a schedule — backups, Chapter 19's missing piece).

HANDS-ON — WATCH A ROLLOUT HAPPEN

```
kubectl -n ts get rs -l app=course-service          # one ReplicaSet
kubectl -n ts set image deployment/course-service \
    course-service=localhost:5000/course-service:1.0.0-SNAPSHOT
kubectl -n ts rollout status deployment/course-service
kubectl -n ts get rs -l app=course-service          # two -- old kept at 0
kubectl -n ts rollout undo deployment/course-service
```

The second `get rs` is the point: the old ReplicaSet still exists, scaled to zero — the undo target, kept by default for the last ten revisions.

CHECK YOURSELF (answers: Appendix A)

1. Why does a Deployment manage ReplicaSets instead of pods directly? What feature falls out of that indirection?
2. State the contract difference between Deployment and StatefulSet in one sentence each, then classify: Eureka server, a Redis cache, a Kafka broker, the api-gateway.
3. During a rolling update, what condition gates old-pod termination, and which chapter's machinery provides it?
4. Explain the exact mechanism by which running Postgres under a Deployment invites corruption where a StatefulSet does not.

Configuration and Secrets in the Cluster

16.1 Two injection mechanisms

Part I established that env vars outrank every file. Kubernetes offers two ways to set them, and the manifests use both deliberately:

```
env:                                  # literal, visible in git
  - { name: SPRING_DATASOURCE_URL,
      value: "jdbc:postgresql://postgres-course:5432/ts_course" }
envFrom:                              # every key of a Secret
  - secretRef: { name: postgres-course-app }
  - secretRef: { name: jwt }
```

The split *is* the security boundary: topology (hostnames, ports, feature flags) is not secret and belongs in reviewable git; credentials never appear in git at all. When you read a Deployment here, every `env:` entry is public configuration and every `envFrom:` line is a pointer to something created out-of-band.

16.2 Secrets: what they are and are not

A Secret is a namespaced object holding base64-encoded key/value pairs. Two honest statements about it:

- Base64 is *encoding*. `echo cm9vdA== | base64 -d` reveals the password. The protections are access control (RBAC decides who may read Secrets — notice Chapter 22 grants Jenkins no such verb), separate lifecycle from code, and optional encryption at rest.
- It is still the right container: every mechanism above it (Sealed Secrets, Vault, cloud secret managers) ultimately materializes a Secret for pods to consume. Learning the plain object first is learning the interface.

16.3 `create-secrets.sh`, annotated

Secrets here are created by an idempotent script, not manifests:

```
kubectl -n "$NS" create secret generic postgres-course \
  --from-literal=POSTGRES_USER="$PG_USER" \
```

```
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_course \
  --dry-run=client -o yaml | kubectl apply -f -      ①

kubectl -n "$NS" create secret generic postgres-course-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -      ②

kubectl -n "$NS" create secret generic jwt \
  --from-literal=JWT_SECRET="$JWT_SECRET"            ③
```

① The idempotent-create idiom: `create` alone fails if the Secret exists; rendering with `–dry-run` and piping to `apply` makes the script safely re-runnable — create or update, either way converging. A tiny declarative-over-imperative lesson in shell form.

② The same credential twice, spelled per consumer: the Postgres image wants `POSTGRES_USER`; Spring wants `SPRING_DATASOURCE_USERNAME`. Two Secrets mean each pod's `envFrom` imports only names it understands — no pod carries a grab-bag of foreign variables.

③ One Secret consumed by two Deployments — auth-service signs JWTs with it, api-gateway validates. The shared-key contract of Chapter 5, expressed as infrastructure: rotating it is one script run and two pod restarts, and the two consumers *cannot* drift.

DECISION — SCRIPT, NOT MANIFESTS — AND NOT IN THE KUSTOMIZATION

Secret manifests in git would put base64'd credentials one `base64 -d` away from anyone with repo access, forever, in history. The script keeps values in the operator's environment (`JWT_SECRET=... ./create-secrets.sh`), defaults matching dev so behavior is identical, and git holding only the *shape*. The kustomization's comment says the same thing from the other side: secrets are deliberately absent from `resources:`.

16.4 ConfigMaps

The Secret's non-secret twin — same mechanics, no pretense of sensitivity, plus a talent Secrets share but ConfigMaps use more: mounting as *files*. This project has none yet for an honest reason: its non-secret config already flows through the config-server and env vars. The natural first ConfigMap arrives with Chapter 26's config-server retirement — the `configurations/*.yml` files mounted into pods, let-

ting Spring read them natively and deleting a service.

PITFALL — CHANGED THE SECRET, NOTHING HAPPENED

Symptom: you re-run `create-secrets.sh` with a new password; services keep using the old one, until some pod restarts days later and starts failing authentication — an outage on a delay timer. Cause: env vars are injected *at container start*; running pods never see Secret updates. Rotation is therefore two steps by design: update the Secret, then `kubectl -n ts rollout restart deployment ...` for every consumer, immediately — while old and new credentials both work if the backing system allows it.

16.5 Outside the project

The gap between this setup and industrial practice is one concept: making secrets *deliverable by pipeline* without being readable in git. Three rungs on that ladder: **Sealed Secrets** (encrypt with the cluster's public key; commit the sealed blob; a controller decrypts in-cluster — GitOps-compatible with one moving part); **External Secrets Operator** (a Secret manifest that is a *pointer* into Vault/AWS Secrets Manager, synced by a controller); and **Vault injection** (secrets never become Kubernetes objects at all). Each replaces the script; none changes how pods consume — `envFrom` survives every migration, which is why the manifests are already future-proof.

CHECK YOURSELF (answers: Appendix A)

1. Base64 is not encryption — so name the three things that actually protect a Secret, and where this project stands on each.
2. Why two Secrets for one Postgres credential? What failure does the split prevent?
3. Describe complete JWT-secret rotation on this cluster: commands, order, and the failure mode of doing only half.
4. Which config item in this project would move to a ConfigMap first, and what service does that retire?

Health Probes and Resource Management

This chapter is the operational heart of Part V: how the kubelet decides a pod is alive and ready, and how the scheduler and kernel share two cores among a dozen JVMs. Nearly every line was learned by watching a real pod misbehave.

17.1 The three probes

startupProbe "still booting — be patient." While it fails, the other probes are suspended. Exhaust its budget and the container is restarted.

readinessProbe "may traffic come *right now*?" Failing pulls the pod out of Service endpoints — no restart, no drama, just no new requests until it passes again.

livenessProbe "is the process beyond saving?" Failing *restarts* the container. The dangerous one.

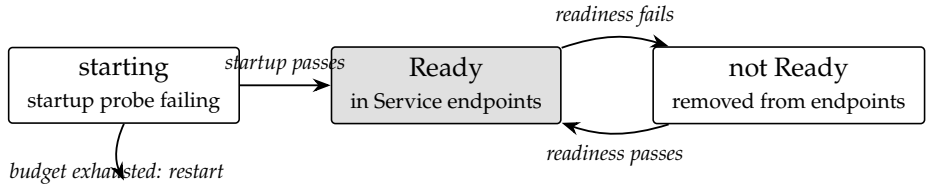

Figure 17.1: The pod's life as the kubelet sees it.

From course-service.yaml:

```
startupProbe:
  httpGet: { path: /v3/api-docs, port: http }   (1)
  periodSeconds: 10
  failureThreshold: 12                          (2)
readinessProbe:
  httpGet: { path: /v3/api-docs, port: http }
  periodSeconds: 5
  failureThreshold: 3
# no livenessProbe                              (3)
```

① The probe-path decision. /actuator/health is the purpose-built endpoint — but course-service and auth-service do not permit it through Spring Security, so it answers 401 and the kubelet counts that as failure. The one unauthenticated path that only answers 200 *after the full context is up* — Flyway migrated, beans built — is springdoc's /v3/api-docs. Dictionary-service and the gateway *do* permit actuator and probe it properly; and notification-service has no Spring Security at all, so its health endpoint is open by default — three services, three answers to the same question. Rule: a probe path must be unauthenticated *and* a true readiness signal; a tcpSocket check passing the moment Tomcat binds is neither.

② $12 \times 10\,\text{s}$ = two minutes of boot grace — sized to a JVM cold-starting on two contended cores.

③ Deliberate absence. Liveness kills; the only thing it would catch beyond readiness is a truly wedged JVM — rare — while a long GC pause on a loaded 2-core box could fail the probe and kill a *healthy* pod. Add liveness when you have observed the hang it is for.

PITFALL — STARTUP PROBE FAILED: STATUSCODE 404 — OR 403

Symptom (milestone 6, builds #6): notification-service logs show Kafka partitions assigned and a fully started application, yet the pod never goes Ready; the probe reports 404. dictionary-service, same story, but 403. Cause: the config-server files *expose* health,info,metrics,prometheus for every service — but only config-server, discovery-server and api-gateway had spring-boot-starter-actuator on the classpath. Exposure configuration for an absent library is silently inert; the URL is just a 404. The 403 variant is Spring Security's contribution: a 404 forwards to /error, /error is not in the permitAll list, and a stateless chain with no entry point answers 403 — so the symptom points at authorization while the cause is a missing jar. Fix: the starter in all four service POMs. Lesson: a probe path must be verified with curl from inside a pod *before* it is trusted in a manifest — the book's earlier claim that dictionary-service "probes health properly" was written from its SecurityConfig, not from a live request.

PITFALL — THE RESTART LOOP THAT CREATES ITSELF

Symptom: a service restarts every few minutes under load, each restart making the load worse. Cause: an aggressive liveness probe timing out during GC pauses or slow downstream calls — the probe *causes* the outage it was meant to detect. This failure mode is why "no liveness probe" is a defensible

default and why liveness, when added, should check only "is the event loop responding," never downstream dependencies.

17.2 Requests, limits, and the JVM

```
resources:
  requests: { memory: 512Mi, cpu: 200m }   ①
  limits: { memory: 768Mi }                ②
env:
  - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" } ③
```

① *Requests* are the scheduler's accounting: the pod is placed only where 512 Mi and 0.2 CPU are unclaimed. Requests *reserve*, they do not cap.

② *Limits* are kernel-enforced ceilings. Exceed the memory limit and the process is OOM-killed — status `OOMKilled`, exit code 137, no Java exception, no log line. Note the asymmetry: a memory limit, but *no CPU limit* — next decision box.

③ The container-aware JVM sizes its heap as a fraction of the *container's* memory limit: 75% of 768 Mi for heap, the rest for metaspace, threads, and off-heap. Without this, defaults leave either waste or an OOM-kill margin of zero.

DECISION — WHY NO CPU LIMIT

CPU limits throttle: a container over its quota is paused until the next scheduling period — and a starting JVM is exactly when a service wants every cycle (class loading, JIT). Throttled startup turns 60 seconds into 200 and fails the startup probe: the limit manufactures the failure. Unlike memory, CPU is compressible — contention slows things instead of killing them — so the pattern used here is standard advice: **always request CPU, rarely limit it**. Memory gets a limit because the failure mode of unbounded memory is the kernel killing *someone*, and you want the boundary predictable.

ON THIS HARDWARE

The requests across all services sum to roughly what 19 GB minus Jenkins, k3s, and the datastores can hold — deliberately. Requests are promises to the scheduler; over-promising on a single node means pods that never schedule (`Pending` forever), under-promising means OOM roulette. On one node you are doing capacity planning by hand; a cloud autoscaler turns the same

arithmetic into node-pool sizing.

HANDS-ON — WATCH A STARTUP THROUGH THE PROBE'S EYES

```
kubectl -n ts delete pod -l app=course-service
kubectl -n ts get pods -w
# Running 0/1  <- startup probe still failing; no traffic yet
# Running 1/1  <- /v3/api-docs answered; endpoints updated
kubectl -n ts describe pod -l app=course-service | tail -n 12
```

The gap between `0/1` and `1/1` is Flyway plus context startup — the window in which, probe-less, requests would have hit a half-started service.

CHECK YOURSELF (answers: Appendix A)

1. For each probe: what does failing it do, and which is safe to be aggressive with?
2. Why is `/v3/api-docs` an honest readiness signal for course-service while `tcpSocket: 8082` is a lie? What makes `/actuator/health` unusable there, and usable on dictionary-service?
3. A pod shows `OOMKilled`, exit 137, clean logs. Explain the mechanism and why no Java exception exists. First two things to check?
4. Requests vs. limits: which does the scheduler read, which does the kernel enforce, and why does this project limit memory but not CPU?

Networking: Services, DNS, Ingress

18.1 The problem pods create

Pods have cluster-routable IPs — and lose them at every replacement. Chapter 15 made pods disposable; something stable must sit in front. That something is the *Service*: a fixed virtual IP and DNS name, backed by a live list of pod endpoints selected by label:

```
apiVersion: v1
kind: Service
metadata: { name: course-service }
spec:
  selector: { app: course-service }     ①
  ports:
    - { port: 8082, targetPort: 8082 }  ②
```

① The endpoint controller continuously lists *Ready* pods matching this selector — readiness (Chapter 17) is the membership test. ② Traffic to the Service's port is forwarded to a chosen endpoint's targetPort.

Cluster DNS gives every Service a name: `course-service` within the namespace, `course-service.ts.svc.cluster.local` from anywhere. This is why the Kubernetes env overrides look so much like the Compose ones — both platforms resolve a service's name on a private network; the project deliberately named its k8s Services after its Compose services so the overrides match.

The default Service type, `ClusterIP`, is reachable *only inside* the cluster — a security property Part II leaned on for the password-less Mongo. `NodePort` and `LoadBalancer` exist to expose Services directly; this project instead funnels everything through one Ingress.

18.2 Ingress and Traefik

A Service exposes one thing; an *Ingress* is the L7 front door: host- and path-based HTTP routing, later TLS termination, declared as data and executed by an *Ingress controller* — in k3s, the bundled Traefik watching Ingress objects. From `api-gateway.yaml`:

```
kind: Ingress
spec:
  rules:
    - host: ts.local                    ①
      http:
        paths:
          - path: /api                  ②
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          # /oauth2 and /login/oauth2 likewise -> api-gateway
```

① Host-based rule: this applies to requests whose `Host` header says `ts.local` — a name that exists only in the Windows hosts file, pointing at the server's tailnet IP. Multiple hosts can share the cluster's single 80/443.
② Path fan-out. Only three prefixes enter, all to the gateway; `/` is reserved for the frontend milestone.

18.3 The full path of one request

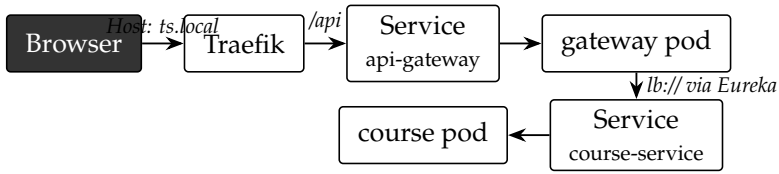

Figure 18.1: Two routers, two discovery systems, one request.

Worth pausing on: the gateway's Eureka lookup returns a *pod IP* (Chapter 3's prefer-ip-address), so hop four technically bypasses the course-service Service. Both discovery systems run in parallel — Eureka for gateway-to-service, Service DNS for everything else (config-server, databases, Kafka). Chapter 26's simplification collapses them into one.

PITFALL — THE SERVICE THAT SELECTS NOTHING

Symptom: `curl` to a Service name times out; pods are Running and healthy. Cause #1: label mismatch — the Service selector and the pod template labels differ by a typo; the endpoint list is empty and nothing complains. Check: `kubectl -n ts get endpoints course-service` — an empty ENDPOINTS column

is this bug. Cause #2: endpoints exist but the pods are not Ready — same command shows them under `NotReadyAddresses`. The Service abstraction fails *silently*; the endpoints object is where it becomes visible.

DECISION — ONE INGRESS HOST + HOSTS-FILE, NOT NODEPORTS

NodePorts (one weird port per service) would work over Tailscale without any DNS games — and produce URLs like `100.85.66.43:30082` that leak topology, dodge the gateway, and can't do TLS sanely. One Ingress host means one address to secure and a browser experience identical to production; the `ts.local` hosts-file entry is the only client-side cost. The upgrade path — a real domain plus cert-manager, or a Cloudflare Tunnel — changes the Ingress host and nothing else.

18.4 Outside the project

On EKS the same Ingress object, annotated for the AWS controller, provisions an actual ALB; on OpenShift it becomes a `Route` — same idea, different noun. The piece this cluster omits: **NetworkPolicies** — firewall rules for pod-to-pod traffic ("only the gateway may reach course-service:8082"), which is the in-cluster enforcement the trusted-edge pattern of Chapter 5 called its remaining soft spot. Flat-network trust is normal for a homelab and negligent for a bank.

CHECK YOURSELF (answers: Appendix A)

1. Pod IPs change at every replacement, yet `course-service:8082` always works. Name each mechanism between the name and the pod's socket.
2. How does readiness interact with Service endpoints, and what does that mean during a rolling update?
3. Diagnose: Service exists, pods Ready, `get endpoints` shows an empty list. What is wrong, and why did nothing error?
4. Why does `ts.local` require a hosts-file entry, and what replaces that entry in the two production upgrade paths?

Storage: Volumes, Claims, and State on a Single Node

19.1 The abstraction stack

Containers' filesystems die with them. Kubernetes separates the *request* for storage from its *provision* through three objects:

PersistentVolume (PV) an actual piece of storage — a directory, an EBS volume, an NFS export.

PersistentVolumeClaim (PVC) a workload's request: "5 Gi, ReadWriteOnce." The claim is what pods mount; it binds to a PV.

StorageClass the factory: how to make a PV on demand when a claim appears (*dynamic provisioning*).

The indirection is portability: `postgres-course.yaml` says only "I need 5 Gi"; *how* that becomes disk is the cluster's business. The same manifest gets a local directory on k3s and an EBS volume on EKS.

19.2 local-path on k3s

k3s bundles the `local-path` StorageClass: a PVC becomes a directory under `/var/lib/rancher/k3s/storage/` on the node's SSD. Simple, fast, and honest about its limits — the data is on *that node's disk*. On multi-node clusters that pins the pod to the node; on this single-node cluster it just means what it says.

ON THIS HARDWARE

local-path provides *persistence* (data survives pod and node restarts), not *durability* (one SSD failure loses everything) and not *backup* (a bad `DELETE` replicates to nowhere). The honest missing piece is a CronJob running `pg_dump` to MinIO or off-box. Rule of thumb worth keeping: a database without tested restores is a cache.

19.3 Two claim styles in the manifests

MinIO (a Deployment with one replica) uses a standalone PVC; the databases use volumeClaimTemplates inside their StatefulSets — a PVC *minted per replica* and, crucially, *re-bound to the replacement pod* rather than a second live pod. That distinction is the storage half of Chapter 15's corruption pitfall.

```
volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: [ReadWriteOnce]      ①
      resources: { requests: { storage: 5Gi } }
# in the pod template:
volumeMounts:
  - { name: data, mountPath: /var/lib/postgresql/data }
env:
  - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }  ②
```

① ReadWriteOnce = mountable read-write by one *node* at a time — fits a database exactly. (RWX — many nodes — needs NFS-like storage and is the mode people want for shared uploads before discovering object storage, Chapter 9, is the better answer.)

② The PGDATA trick, a genuine classic: the Postgres image refuses to initialize into a non-empty directory, and ext4 volumes arrive with lost+found. Pointing PGDATA one directory deeper sidesteps it. You will meet this exact pattern in StackOverflow-years' worth of Postgres-on-Kubernetes setups.

PITFALL — DELETED THE STATEFULSET, DATA STILL THERE — OR GONE

Two surprises in one. First: deleting a StatefulSet does *not* delete its PVCs — by design, data outlives orchestration mistakes; re-create the StatefulSet and it re-binds. Second, the reverse trap: deleting the PVC while experimenting *does* delete the PV and its directory (reclaimPolicy Delete, the dynamic-provisioning default). The asymmetry is worth memorizing before you clean up a namespace: workload objects are disposable; claims are where the data is.

HANDS-ON — SEE THROUGH THE ABSTRACTION

```
kubectl -n ts get pvc                    # claims and their PVs
kubectl get pv                           # cluster-scoped, note the paths
ssh ts-server 'sudo ls /var/lib/rancher/k3s/storage/'
# pvc-...  one directory per PV -- your databases, as plain files
```

Demystification is the point: a PVC on this cluster is a directory. On EKS the same two kubectl commands would show EBS volume IDs — same objects, different factory.

19.4 Outside the project

On managed Kubernetes, StorageClasses map to cloud disks (EBS/gp3 on EKS, Persistent Disks on GKE) with snapshots, encryption and resizing — `VolumeSnapshot` objects make backup part of the same declarative model. The industrial pattern this chapter points toward, though, is subtraction: teams increasingly keep *databases outside the cluster* (RDS, Cloud SQL) and let Kubernetes run only stateless things — trading Chapter 15's StatefulSet care for a managed bill. Both sides of that trade are now visible to you.

CHECK YOURSELF (answers: Appendix A)

1. PV, PVC, StorageClass: which does a pod mount, which is cluster-scoped, and what does the indirection buy this project's manifests on a future EKS?
2. Why is `ReadWriteOnce` sufficient for Postgres, and what kind of workload pushes people toward RWX — usually wrongly?
3. Explain both directions of the deletion asymmetry: StatefulSet gone/PVC stays, PVC gone/data gone.
4. What do persistence, durability, and backup each mean, and which does local-path provide?

Kustomize: Manifests as a Set

20.1 The problem

Part V produced a dozen YAML files. Applying them one by one invites forgetting one; and the moment a second environment exists ("prod, but with 2 replicas and real secrets"), copy-pasting manifests per environment guarantees drift. Two tools dominate this space; this project uses the one built into kubectl.

20.2 The kustomization, annotated

`kubectl apply -k deploy/k8s/base` reads one file:

```
# deploy/k8s/base/kustomization.yaml
namespace: ts                    (1)
resources:                       (2)
  - postgres-course.yaml
  - postgres-auth.yaml
  - mongodb.yaml
  - minio.yaml
  # kafka.yaml requires the Strimzi operator (one-time admin step:
  # ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
  - kafka.yaml
  - kafka-topics.yaml
  - maildev.yaml
  - config-server.yaml
  - discovery-server.yaml
  - course-service.yaml
  - auth-service.yaml
  - dictionary-service.yaml
  - notification-service.yaml
  - api-gateway.yaml
commonLabels:                    (3)
  app.kubernetes.io/part-of: teacher-supporter
```

① Stamped onto every resource — no manifest needs its own `namespace:` line, and retargeting the whole app to another namespace is a one-line change.

② The definitive list. Two deliberate absences teach more than the presences: `namespace.yaml` is out because it is cluster-scoped and Jenkins' namespaced

Role could not apply it (Chapter 22); Secrets are out for Chapter 16's reasons. A third object with the same admin-only status is not a file at all: the Strimzi operator and its CRDs, installed once by install-strimzi.sh (Chapter 25) — kafka.yaml is listed here but cannot apply until they exist. The listed order is documentation only — the API server accepts everything and reconciliation sorts out reality.

③ A label stamped on all objects, making the whole application one selector: kubectl get all -l app.kubernetes.io/part-of=teacher-supporter — or, in a disaster drill, one delete.

20.3 Bases and overlays

Kustomize's real feature arrives with the second environment. A *base* holds the truth; *overlays* patch it:

```
# deploy/k8s/overlays/prod/kustomization.yaml
resources:
  - ../../base
patches:
  - target: { kind: Deployment, name: course-service }
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 2
images:
  - name: localhost:5000/course-service
    newTag: v1.4.2
```

No templates, no variables — overlays are *merges of plain YAML*. Every layer is valid Kubernetes YAML you can read, diff, and pipe through kubectl kustomize to inspect the final output before anything touches the cluster. The directory layout is already overlay-shaped (base/), so the day a prod overlay is needed, it is an mkdir, not a migration.

DECISION — KUSTOMIZE, NOT HELM

Helm is the other answer: manifests as Go templates (replicas: {{ .Values.replicaCount }}) with per-environment value files, packaged as versioned, shareable *charts*. Templating is strictly more powerful — loops, conditionals, one chart deploying anyone's app — and that is the point of divergence: Helm shines when you *distribute* software to strangers' clusters; for deploying your own app, templates make every manifest unreadable until rendered. Kustomize keeps plain YAML plain, ships inside kubectl, and covers replicas-

/images/labels patching — the whole need here. You will still *consume* Helm charts for third-party software (Strimzi, kube-prometheus-stack); writing one can wait until you have users.

PITFALL — SPEC.SELECTOR: FIELD IS IMMUTABLE

Symptom: the pipeline's `kubectl apply -k` fails on one Deployment with this error, although the same file applies fine by hand. Cause: `commonLabels` stamps its label not only onto metadata but into every Deployment's *selector* — and selectors are immutable once created. An object first created *outside* Kustomize (`kubectl apply -f maildev.yaml` during milestone 6's setup, selector `{app: maildev}`) can never be updated by Kustomize afterwards (selector `{app: maildev, part-of: teacher-supporter}`). Fix: delete the hand-made object and let the pipeline recreate it. Rule: an object managed by a kustomization is applied *only* through it — for a one-off test, render with `kubectl kustomize` and pipe that, never the raw file. (Modern Kustomize's `labels:` with `includeSelectors: false` avoids selector injection — but switching now would break every selector already created the old way.)

PITFALL — IT WORKED YESTERDAY: THE PRUNE ILLUSION

Symptom: you delete a manifest from `resources:`, re-apply, and the object is still running — for months, until it conflicts with something. Cause: apply-the-set is still apply — Chapter 14's rule that apply never deletes holds for Kustomize too. The removed Ingress of Chapter 4 needed a manual `kubectl delete`. The systematic fixes: `kubectl apply -k –prune` (still gated behind flags for good reason — a mislabeled selector can mass-delete), or GitOps controllers (Chapter 26) whose whole job is making git and cluster converge, deletions included.

HANDS-ON — RENDER BEFORE YOU TRUST

```
kubectl kustomize deploy/k8s/base | less      # the final YAML, merged
kubectl kustomize deploy/k8s/base | grep -c 'kind:'   # object count
kubectl apply -k deploy/k8s/base --dry-run=server     # validate, no changes
```

`kubectl kustomize` is pure text transformation — reading its output is how you review what an overlay actually changes, and `–dry-run=server` is the closest thing to a manifest unit test.

CHECK YOURSELF (answers: Appendix A)

1. What two problems does a kustomization solve over `kubectl apply -f` per file, even with zero overlays?
2. Why are the namespace and the Secrets deliberately missing from `resources:`? (Two different reasons.)
3. Helm vs. Kustomize: state each tool's home turf in one sentence, and name the third-party software in this project's future that will arrive as a chart.
4. A teammate removes `mongodb.yaml` from the list to "decommission" it. What actually happens, and what are the two correct procedures?

Part VI

CI/CD

CI/CD: The Concepts

21.1 Two practices, one pipeline

Continuous Integration every change is merged into the shared branch frequently and *verified automatically* — compiled, tested, packaged. CI's product is confidence: the main branch is always in a known-good state, and a breakage is caught minutes after the commit that caused it, while its author still has context.

Continuous Delivery/Deployment every verified change is automatically prepared for release (Delivery) or actually released (Deployment — no human gate). This project practices the stronger form: a push to `main` ends as running pods, no button pressed.

The pipeline is the machine that enforces both:

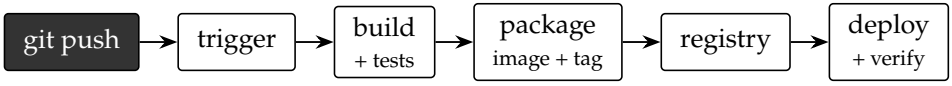

Figure 21.1: The universal pipeline shape. Tool names vary; stages do not.

Whether it is Jenkins+Jib+k3s (this project), GitHub Actions+Docker+EKS, or GitLab CI+Kaniko+OpenShift — source in, tested artifact into a registry, orchestrator told to converge on it, verification that it did. Learn the shape once and every employer's pipeline is a renaming exercise.

21.2 The vocabulary

Trigger what starts a run. *Webhook*: the git host calls the CI server on push — instant, but requires the CI server to be reachable from the internet. *Polling*: CI asks the repo "anything new?" on a schedule — reachable-from-nowhere, at the cost of latency. This project polls (`H/5 * * * *`: about every five minutes) precisely because Jenkins sits behind home NAT with no public address — topology decided the trigger, a pattern you will see in every locked-down corporate network.

Stage / step the pipeline's units: stages are the named phases (visible in the UI, gating progress); steps are the commands inside.

Artifact anything a build produces worth keeping: the jar, the image, test reports. Artifacts must be *immutable and traceable* — hence tagging images with the git SHA, so a running pod can be traced to the exact commit it runs.

Test gate the property that later stages run only if earlier ones passed. A pipeline that deploys despite failing tests is a deployment script with extra steps.

Environment promotion the same artifact moving dev → staging → prod. The iron rule: *promote artifacts, never rebuild them* — a rebuild "for prod" is a different binary than the one you tested. Chapter 1's env-var machinery exists exactly so one image fits all environments.

Rollback the escape hatch. Because the registry keeps SHA-tagged images and Kubernetes keeps old ReplicaSets (Chapter 15), rolling back is re-pointing at a previous artifact — seconds, not a rebuild.

DECISION — DEPLOY ON EVERY PUSH TO MAIN — FOR ONE DEVELOPER

Continuous Deployment without approvals is the right call here: one developer, a homelab blast radius, and the fastest possible feedback loop. The same choice at a bank would be wrong not because automation is risky but because *regulated* changes need evidence of review. The graduation path keeps the pipeline and adds gates: a manual approval stage before prod, or — Chapter 26 — GitOps, where "deploy" becomes a reviewable pull request to a config repo. The pipeline shape survives; only the gate policy changes.

PITFALL — THE PIPELINE THAT LIES GREEN

Symptom: every run is green; production still breaks. The classic causes: tests skipped in the build command (`-DskipTests` — which *this* pipeline does in its build stage, accepted consciously because `mvn verify` runs in CI on GitHub Actions — know your own gates); deploy stage not verifying rollout (this one does — `rollout status` fails the build if pods never go Ready); or success measured as "commands exited 0" rather than "service answers requests." A green pipeline is a claim; know precisely what it claims.

CHECK YOURSELF (answers: Appendix A)

1. CI and CD make different promises. State each in one sentence, and identify which stage of this project's pipeline delivers which.
2. Webhook vs. polling: name the deciding constraint in this project, and a corporate setting with the same constraint.
3. Why must a promoted artifact never be rebuilt per environment, and which mechanism from Chapter 1 makes single-artifact promotion pos-

sible?
4. Your pipeline is green and prod is down. List three distinct ways a pipeline can lie, and the fix for each.

Jenkins: The CI Server

22.1 Architecture

Jenkins is a Java web application — the *controller* — that stores jobs, credentials, plugins and build history in one directory, JENKINS_HOME, and executes builds either on itself (via *executors*, its parallel-build slots) or on remote *agents*. The enterprise pattern is a controller that builds nothing, farming work out to disposable agents (often pods, via the Kubernetes plugin).

DECISION — BUILDS ON THE CONTROLLER, JENKINS OUTSIDE THE CLUSTER

Two homelab inversions of enterprise practice, both CPU arithmetic. Agent pods would cost scheduling and image pulls per build on the same two cores the build needs; the controller's two executors spend those cycles on Maven instead. And Jenkins runs in Docker on the host, not in k3s: if you break the cluster — and while learning, you will — the thing that can rebuild the cluster must not have died with it. Both revert cleanly at enterprise scale: in-cluster Jenkins with pod agents is Chapter 26 material, and the migration itself is the lesson.

22.2 The controller image, annotated

Rather than clicking tools into a stock Jenkins, the project bakes its toolchain into an image — deploy/jenkins/Dockerfile:

```
FROM jenkins/jenkins:lts-jdk21          (1)
USER root
RUN ... temurin-25-jdk ...              (2)
RUN ... apache-maven-3.9.9 ...          (3)
RUN wget -qO /usr/local/bin/kubectl ... (4)
ENV JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64  (5)
USER jenkins
RUN jenkins-plugin-cli --plugins \
      git workflow-aggregator ... credentials-binding  (6)
```

(1) LTS Jenkins running on JDK 21 — the version *Jenkins itself* runs on.

② The project targets Java 25, so a second JDK is installed for *builds*. Note JAVA_HOME is *not* changed globally ⑤ — the Jenkinsfile sets it per pipeline. Controller runtime and build toolchain are independent concerns; this Dockerfile keeps them visibly separate.
③ Maven pinned to 3.9.9 — distro packages lag.
④ kubectl: the deploy stage is just this binary plus a kubeconfig.
⑥ Plugins pinned in the image, not clicked in the UI: pipeline support, git, JUnit reporting, and credentials-binding (used by withCredentials in Chapter 23). The whole point: *the CI server itself is now reproducible* — destroy the container, rebuild from the Dockerfile, and only JENKINS_HOME (a named volume) carries state.

One run-time flag from run.sh deserves its own paragraph: –network host. Jib inside Jenkins pushes to localhost:5000 — and inside a bridge-networked container, localhost would be the *container*. Host networking makes Jenkins share the host's network namespace, so its localhost:5000 is the same registry k3s pulls from (Chapter 13's naming contract). Not a shortcut — a requirement of the image-naming scheme.

22.3 Credentials

Jenkins stores secrets in an encrypted store, referenced by ID; pipelines bind them to environment variables only for the duration of a block (withCredentials), and the log masker redacts their values from console output. This project stores one credential: kubeconfig-ts, a file credential holding the kubeconfig Jenkins uses to deploy — which is the interesting part.

22.4 The deploy identity: a ServiceAccount, not admin

The lazy path is giving Jenkins the k3s admin kubeconfig — cluster-admin, licensed to delete every namespace. Instead, deploy/k8s/jenkins-rbac.yaml builds a minimal identity:

```
kind: ServiceAccount          # the identity
metadata: { name: jenkins, namespace: ts }
---
kind: Role                    # namespaced permissions
metadata: { name: jenkins-deployer, namespace: ts }
rules:
  - apiGroups: [”apps”]
    resources: [”deployments”, ”statefulsets”]
    verbs: [”get”, ”list”, ”watch”, ”create”, ”patch”, ”update”] ①
```

```
  - apiGroups: ["apps"]
    resources: ["replicasets"]
    verbs: ["get", "list", "watch"]                    ②
  # + services, PVCs, secrets, configmaps, ingresses, pods/log ...
  - apiGroups: ["kafka.strimzi.io"]                    ③
    resources: ["kafkas", "kafkanodepools", "kafkatopics"]
    verbs: ["get", "list", "create", "patch", "update"]
---
kind: RoleBinding              # identity <- permissions
```

① The verbs are derived from the pipeline's actual commands: `kubectl apply` needs get+create+patch; `set image` needs get+patch. *No delete anywhere*: CI rolls forward; a human deletes. A compromised pipeline can thus corrupt deployments in `ts` but cannot wipe them, and cannot see beyond the namespace — a Role, not a ClusterRole, is the blast-radius decision.

② Discovered the honest way: `rollout status` watches the Deployment's ReplicaSets, so read access to them is required — the kind of permission you find by running the pipeline against the Role and reading the first `Forbidden`.

③ Added with milestone 6: the pipeline may declare Kafka clusters and topics (namespaced custom resources), but installing the operator and its CRDs — cluster-scoped — stays an admin act (Chapter 25). RBAC grows one rule per API group the pipeline touches; the Role file is a changelog of what CI has been trusted with.

The kubeconfig stored in Jenkins simply pairs the cluster address with this ServiceAccount's token instead of the admin certificate.

PITFALL — FORBIDDEN: CANNOT CREATE RESOURCE "NAMESPACES"

Symptom: the deploy stage dies with a Forbidden error on `namespaces` although the RBAC file looks complete. Cause: namespaces are *cluster-scoped*, and a namespaced Role cannot grant them — which is exactly why `namespace.yaml` is excluded from the kustomization (Chapter 20) and the namespace is created once by an admin. RBAC debugging in general: `kubectl auth can-i patch deployments -n ts –as system:serviceaccount:ts:jenkins` answers "would this be allowed" without running the pipeline.

HANDS-ON — PROVE THE BOUNDARY EXISTS

```
kubectl auth can-i patch deployments -n ts \
  --as system:serviceaccount:ts:jenkins        # yes
kubectl auth can-i delete deployments -n ts \
  --as system:serviceaccount:ts:jenkins        # no
```

```
kubectl auth can-i list pods -n kube-system \
  --as system:serviceaccount:ts:jenkins        # no
```

CHECK YOURSELF (answers: Appendix A)

1. What lives in JENKINS_HOME, and why does baking plugins into the image plus a volume for home make the CI server disposable?
2. Why does the controller run two JDKs, and where is each selected?
3. Reconstruct the RBAC verb list for a pipeline that only ever runs `kubectl apply -k` and `rollout status` — which verb in the project's Role could then be dropped?
4. Explain `–network host` on the Jenkins container in terms of Chapter 13's image-naming contract.
5. ServiceAccount, Role, RoleBinding: which is identity, which is capability, which is the join — and why is Role-not-ClusterRole the security headline?

The Pipeline, Line by Line

Everything in this book converges on one 70-line Groovy file. A declarative Jenkinsfile lives in the repository it builds — pipeline as code, versioned and reviewed like everything else.

23.1 Preamble and environment

```
pipeline {
  agent any                                  ①
  options {
    timestamps()
    disableConcurrentBuilds()                ②
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }
  triggers { pollSCM('H/5 * * * *') }        ③
  environment {
    JAVA_HOME = "${env.JDK25_HOME}"          ④
    PATH      = "${env.JDK25_HOME}/bin:${env.PATH}"
    TAG       = "${env.GIT_COMMIT.take(7)}"  ⑤
    NS        = 'ts'
    REGISTRY  = 'localhost:5000'
    SERVICES  = 'config-server discovery-server course-service api-gateway auth-service
      ↪ dictionary-service notification-service'
    MODULES   = 'common,config-server,discovery-server,course-service,api-gateway,auth-
      ↪ service,dictionary-service,notification-service'
  }
```

① Any executor on the controller — Chapter 22's homelab compromise in one word.

② Two builds at once on two cores would thrash; queued runs also collapse (a new commit during a build supersedes waiting ones).

③ Polling because NAT blocks webhooks. The `H` is a hash-spread: "some fixed minute in each 5," so many jobs don't all poll at :00.

④ The per-pipeline JDK switch the controller image prepared: builds see Java 25, Jenkins itself stays on 21.

⑤ The deploy tag is the short git SHA — every image traceable to its commit, every deploy pinned to an immutable tag (Chapter 13's tags-vs-digests lesson

applied).

SERVICES/MODULES the pipeline's registry of what exists: MODULES feeds Maven (comma-separated, includes common which is built but produces no image); SERVICES feeds kubectl (space-separated, deployables only). Adding a service to the cluster means extending both — the one manual step per new service.

23.2 The three stages

```
stage('Build') {
  steps {
    dir('Project/TeacherSupporter') {          ①
      sh 'mvn -B -DskipTests -pl ${MODULES} -am verify'  ②
    } } }
stage('Push images') {
  steps {
    dir('Project/TeacherSupporter') {
      sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
      ↪ ③
    } } }
stage('Deploy') {
  steps {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts',
                            variable: 'KUBECONFIG')]) {   ④
        sh '''
          set -eu                                  ⑤
          kubectl apply -k deploy/k8s/base                ⑥
          for s in ${SERVICES}; do
            kubectl -n ${NS} set image deployment/${s} \
              ${s}=${REGISTRY}/${s}:${TAG}                ⑦
          done
          for s in ${SERVICES}; do
            kubectl -n ${NS} rollout status \
              deployment/${s} --timeout=300s              ⑧
          done
        '''
      } } } }
```

① The repository root is a study monorepo; the project lives in a subdirectory, so every stage dir()s into it.

② -pl (project list) + -am (also make dependencies): build only the listed modules and what they need — monorepo economics on two cores. -B is batch mode (no interactive output). -DskipTests is the consciously accepted gate-

weakening from Chapter 21's pitfall box.

③ One command builds and pushes all seven images — Chapter 13 in a single line. -Djib.to.tags adds the SHA tag alongside the SNAPSHOT tag. The package in front of jib:build is not decoration — see the pitfall below; it cost two failed builds on the day milestones 5 and 6 shipped.

④ The file credential materializes as a temp file; KUBECONFIG is the env var kubectl reads. Outside this block the file does not exist; in the log its content never appears.

⑤ Fail on any error and on unset variables — without it, a failed apply would be sailed past and "deployed" reported.

⑥ Converge the cluster on the manifests — idempotent (Chapter 14), so it is safely re-run every build whether or not anything changed.

⑦ Repoint each Deployment at the SHA-tagged image. This is what actually triggers rollouts — and note the manifests themselves keep the SNAPSHOT tag, so *apply* alone would not redeploy; the pipeline's pinning and the manifests' default coexist.

⑧ The verification gate: block until each rollout completes; fail the build if any pod never goes Ready within five minutes. This line is what makes the pipeline's green mean "running," not "commands executed" — it delegates truth to the probes of Chapter 17.

PITFALL — COULD NOT FIND ARTIFACT COM.TS:COMMON:JAR:1.0.0-SNAPSHOT

Symptom: the Build stage passes, then the Push stage fails on the first module that depends on common, unable to resolve a jar that the previous stage just built. Cause: mvn jib:build is a *bare goal* — Maven runs it directly with no lifecycle phases, so nothing in the reactor is compiled or packaged in that session, and sibling dependencies fall back to the local repository, where common was never installed (verify stops short of install). Modules without a sibling dependency (config-server, discovery-server) sail through, which makes the failure look module-specific. Fix: put a phase in front — package jib:build — so the reactor packages common before Jib asks for it. General rule: a plugin goal that needs other modules' artifacts must ride a lifecycle phase, not be invoked alone.

Two loops, not one, is a small correctness point: all rollouts start first, then all are awaited — otherwise service N's wait serializes service N+1's start, stretching total deploy time for no gain.

23.3 Failure handling

```
post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([...]) {
        sh 'kubectl -n ${NS} get pods || true'
      } } } }
```

On failure, the build log ends with a pod listing — the first command a human would run, already run. The `|| true` keeps the diagnostic itself from masking the build's real exit status.

23.4 The same pipeline, elsewhere

For calibration, the identical shape as GitHub Actions:

```
on: { push: { branches: [main] } }        # webhook, not polling
jobs:
  deploy:
    runs-on: ubuntu-latest                # disposable agent, not controller
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 25 }
      - run: mvn -B -pl $MODULES -am verify
      - run: mvn -B -pl $MODULES -am jib:build -Djib.to.tags=${GITHUB_SHA::7}
      - run: kubectl apply -k deploy/k8s/base && ...
        env: { KUBECONFIG_DATA: ${{ secrets.KUBECONFIG }} }
```

Same stages, same tag scheme, same secret-binding idea. The differences are the infrastructure trade: GitHub's runners are disposable VMs (agents done right, for free) but they live on the internet — so this variant would need the registry and cluster reachable from outside, the exact constraint that shaped the homelab design. Pipelines are portable; *network topology is what actually differs between shops*.

CHECK YOURSELF (answers: Appendix A)

1. Why tag deploys with the git SHA when the SNAPSHOT tag also exists? Name the failure mode SNAPSHOT-only deploys invite (Chapter 13) and what traceability the SHA adds.
2. Explain `-pl ... -am` and what it saves in a monorepo. What breaks if you

forget to extend MODULES for a new service?
3. Which single line makes this pipeline's success mean "pods are Ready," and which chapter's machinery does it lean on?
4. Why two loops in the deploy stage instead of set-image-then-wait per service?
5. List three things that change and three that do not when this Jenkinsfile is translated to GitHub Actions.

Part VII

Operations and the Road Ahead

Operating the Stack: A Debugging Field Manual

This chapter is the one to reread with a broken cluster in front of you. It assembles the debugging workflow the earlier chapters earned, ordered the way real incidents unfold.

24.1 The four questions

Almost every Kubernetes incident is located by asking, in order:

1. **Is the pod running?** `kubectl -n ts get pods` — read the STATUS and RESTARTS columns; the taxonomy below.
2. **What does the platform say about it?** `kubectl -n ts describe pod <pod>` — the Events section at the bottom is the kubelet and scheduler narrating their decisions: image pulls, probe failures, OOM kills, scheduling refusals.
3. **What does the app say?** `kubectl -n ts logs <pod>` (add `–previous` for the container that just died — the single most-forgotten flag in incident response).
4. **Can I reproduce its view of the world?** `kubectl -n ts exec -it <pod> – sh` and try the connection the app claims is failing.

24.2 A status taxonomy

HANDS-ON — BREAK IT ON PURPOSE — THREE DRILLS

Deliberate sabotage, on a quiet day, teaches more than any outage:

```
# 1. wrong image tag -> watch ImagePullBackOff, then roll back
kubectl -n ts set image deployment/course-service \
    course-service=localhost:5000/course-service:nope
kubectl -n ts rollout undo deployment/course-service

# 2. scale the database down -> watch dependents degrade honestly
kubectl -n ts scale statefulset/postgres-course --replicas=0
kubectl -n ts get pods -w      # course-service readiness reacts
kubectl -n ts scale statefulset/postgres-course --replicas=1
```

Status	Meaning and first move
Pending	unschedulable: requests exceed free resources, or PVC unbound. describe shows which.
ImagePullBackOff	registry unreachable, name typo'd, or the HTTPS-vs-HTTP trap of Chapter 13.
CrashLoopBackOff	process exits soon after start; backoff doubles between retries. logs –previous.
OOMKilled (exit 137)	memory limit hit; no Java trace exists — Chapter 17's kernel kill. Raise limit or shrink heap.
Running 0/1	alive but readiness failing: normal during startup; a problem when persistent — probe the probe's own URL.
Completed/Error	for Jobs; for a service pod, Error means the process exited nonzero once.

Table 24.1: What the STATUS column is trying to tell you.

```
# 3. kill Eureka -> verify existing routes survive on client caches
kubectl -n ts delete pod -l app=discovery-server
```

Each drill pairs a chapter's theory with its observable behavior: 13 (naming), 17–18 (probes and endpoints), 3 (client-side caching).

24.3 Reading a service's world from inside

When logs claim a connection failure, verify the claim from the pod's own network position — DNS and reachability differ between your machine and the pod:

```
kubectl -n ts exec -it deploy/course-service -- sh
wget -qO- http://config-server:8888/actuator/health   # DNS + service up?
nc -zv postgres-course 5432                           # TCP reachable?
env | grep SPRING                                     # what was injected?
```

That last command settles arguments fast: it prints what configuration the pod *actually* received, which beats re-reading manifests.

24.4 The routine that keeps weekends free

After every deploy the pipeline already verifies rollouts; glance at `kubectl -n ts get pods` anyway — RESTARTS creeping upward is tomorrow's outage announcing itself.

Weekly `kubectl top pods -n ts` (requires metrics-server, bundled with k3s): compare actual usage against the requests of Chapter 17 and correct the fiction. Check disk: `df -h` on the server — registries and log files grow; `registry:2` needs occasional garbage collection.

Before demos *do not build* — Chapter 5's hardware box: two cores mean a Maven build and a demo contend for the same CPU that keeps readiness probes answering.

ON THIS HARDWARE

ufw on the server must exempt the pod and service CIDRs (`10.42.0.0/16`, `10.43.0.0/16`) — firewalling them breaks DNS and pod-to-pod traffic in ways that impersonate application bugs: intermittent timeouts, resolvable-sometimes names. When *weird* networking happens on k3s, suspect the host firewall before the app. And k3s reads `registries.yaml` only at startup — after editing, `systemctl restart k3s`, which briefly restarts every pod: not during a demo.

PITFALL — DEBUGGING THE WRONG LAYER

The most expensive habit in Kubernetes debugging is starting where the symptom appeared instead of where the cause lives. A browser 502 can be Traefik (no endpoints), the gateway (Eureka staleness), the service (readiness failing), the database (pod evicted), or the disk (full). The four questions exist to walk the layers in order; resist the temptation to open application logs first just because they are the most familiar layer. `describe` before `logs` — the platform usually already knows.

CHECK YOURSELF (answers: Appendix A)

1. Order the four questions and name the command for each. Which one uses a flag people forget, and when does that flag matter?
2. `CrashLoopBackOff` vs. `Running 0/1`: different layers are failing — which probe concept distinguishes them?
3. A service claims it cannot reach Postgres; from your laptop Postgres answers. What is the next diagnostic step and why is your laptop's success irrelevant?

4. Why can a host firewall change masquerade as an application bug on k3s, and which two CIDRs must stay exempt?

Kafka in the Cluster: Strimzi and the Mail Flow

Milestone 6 moves the broker of Chapter 10 from Compose into k3s and deploys the last service — notification-service — so the first end-to-end *event* flow runs in the cluster: register a user, receive a mail. It is also the chapter where Chapter 14's reconciliation idea grows its most important extension: the *operator*.

25.1 Operators and custom resources

Kubernetes ships controllers for its built-in kinds — Deployments, Services. An *operator* is a controller for kinds you add: a *CustomResourceDefinition* (CRD) teaches the API server a new noun (Kafka, KafkaTopic), and the operator's reconciliation loop turns instances of it into ordinary objects — pods, Services, ConfigMaps — and keeps them converged. Operational knowledge that used to live in runbooks (how to roll a broker safely, how to compute advertised listeners, how to add a partition) is encoded in the operator's code. Strimzi is the Kafka operator; the pattern is universal (Postgres operators, Prometheus operator, cert-manager).

25.2 Installing the operator: an admin act

```
# deploy/k8s/install-strimzi.sh (abridged)
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS"   ①
kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator
```

① The bundle contains the CRDs, the operator Deployment, and its RBAC — much of it *cluster-scoped*. Two consequences: it is applied by an admin, not by Jenkins (whose Role, Chapter 22, stops at the namespace), and it is applied *server-side* because the CRD documents are too large for the client-side last-applied annotation. Idempotent, so it doubles as the upgrade command. The ?namespace= parameter scopes the operator to watch ts only.

The kustomization (Chapter 20) lists kafka.yaml, but that file cannot apply until this script has run once — the third member of the "admin prerequisites" set, after the namespace and the secrets.

PITFALL — NO MATCHES FOR KIND "KAFKA" IN VERSION V1BETA2

Symptom: applying a Kafka manifest copied from any tutorial fails with this error even though the operator is installed and healthy. Cause: Strimzi 1.x serves only kafka.strimzi.io/v1; the v1beta2 version that a decade of blog posts use is gone. Check what a CRD actually serves before trusting an example: kubectl get crd kafkas.kafka.strimzi.io -o jsonpath={.spec.versions[*].name}. This project hit it live on installation day.

25.3 The cluster, declared

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool                          (1)
metadata:
  name: dual-role
  labels: { strimzi.io/cluster: ts-kafka }   (2)
spec:
  replicas: 1
  roles: [controller, broker]
  storage: { type: persistent-claim, size: 5Gi, deleteClaim: false }   (3)
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
spec:
  kafka:
    listeners:
      - { name: plain, port: 9092, type: internal, tls: false }   (4)
    config:
      offsets.topic.replication.factor: 1   (5)
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168
    jvmOptions: { -Xms: 256m, -Xmx: 512m }   (6)
  entityOperator:
    topicOperator: {}                        (7)
```

① Two objects: the *pool* says where Kafka runs (replicas, roles, storage, resources); the Kafka resource says how it behaves. One dual-role KRaft node is the Kubernetes spelling of Compose's process.roles=broker,controller.
② The label joins pool to cluster; the operator matches on it. A mismatch is silent — the pool simply never materializes.
③ A PVC via local-path, exactly like the databases (Chapter 19). deleteClaim: false: deleting the Kafka resource keeps the log — the same asymmetry StatefulSets have by default.
④ Compare with the three-listener, two-advertised-address block in docker-compose.yml: gone. You state *one internal listener*; the operator computes bootstrap and per-broker Services, the advertised addresses, and the KRaft controller listener. Chapter 10's maze was operational knowledge; it now lives in the operator.
⑤ Single broker means every replication factor must be 1, or the first topic creation fails asking for brokers that do not exist.
⑥ Kafka's default heap assumes a server; half a GiB serves a homelab trickle and leaves memory for the JVM services.
⑦ The entity operator's topic half watches KafkaTopic objects and reconciles them into real topics. The user half is omitted: no authentication, no users to manage.

Strimzi names the bootstrap Service <cluster>-kafka-bootstrap; every producer and consumer therefore gets SPRING_KAFKA_BOOTSTRAP_SERVERS=ts-kafka-kafka-bootstrap:9092 in its Deployment — the localhost:9092 config default of Part III overridden the way every other address was.

DECISION — OPERATOR, NOT A HAND-WRITTEN STATEFULSET

A plain KRaft StatefulSet (one pod, one PVC, the compose env block copied in) would cost less memory — no operator, no entity operator — and it is the honest fallback if CPU contention ever bites. The operator wins because it is what Kubernetes shops run (Red Hat sells it as AMQ Streams), because topics become declarative objects, and because broker upgrades and rolling restarts become someone else's tested code. Learning it is the point; the fallback stays documented in STACK-K3S-JENKINS.md.

25.4 Topics as objects

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
```

```
metadata:
  name: ts.user.registered
  labels: { strimzi.io/cluster: ts-kafka }
spec: { partitions: 1, replicas: 1 }
```

Kafka would auto-create these on first use; declaring them makes partition count a reviewed decision, removes the day-one race between producer and topic, and turns `kubectl get kafkatopics` into the inventory of every event contract in the system (four, matching Chapter 6's table). One partition per topic: total order, which trivially preserves the per-user ordering the publishers key for. Partitions can be added later, never removed.

25.5 The last service, and its mailbox

notification-service's manifest is the standard Deployment of Chapter 15 with three specifics: the bootstrap address above; `SPRING_MAIL_HOST=maildev` / `PORT=1025` overriding the config's `${MAIL_HOST:localhost}` default via relaxed binding; and probes on `/actuator/health` — this service has no Spring Security at all, so the endpoint is open by default (the third answer to Chapter 17's probe question). Maildev is a tiny Deployment plus Service (ports 1025 SMTP, 1080 web); it delivers nothing and shows everything it caught. No Ingress: it is an operator's tool, reached by `port-forward`. It also supplied this milestone's second live lesson: applied by hand before the pipeline ran, its selector lacked the label Kustomize injects, and the first pipeline deploy died on "field is immutable" — Chapter 20's pitfall, learned the expensive way.

PITFALL — NOTREADY — FATALPROBLEM, WHILE THE POD IS RUNNING

Symptom: minutes after applying, `kubectl get kafka` shows `READY` empty and a condition reading "Pod is unschedulable or is not starting" — yet `get pods` shows the broker `1/1 Running`. Cause: the operator's reconciliation timed out while the 600 MB Kafka image was still pulling over Wi-Fi, and wrote a failure status; the *next* periodic loop (every two minutes) observed the running pod and flipped it to Ready. Operator status is eventually honest. Before reacting to a red condition, check the pod it is describing — and check the operator log, where "reconciled" lines show the loops still turning.

ON THIS HARDWARE

Operator + broker + entity operator cost about 1.5 GB here, with the broker's heap capped at 512 MiB. The first image pull is the expensive part — expect

ten minutes on residential Wi-Fi, and the stale-status pitfall above. Everything afterwards is fast: the image is cached on the node, which the smoke test below exploits.

HANDS-ON — PROVE THE BROKER, THEN PROVE THE FLOW

A throwaway pod with the Kafka CLI, in the same network position as the services, using the image the node already holds:

```
kubectl -n ts run kafka-smoke --rm -i --restart=Never \
  --image=quay.io/strimzi/kafka:1.2.0-kafka-4.3.1 -- sh -c '
  B=ts-kafka-kafka-bootstrap:9092
  bin/kafka-topics.sh --bootstrap-server $B --create --topic smoke \
      --partitions 1 --replication-factor 1
  echo hello | bin/kafka-console-producer.sh --bootstrap-server $B --topic smoke
  bin/kafka-console-consumer.sh --bootstrap-server $B --topic smoke \
      --from-beginning --max-messages 1 --timeout-ms 20000
  bin/kafka-topics.sh --bootstrap-server $B --delete --topic smoke'
```

Use a scratch topic, not a real one: a plain-text "hello" on ts.user.registered is a poison pill for the JSON consumer (Chapter 11) — skipped gracefully thanks to ErrorHandlingDeserializer, but why find out in production. Then the real flow, once the pipeline has deployed the services:

```
curl -X POST http://ts.local/api/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"t@example.com","password":"Passw0rd!","firstName":"Test",
       "lastName":"User","role":"TEACHER","activationMethod":"EMAIL"}'
# -> 201 {"message":"Registration successful. Check your email ..."}
kubectl -n ts logs deploy/notification-service | grep -i registered
# maildev's REST API, from inside the cluster (or port-forward 1080 for the UI):
kubectl -n ts run mailcheck --rm -i --restart=Never \
  --image=curlimages/curl:8.10.1 -- -s http://maildev:1080/email
# -> [{"subject":"Activate your TeacherSupporter account","to":"t@example.com",...
```

That request crosses every part of this book: Ingress → gateway → auth-service → Postgres → Kafka → notification-service → SMTP. It ran for the first time on 2026-08-25, pipeline build #7 — after builds #3–#6 each taught one of this chapter's and Chapters 17, 20 and 23's pitfalls. One loose end it exposed: the activation link in the mail still says localhost:8080, the dev default of the link base URL — an env override for milestone 7, when the frontend gives the cluster a real public address.

25.6 Outside the project

Managed Kafka — AWS MSK, Confluent Cloud, Azure Event Hubs' Kafka endpoint — is the operator's operator: the same `Kafka`-shaped declaration made in a cloud console, with brokers you never see. Migration from here is a bootstrap string plus SASL/TLS on the listener; the `KafkaTopic` objects would give way to Terraform resources saying exactly the same thing. The other lesson to carry: the operator pattern is how Kubernetes hosts *any* stateful platform component — Postgres (CloudNativePG), Redis, Elasticsearch — and having installed, misversioned, and status-debugged one, you know the shape of all of them.

CHECK YOURSELF (answers: Appendix A)

1. Define an operator in terms of Chapter 14's controllers, and name the two kinds of object Strimzi added to this cluster.
2. Why is `install-strimzi.sh` an admin step while `kafka.yaml` is applied by Jenkins? Which RBAC concept draws the line?
3. List three things Compose's Kafka block configured by hand that the `Kafka` resource does not mention, and say who computes them now.
4. You see `NotReady / FatalProblem` on the Kafka resource. What two things do you check before acting, and why might the status be wrong?
5. Why declare topics as objects when Kafka auto-creates them, and why one partition each?

The Road Ahead

With Kafka in the cluster (Chapter 25), every service the project has is deployable. The remaining milestones each swap something hand-rolled for the industrial mechanism — and each makes most sense now that you know what it replaces.

26.1 Observability

Beyond `kubectl logs` lies the standard trio: **metrics** (Prometheus scraping the `/actuator/prometheus` endpoints every service already exposes; Grafana dashboards over the RED trio — rate, errors, duration), **traces** (the Zipkin wiring already in every config, currently sampled at zero in the cluster — one env var turns it on once a Zipkin pod exists), and **alerts** (Alertmanager: a page when a service's error rate or a PVC's fullness crosses a line). This lands via the `kube-prometheus-stack` Helm chart — the promised moment where you *consume* Helm without authoring charts (Chapter 20's decision box, honored). Strimzi can export broker metrics into the same Prometheus with a few lines of `metricsConfig` on the Kafka resource — consumer-group lag is the first Kafka alert worth having.

ON THIS HARDWARE

The full stack costs ~1.5 GB and nontrivial CPU — the budget says it fits, but it is the first thing to shed under load. Droppable observability is a homelab luxury; production teams size for it first.

26.2 Frontend and the single host

Milestone 7 serves the React frontend from the cluster and claims the `/` path on `ts.local` that Chapter 18's Ingress deliberately left free. One host, one origin — no CORS, cookies and OAuth redirects behave — and the two-router division of Chapter 4 is complete: Traefik fans out static assets to the frontend and `/api` to the gateway.

26.3 GitOps: Argo CD

The pipeline *pushes* deployments (Jenkins holds a kubeconfig and runs kubectl). GitOps inverts the arrow: an in-cluster agent (Argo CD) *pulls* — continuously comparing a git repository's manifests with the cluster and converging. What that buys, in the vocabulary you now have: the cluster credential never leaves the cluster (Chapter 22's RBAC concern dissolves — CI only pushes images and commits a tag bump); deletions finally propagate (Chapter 20's prune pitfall, solved by design); drift becomes visible (a hand-edited Deployment shows as "OutOfSync"); and rollback is `git revert`. Jenkins keeps CI — build, test, push, update the tag in git — Argo CD takes CD. The Jenkinsfile's deploy stage, this book's Chapter 23, becomes a commit.

26.4 Retiring the classic stack

The deliberate redundancy of Chapters 2 and 3 can now be paid off, service by service:

- **Eureka out**: set the gateway's routes from `lb://course-service` to `http://course-service:8082` — Service DNS and readiness-gated endpoints do everything the registry did here. Delete a deployable, free 512 Mi.
- **Config-server out**: mount the `configurations/` files as a ConfigMap and point `spring.config.import` at the mounted directory (`optional:file:/config/`). Same files, one fewer service, and config changes become manifest changes flowing through GitOps.

Both migrations are résumé material precisely because they run *toward* the platform: knowing why Eureka exists *and* why Kubernetes obsoletes it is the difference between using either and understanding both.

26.5 Jenkins into the cluster

Chapter 22's homelab inversion — builds on the controller, Jenkins outside k3s — reverses cleanly once the pipeline is stable: the Kubernetes plugin spawns an agent pod per build, and Jenkins itself becomes a StatefulSet with `JENKINS_HOME` on a PVC. The migration touches every concept in Part V at once, which is why it is scheduled last.

26.6 OpenShift, briefly

The manifests were kept portable on purpose: Jib's non-root images (Chapter 13) satisfy OpenShift's arbitrary-UID security model, so deploying to a free OpenShift sandbox should need only the Ingress-to-Route translation (Chapter 18). Running that experiment once validates the portability claims this book has been making — and OpenShift vocabulary (Routes, SCCs, BuildConfigs) maps one-to-one onto chapters you have read.

26.7 Where this leaves you

Strip the project names away and the inventory reads like a platform job description: the property-resolution machinery that makes one artifact run anywhere; logs, consumer groups and delivery semantics; images, layers and registries; reconciliation, probes, RBAC, storage classes and operators; pipelines, RBAC-scoped deploy identities, and the GitOps inversion. The homelab constraints were not limitations — they were the syllabus: every "on this hardware" box forced a decision that money usually hides.

CHECK YOURSELF (answers: Appendix A)

1. Which metric would you alert on first for the Kafka path, and what does it measure in Chapter 10's terms?
2. Push-based vs. pull-based deployment: locate the cluster credential in each, and explain which chapter's pitfall GitOps pruning resolves.
3. Sketch the Eureka retirement: what changes in the gateway's config, which two Kubernetes mechanisms replace registration and liveness, and what can be deleted?
4. Why do Jib's images make the OpenShift experiment cheap, and which object translation remains?

Quiz Answers

Chapter 1 — Spring Boot

1. The env var wins: env vars outrank the config-server document, which outranks the packaged YAML. Right for containers because the image must be immutable across environments; the runtime injects what differs.
2. `@Configuration`, `@ComponentScan`, `@EnableAutoConfiguration`. The last: auto-configuration is conditional on the classpath, so removing a dependency silently removes the beans it triggered.
3. It resolves to the literal string `placeholder`. The service can therefore *start* without Google credentials — OAuth fails at click time instead of boot time.
4. Exposure makes the endpoint exist; Spring Security still authorizes each request. If `/actuator/**` is not permitted, probes get 401, which the kubelet counts as failure.

Chapter 2 — Config Server

1. Env var > config-server > packaged YAML.
2. Native serves files from the classpath directory inside the jar; git adds audit, review, revert, and config changes without rebuilding the config-server.
3. `optional:` lets a service boot when the config-server is down — but then it boots on localhost defaults and fails later, far from the cause.
4. In its own `application.yml`, packaged in its jar. The bootstrapping principle: the source of configuration cannot configure itself; every such chain needs a self-contained root.

Chapter 3 — Eureka

1. course-service registers (POST) at startup; the gateway's discovery client refreshes its registry cache (default up to 30 s); routing then load-balances from cache. Worst case for a new instance to receive traffic: registration interval + cache refresh, tens of seconds.
2. Compose makes service *names* resolvable, and hostnames match them; in Kubernetes the pod hostname is the pod name, which no one can resolve

— so the IP must be registered instead.
3. Availability of registry data over accuracy. Helps: a switch reboot silences all heartbeats but instances still run. Hurts: mass real crashes leave dead instances registered.
4. The Service's endpoint list replaces the registry; readiness probes replace heartbeats; kube-proxy/DNS replace client-side lookup; the ReplicaSet replaces self-registration of new instances.

Chapter 4 — API Gateway

1. Browser → ts.local (hosts file) → Traefik (Ingress: `/api` → api-gateway Service) → gateway pod (predicate `/api/auth/**`, StripPrefix=2, JWT not required for `/api/auth`) → Eureka-resolved auth-service pod, arriving as `/login` → controller.
2. Google validates the redirect URI byte-for-byte; it was registered with the full `/login/oauth2/...` path, so it must arrive unmodified. API routes strip because controllers are written prefix-less.
3. Every request with a token signed by auth-service fails the gateway's validation — universal 401s on protected routes while login itself appears to work.
4. A proxy is I/O-bound with high concurrency — event loops excel. course-service does blocking JDBC work; reactive would complicate it without benefit until proven otherwise.

Chapter 5 — Security

1. CSRF rides an ambient credential (the session cookie sent automatically). A JWT in an Authorization header is attached explicitly by code; a forged cross-site request cannot add it.
2. Fifteen minutes (its expiry). Nothing — a signature cannot be revoked. `/refresh`: the refresh token is checked against the database, where revocation lives.
3. Invariant: every network path to a service passes the gateway's JWT filter. Breaks: an Ingress routing straight to a service (the deleted `/courses` route), or any in-cluster caller forging headers on the flat pod network.
4. The client-secret never leaves servers; the authorization code transits the browser. The code is single-use, short-lived, and worthless without the client-secret at exchange time.

5. HS256: one shared secret; anyone who can validate can also sign. RS256: private key signs, public key validates — so an IdP can let thousands of services validate without any of them being able to mint tokens.

Chapter 6 — Services and Common

1. (a) HTTP — the caller needs the data now. (b) Event — a fact others react to; grading must not wait on mail. (c) Neither: validation is local cryptography with the shared secret; no call.
2. auth-service only *produces* — sends fail per-call, boot succeeds. notification-service only *consumes*; without a broker it is pure idle cost, so it ships with Kafka.
3. Belongs: event records — both sides must agree on shape. Never: business logic/entities — shared behavior couples services' release cycles and hides the boundary.
4. Consumers deploy before producers, and changes are additive-only (new optional fields). Then old consumers ignore new fields and new consumers tolerate old events.

Chapter 7 — Postgres, JPA, Flyway

1. Two developers' schemas silently diverging (update applies per-database, no history); and destructive changes being impossible — update never drops/renames, so entity refactors quietly desync from the schema.
2. Each history row stores a checksum of its script; an edited script mismatches and Flyway refuses to start — the history is the audit trail, and edits would falsify it.
3. Release 1: add new column, write both (or trigger). Backfill migration. Release 2: read new column, stop writing old. Release 3: drop old column. Each step compatible with the version before it.
4. For: fault and blast-radius isolation — one server's crash, upgrade, or misconfiguration cannot touch the other's data. Against: double memory and operational surface for a homelab. Managed clouds usually take credential-isolation on one instance because servers are the billing unit.

Chapter 8 — MongoDB

1. Dictionary entries: self-contained, read/written as a unit, naturally nested, no cross-entry consistency. Enrollments: reference two other aggregates

and require consistency between them — joins and transactions are the point.
2. Into the application code (and optional collection validators). Disciplines: backward-readable document classes (new fields optional), and explicit data-fix scripts for shape changes.
3. Postgres refuses connections to a nonexistent database; Mongo creates databases on first write, so a wrong name yields empty results. Loud — failures you notice at deploy beat failures you notice at audit.
4. ClusterIP-only + no Ingress means only in-cluster callers can reach it; the compose parity also keeps dev friction zero. Invalidated the moment it is exposed beyond the cluster or shares the cluster with untrusted workloads.

Chapter 9 — MinIO / S3

1. The service signs (microseconds) instead of streaming (seconds); storage serves bytes directly. New failure mode: the URL embeds an endpoint and expiry — wrong endpoint or clock skew breaks downloads in the browser only, invisible to the service.
2. The signing service reaches MinIO as `minio:9000` (cluster DNS); the browser cannot resolve that name. The signature covers the host, so rewriting after signing invalidates it — hence signing against a second, public endpoint.
3. Expiry (15 minutes) bounds the damage window; the URL grants one object, not the bucket; and it cannot be re-signed without the keys.
4. Changes: endpoint(s), credentials (ideally to an IAM role), durability guarantees. Unchanged: every SDK call, bucket/key/presign logic — the API is the standard.

Chapter 10 — Kafka Core

1. Key by userId — per-key order is per-partition order. Partitions ≥ max consumers you ever want in the group; parallelism is capped by partition count.
2. Groups are independent: each keeps its own offsets per partition, so both groups traverse the full log; consumption is non-destructive reading plus a per-group cursor.
3. `LISTENERS` = sockets bound; `ADVERTISED_LISTENERS` = addresses *returned to clients* in metadata. Bootstrap uses the address you supplied; all subsequent connections use the advertised ones — so the failure begins after a successful hello.

4. Retries and post-processing crashes both re-deliver; the broker cannot know if you acted. Notification: dedupe by event id (or accept a rare duplicate mail — idempotence by tolerance).

Chapter 11 — Spring Kafka

1. `send()` buffers client-side and returns a future; no broker needed until delivery. The outage surfaces per-send: futures fail after timeouts, and buffered events are lost if the process dies.
2. With no committed offsets, `earliest` starts from the oldest retained record — history is processed. Once the group commits offsets, the setting is never consulted again.
3. Without: deserialization throws inside poll, offset never advances, the partition stalls on one record forever. With: the error is materialized as a typed failure, the error handler skips (or retries then routes) — in full production, to `<topic>.DLT`.
4. JSON deserialization instantiates classes named in headers; an unrestricted trust list lets a message make the consumer construct arbitrary classpath types — a gadget-injection vector.
5. When "DB write + event publish" must not diverge. It makes the state change and the *recording of the intent to publish* atomic — one local transaction; a relay makes the publish eventually happen.

Chapter 12 — Docker

1. Namespaces (isolated views) and cgroups (resource limits). No guest kernel: millisecond starts and no per-instance OS cost — but also: the kernel is shared (weaker isolation than a VM) and the JVM sees host resources unless told otherwise.
2. Inside a container `localhost` is the container itself. Compose fixes it with service-name DNS on the bridge network; Kubernetes with Service DNS — both via env-var overrides of the localhost defaults.
3. Each container has its own network namespace, so both bind "their" 5432. Conflicts return at the shared host: published host ports (hence 5433/5434) — and with `–network host`, where there is no namespace separation at all.
4. Plain `depends_on` waits for *started*; `service_healthy` waits for the healthcheck to pass — here, config-server's curl of its own `/actuator/health`, exposed by its management config.

Chapter 13 — Jib and the Registry

1. No daemon in CI, volatility-based layering (small re-pushes), non-root/OpenShift-ready images. The security one: no daemon means no docker socket in CI — the socket is root on the host.
2. The image name doubles as pull address for the cluster. They agree here because Jenkins shares the host's network namespace; if Jenkins were bridge-networked, its `localhost:5000` would be itself and pushes would fail while pulls worked.
3. An import per node per deploy, minutes of serialization, and no immutable tag history; the pipeline's `set image` to a SHA-tagged name has nothing to point at — the registry is what makes tags addressable.
4. Same tag, new content: with `IfNotPresent` the node's cached image wins silently. Fixes: `imagePullPolicy: Always` (the manifests' choice), or deploy by immutable tags/digests (the pipeline's choice) — this project does both.

Chapter 14 — Cluster Anatomy

1. You write desired state; controllers reconcile actual toward it forever. A second apply produces zero diff, so zero action — idempotence by architecture.
2. Ordering is a procedural concept; convergence replaces it: everything starts, dependents crash and retry with backoff until dependencies exist. CrashLoop during convergence is the mechanism, not a bug.
3. kubectl sends a REST delete to the API server; state store updates; the ReplicaSet controller's loop sees replicas 0 < 1 and creates a pod object; the scheduler assigns it; the kubelet on the node starts the container.
4. Apply merges desired state; it never deletes what is absent from input. `kubectl delete ingress course-service -n ts`, or an apply with `–prune` (or a GitOps controller) would.

Chapter 15 — Workloads

1. Each template change births a new ReplicaSet while old ones are retained at scale 0 — rolling update and one-command rollback fall out of keeping the counter separate from the versioned template.
2. Deployment: N interchangeable replicas, any can die, order irrelevant. StatefulSet: named replicas with per-identity storage and ordered replacement. Eureka: Deployment (registry rebuilt by re-registration). Redis

cache: Deployment if truly cache. Kafka broker: StatefulSet. Gateway: Deployment.
3. New pods must pass readiness before old pods terminate — Chapter 17's probes gate Chapter 15's rollout.
4. A Deployment update runs old and new pods concurrently; two postgres processes opening one data directory corrupt it. The StatefulSet replaces one-at-a-time and re-binds the claim to the successor, never two at once.

Chapter 16 — Config and Secrets

1. RBAC on who may read Secret objects; lifecycle separation from git; encryption at rest (optional). This project: RBAC yes (Jenkins' Role omits secret-read... it grants create/patch but the practical control is namespace scoping), git-separation yes (script), encryption at rest no.
2. Server image reads `POSTGRES_*`; Spring reads `SPRING_DATASOURCE_*`. Split Secrets mean each pod imports only names it understands — no foreign variables that mask typos or leak scope.
3. Update: `JWT_SECRET=new ./create-secrets.sh`; then immediately `kubectl -n ts rollout restart deployment/auth-service deployment/api-gateway`. Half-done: signer and validator hold different keys — every login 401s, or worse, only after the next unrelated restart.
4. The config-server's `configurations/*.yml` tree, mounted as files — retiring the config-server itself.

Chapter 17 — Probes and Resources

1. Startup failing past budget: restart. Readiness failing: removed from endpoints, no restart — safe to be aggressive. Liveness failing: restart — the one to be conservative with.
2. `/v3/api-docs` answers only after the full context (Flyway included) is up and is permitAll'd; a bound port proves only Tomcat exists. `/actuator/health` is 401 where security does not permit it (course/auth) and honest where it does (dictionary, gateway).
3. The kernel's OOM killer terminated the cgroup for exceeding its memory limit — no JVM involvement, so no exception. Check: the limit vs. `MaxRAMPercentage` arithmetic, and whether load or a leak grew the heap.
4. Scheduler reads requests (placement); kernel enforces limits. Memory is incompressible — exceeding it kills someone, so bound it predictably; CPU is compressible — limits would throttle startup exactly when the JVM needs

burst.

Chapter 18 — Networking

1. DNS resolves the Service name to a stable virtual IP; the endpoint controller keeps the Ready-pod list current; kube-proxy (or k3s' equivalent) rewrites virtual-IP traffic to a chosen pod IP; readiness decides membership.
2. Only Ready pods are endpoints, so a rolling update shifts traffic exactly when new pods pass probes and old ones terminate — zero-downtime is the probe-endpoint interaction.
3. Selector/label mismatch — the Service matches zero pods. Nothing errors because a Service with no endpoints is a legal state; the symptom is only visible in `get endpoints`.
4. `ts.local` is not real DNS; the hosts file pins it to the tailnet IP. Replacements: a real domain with actual DNS + cert-manager TLS, or a Cloudflare Tunnel publishing the hostname.

Chapter 19 — Storage

1. Pods mount PVCs; PVs (and StorageClasses) are cluster-scoped; the indirection lets the same claim get local-path here and EBS on EKS — manifests unchanged.
2. Postgres runs on one node at a time — RWO suffices. Shared uploads push people to RWX; object storage is usually the right answer instead.
3. StatefulSet deletion spares PVCs so data survives orchestration mistakes; PVC deletion (reclaim `Delete`) removes PV and data — claims, not workloads, are where data lives.
4. Persistence: survives restarts (yes). Durability: survives hardware failure (no — one SSD). Backup: survives your own mistakes (no — absent, the honest gap).

Chapter 20 — Kustomize

1. A definitive set (nothing forgotten, one command) and cross-cutting edits (namespace, labels) stamped consistently.
2. Namespace: cluster-scoped — Jenkins' namespaced Role cannot apply it. Secrets: values must not exist in git at all — shape in script, values in the operator's environment.

3. Kustomize: deploying your own app as readable YAML. Helm: distributing parameterized software to other people's clusters. Arriving as charts: Strimzi and kube-prometheus-stack.
4. Nothing — apply does not prune; mongodb keeps running, now unmanaged. Correct: `kubectl delete -f mongodb.yaml` (then remove from the list), or prune/GitOps machinery that reconciles deletions.

Chapter 21 — CI/CD Concepts

1. CI: main is always verified buildable — the Build stage. CD: every verified change reaches production unaided — the Deploy stage plus its rollout gate.
2. Jenkins sits behind NAT with no public address — GitHub cannot call in, so Jenkins polls. Same constraint: CI inside a corporate network that the cloud git host cannot reach.
3. A rebuild is a different binary — different dependency resolutions, timestamps, toolchain — so tests certified something else. Chapter 1's env-var precedence lets one artifact take per-environment config at runtime.
4. Tests skipped (re-enable, or gate elsewhere and know it); deploy not verified (rollout status/smoke tests); success defined as exit-codes not behavior (probe-backed verification, alerts).

Chapter 22 — Jenkins

1. Jobs, credentials, plugin state, build history. Image = reproducible binary+plugins; volume = irreplaceable state; together the server is cattle with a pet directory.
2. Jenkins runs on the base image's JDK 21; builds export `JAVA_HOME=JDK25_HOME` in the Jenkinsfile's environment block — per-pipeline, not global.
3. apply -k needs get/list + create/patch/update on every applied kind; rollout status needs get/list/watch on deployments and replicasets. Droppable without `set image`: nothing new — set image's get+patch is already covered; the pods/log read rules exist only for the failure handler.
4. Jib pushes to `localhost:5000`, and that name must mean the same machine to pusher and puller; host networking makes Jenkins' localhost the host's, which is what the image name promised k3s.
5. ServiceAccount = identity; Role = capability list; RoleBinding = the join. Role-not-ClusterRole caps a compromised pipeline at one namespace — blast radius as a type choice.

Chapter 23 — The Pipeline

1. SNAPSHOT is mutable — a node could serve any historical build under the same name; the SHA pins the exact commit and makes "what is running?" answerable from `kubectl describe`.
2. Build only listed modules plus their dependencies. A forgotten MODULES entry: the new service never gets an image; the deploy loop then fails (or deploys a stale image if one exists) — the pipeline's one manual coupling.
3. `kubectl rollout status –timeout=300s` — it converts probe results (Chapter 17) into pipeline exit codes.
4. All rollouts should proceed concurrently; waiting per-service serializes convergence that Kubernetes happily parallelizes — same total work, longer wall clock.
5. Changes: trigger (webhook), where builds run (ephemeral runners), secret mechanism (repo secrets). Unchanged: stage order, Maven commands, SHA tagging, the rollout verification.

Chapter 24 — Operations

1. get pods → describe pod → logs (`–previous` — essential for CrashLoop, where the current container has no history yet) → exec and probe from inside.
2. CrashLoop: the process exits — application layer. `Running 0/1`: the process lives but readiness fails — the probe's target (often a dependency) is the layer to inspect.
3. Exec into the pod and try the connection from there — the pod's DNS, network path, and injected env differ from your laptop's; your laptop proves only that Postgres is alive, not that the pod can reach it.
4. k3s pod/service traffic traverses the host firewall; blocking `10.42.0.0/16` or `10.43.0.0/16` breaks DNS and pod-to-pod calls intermittently — symptoms indistinguishable from flaky applications.

Chapter 25 — Kafka in the Cluster

1. A controller whose reconciliation loop targets custom resource kinds defined by CRDs. Strimzi added `Kafka`/`KafkaNodePool` (the cluster) and `KafkaTopic` (the contracts) — plus user, connect and bridge kinds this project does not use.

2. The operator bundle contains cluster-scoped objects (CRDs, ClusterRoles) that a namespaced Role cannot grant; `kafka.yaml` contains only namespaced custom resources, which the Role now permits. Role vs. ClusterRole is the line.
3. Advertised listeners, the controller quorum voters, the per-listener security protocol map (also the cluster ID and per-broker Services). The operator computes them from the listener declaration and the node pool.
4. The broker pod's actual status (`get pods`) and the operator log's reconciliation lines. The condition may be a stale write from a reconciliation that timed out during a long image pull; the next periodic loop corrects it.
5. Reviewed partition counts, no day-one producer/topic race, and an inventory of event contracts as objects. One partition: total order per topic, sufficient while every consumer group has one instance; partitions can be added later but never removed.

Chapter 26 — The Road Ahead

1. Consumer-group lag: the distance between the newest offset in a partition and the group's committed offset — growing lag means notification-service is falling behind or down.
2. Push: the credential lives in CI (Jenkins' kubeconfig). Pull: it never leaves the cluster; CI only writes git. Pruning resolves Chapter 20's apply-never-deletes pitfall.
3. Gateway routes change `lb://name` to `http://name:port`; Service endpoints replace registration, readiness probes replace heartbeats; delete discovery-server (and eureka client deps), free its resources.
4. They already run as non-root with group-0 permissions, so OpenShift's arbitrary-UID SCC admits them unmodified; only Ingress → Route remains.

Glossary

Actuator Spring Boot's operational endpoints (/actuator/health, /metrics, /prometheus); exposure and authorization are separate decisions (Ch. 1).

Advertised listener The address a Kafka broker tells clients to use after bootstrap — the usual cause of connects-then-times-out (Ch. 10).

Artifact Anything a build produces worth keeping (jar, image, report); must be immutable and traceable (Ch. 21).

Auto-configuration Spring Boot conditionally applying configuration based on classpath, properties, and user beans (Ch. 1).

Bootstrap servers The initial Kafka addresses a client contacts to fetch metadata; all later traffic uses advertised listeners (Ch. 10).

ClusterIP Default Service type: a stable virtual IP reachable only inside the cluster (Ch. 18).

Consumer group A named team of Kafka consumers splitting a topic's partitions; each group keeps its own offsets (Ch. 10).

Controller (k8s) A reconciliation loop converging actual state to desired state for one kind of object (Ch. 14).

CrashLoopBackOff Pod status: the container keeps exiting; restart delay doubles each time (Ch. 24).

CRD / Operator Custom resource kinds plus a controller for them — how Strimzi manages Kafka declaratively (Ch. 25).

Dead-letter topic Where a consumer routes messages that failed processing repeatedly (Ch. 11).

Deployment Workload controller for interchangeable replicas; manages ReplicaSets; enables rolling updates and rollback (Ch. 15).

Digest The immutable content hash of an image (@sha256:...); tags move, digests do not (Ch. 13).

Endpoint (k8s) A Ready pod backing a Service; the live list is maintained by label selector and readiness (Ch. 18).

Eureka Netflix/Spring Cloud service registry: registration, heartbeats, client-side caching (Ch. 3).

Flyway Versioned, immutable SQL migrations applied at startup and recorded in a history table (Ch. 7).

GitOps Deployment model where an in-cluster agent pulls desired state from git and converges the cluster to it (Ch. 26).

Ingress L7 HTTP routing object (host/path → Service), executed by an Ingress controller such as Traefik (Ch. 18).

Jib Maven plugin building layered OCI images without a Docker daemon (Ch. 13).

JWT Signed, self-contained identity token: header.payload.signature; validated without a database (Ch. 5).

KRaft Kafka's built-in Raft consensus replacing ZooKeeper (Ch. 10).

kubelet The per-node agent that runs pods and executes probes (Ch. 14).

Kustomize Template-free manifest composition: bases, overlays, patches; built into kubectl (Ch. 20).

Liveness probe "Restart me if this fails" — the probe to be most conservative with (Ch. 17).

Namespace Cluster partition; scope for names and RBAC (Ch. 14).

OOMKilled Exit 137: the kernel killed the container for exceeding its memory limit; no application trace exists (Ch. 17).

Outbox pattern Making DB-write + event-publish effectively atomic by writing the event to a table in the same transaction (Ch. 11).

Partition Kafka's unit of ordering and parallelism; an ordered immutable record sequence (Ch. 10).

Poison pill A message that fails deserialization forever, stalling its partition unless error handling skips it (Ch. 11).

Presigned URL A time-limited, signed link letting a browser access one object directly on S3/MinIO (Ch. 9).

Probe (startup/readiness/liveness) kubelet health checks gating restart, traffic, and boot grace respectively (Ch. 17).

PV / PVC / StorageClass Provisioned storage, a workload's claim on it, and the factory that makes PVs on demand (Ch. 19).

RBAC Role-based access control: ServiceAccount (identity) + Role (verbs on resources) + RoleBinding (the join) (Ch. 22).

Reconciliation The control loop: compare desired and actual state, act on the difference, forever (Ch. 14).

Registry Image storage speaking the Docker/OCI protocol; the interface between builder and cluster (Ch. 13).

Relaxed binding Spring's mapping of `SPRING_DATASOURCE_URL` to `spring.datasource.url` (Ch. 1).

ReplicaSet The counter keeping N pods of one template alive; managed by Deployments, kept for rollback (Ch. 15).

Rolling update Replacing pods incrementally, gated on readiness, so service never fully stops (Ch. 15).

Secret (k8s) Namespaced key/value object for credentials; base64-encoded, protected by RBAC not cryptography (Ch. 16).

Service (k8s) Stable name + virtual IP over a label-selected, readiness-filtered set of pods (Ch. 18).

ServiceAccount A non-human identity for API access — what Jenkins deploys as (Ch. 22).

StatefulSet Workload controller for stateful replicas: stable names, per-replica PVCs, ordered replacement (Ch. 15).

Trusted edge Gateway validates JWTs once and forwards identity headers; services trust them — valid only if no path bypasses the gateway (Ch. 5).

TOTP Time-based one-time passwords: shared secret + clock, RFC 6238 (Ch. 5).

Command Cheat Sheet

kubectl — daily

```
kubectl -n ts get pods                      # the first command, always
kubectl -n ts get pods -w                   # ...and watch it change
kubectl -n ts describe pod <pod>            # platform's view; read Events
kubectl -n ts logs <pod>                    # app's view
kubectl -n ts logs <pod> --previous         # the container that just died
kubectl -n ts exec -it <pod> -- sh          # see the world from inside
kubectl -n ts get endpoints <svc>           # is anyone behind the Service?
kubectl -n ts get events --sort-by=.lastTimestamp | tail
```

kubectl — deploy and rollback

```
kubectl apply -k deploy/k8s/base            # converge on the manifests
kubectl kustomize deploy/k8s/base | less    # render without applying
kubectl -n ts set image deployment/<s> <s>=localhost:5000/<s>:<tag>
kubectl -n ts rollout status deployment/<s> --timeout=300s
kubectl -n ts rollout undo deployment/<s>   # back to previous ReplicaSet
kubectl -n ts rollout restart deployment/<s>  # re-read Secrets, re-pull
kubectl -n ts get rs -l app=<s>             # revision history, live
```

kubectl — capacity and RBAC

```
kubectl top pods -n ts                      # actual vs requested
kubectl -n ts get pvc; kubectl get pv       # storage and its backing
kubectl auth can-i <verb> <resource> -n ts \
    --as system:serviceaccount:ts:jenkins   # test permissions dry
```

docker — host services on ts-server

```
docker ps                                   # jenkins, registry up?
```

```
docker logs -f jenkins
docker restart jenkins
curl -s localhost:5000/v2/_catalog          # what the registry holds
curl -s localhost:5000/v2/<name>/tags/list
docker system df; docker system prune       # disk hygiene (careful)
```

maven — local and CI

```
mvn -B -pl <modules> -am verify             # build listed + deps
mvn -B -pl course-service -am jib:build     # image for one service
mvn -B jib:build -Djib.to.tags=<sha>        # extra tag, as CI does
mvn dependency:tree -pl course-service      # who pulled that jar in
mvn spring-boot:run -pl course-service      # run one service raw
```

compose — the dev stack

```
docker compose up -d postgres-course kafka minio   # infra only
docker compose up --no-deps course-service         # one service, own terminal
docker compose logs -f --tail=100 <svc>
docker compose down                                # keep volumes
docker compose down -v                             # DESTROY volumes too
```

k3s — the cluster itself

```
sudo systemctl status k3s
sudo systemctl restart k3s          # rereads registries.yaml; pods bounce
sudo ls /var/lib/rancher/k3s/storage/       # PVs as directories
kubectl get nodes -o wide
```

kafka — the broker in the cluster (Ch. 25)

```
kubectl -n ts get kafka,kafkanodepools,kafkatopics   # operator's view
kubectl -n ts logs ts-kafka-dual-role-0 --tail=50    # the broker itself
# a throwaway pod with the Kafka CLI, same network position as services:
kubectl -n ts run kcli --rm -it --restart=Never \
  --image=quay.io/strimzi/kafka:1.2.0-kafka-4.3.1 -- sh
  B=ts-kafka-kafka-bootstrap:9092
```

```
  bin/kafka-topics.sh --bootstrap-server $B --list
  bin/kafka-console-consumer.sh --bootstrap-server $B \
      --topic ts.user.registered --from-beginning
  bin/kafka-consumer-groups.sh --bootstrap-server $B \
      --describe --group notification-group    # lag per partition
kubectl -n ts port-forward svc/maildev 1080:1080      # caught mail UI
```

The Configuration Files, Verbatim

The files as they stand in the repository at build time — included directly from source, so this appendix cannot drift from reality. Line-by-line commentary lives in the chapters; this is the reference copy for reading offline.

Jenkinsfile

```
pipeline {
  agent any

  options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }

  triggers { pollSCM('H/5 * * * *') }

  environment {
    JAVA_HOME = "${env.JDK25_HOME}"
    PATH      = "${env.JDK25_HOME}/bin:${env.PATH}"
    TAG       = "${env.GIT_COMMIT.take(7)}"
    NS        = 'ts'
    REGISTRY  = 'localhost:5000'
    SERVICES  = 'config-server discovery-server course-service api-gateway auth-service dictionary-
      ↪ service notification-service'
    MODULES   = 'common,config-server,discovery-server,course-service,api-gateway,auth-service,
      ↪ dictionary-service,notification-service'
  }

  stages {
    stage('Build') {
      steps {
        dir('Project/TeacherSupporter') {
          sh 'mvn -B -DskipTests -pl ${MODULES} -am verify'
        }
      }
    }

    stage('Push images') {
      steps {
        dir('Project/TeacherSupporter') {
          // `package` before jib:build is load-bearing: a bare goal runs no
          // lifecycle, so sibling modules (common) are never built in this
          // session and Jib cannot resolve com.ts:common from the reactor.
```

```
        sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
      }
    }
  }

  stage('Deploy') {
    steps {
      dir('Project/TeacherSupporter') {
        withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
          sh '''
            set -eu
            kubectl apply -k deploy/k8s/base
            for s in ${SERVICES}; do
              kubectl -n ${NS} set image deployment/${s} ${s}=${REGISTRY}/${s}:${TAG}
            done
            for s in ${SERVICES}; do
              kubectl -n ${NS} rollout status deployment/${s} --timeout=300s
            done
          '''
        }
      }
    }
  }
}

post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
        sh 'kubectl -n ${NS} get pods || true'
      }
    }
  }
}
}
```

deploy/jenkins/Dockerfile

```
# Jenkins controller for ts-server, with the build toolchain baked in.
#
# Homelab compromise, stated openly: builds run ON the controller (2 executors),
# not on agent pods. On a 2-core box, agent pods would cost more CPU than they
# save. Migrating to the Kubernetes plugin is a later, self-contained exercise.
#
#   docker build -t ts-jenkins:1 deploy/jenkins
#   see run.sh for the docker run line
FROM jenkins/jenkins:lts-jdk21

USER root

# --- JDK 25 (Temurin) -- the project targets Java 25 -----------------------
```

```
RUN apt-get update && apt-get install -y --no-install-recommends wget gpg apt-transport-https \
 && wget -qO- https://packages.adoptium.net/artifactory/api/gpg/key/public \
      | gpg --dearmor -o /usr/share/keyrings/adoptium.gpg \
 && echo "deb [signed-by=/usr/share/keyrings/adoptium.gpg] https://packages.adoptium.net/
      ↪ artifactory/deb $(. /etc/os-release && echo $VERSION_CODENAME) main" \
      > /etc/apt/sources.list.d/adoptium.list \
 && apt-get update && apt-get install -y --no-install-recommends temurin-25-jdk \
 && rm -rf /var/lib/apt/lists/*

# --- Maven 3.9 (Debian's apt maven is 3.8, too old for some plugins) --------
ARG MAVEN_VERSION=3.9.9
RUN wget -qO /tmp/mvn.tgz https://archive.apache.org/dist/maven/maven-3/${
      ↪ MAVEN_VERSION}/binaries/apache-maven-${MAVEN_VERSION}-bin.tar.gz \
 && tar -xzf /tmp/mvn.tgz -C /opt && ln -s /opt/apache-maven-${MAVEN_VERSION} /opt/
      ↪ maven \
 && rm /tmp/mvn.tgz

# --- kubectl: the deploy stage talks to k3s with this -----------------------
RUN wget -qO /usr/local/bin/kubectl \
      "https://dl.k8s.io/release/$(wget -qO- https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/
      ↪ kubectl" \
 && chmod +x /usr/local/bin/kubectl

# Jenkins itself keeps running on the bundled JDK 21; only *builds* see JDK 25.
# JAVA_HOME is therefore NOT changed here -- the Jenkinsfile sets it per stage.
ENV MAVEN_HOME=/opt/maven \
    JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64 \
    PATH=/opt/maven/bin:${PATH}

USER jenkins

# Plugins pre-installed so the first boot needs no wizard clicks beyond unlock.
RUN jenkins-plugin-cli --plugins \
      git workflow-aggregator pipeline-stage-view junit timestamper credentials-binding
```

deploy/k8s/jenkins-rbac.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: ts
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: jenkins-deployer
  namespace: ts
rules:
  # How to read a rule: "may perform <verbs> on <resources> in <apiGroups>".
  # Rules are additive; anything not listed is Forbidden. Namespace-scoped
  # (this is a Role, not a ClusterRole), so nothing here reaches kube-system.
```

```
  #
  # Verb cheat-sheet for the pipeline's commands:
  #   kubectl apply            -> get + create (new) or patch (existing)
  #   kubectl set image        -> get + patch
  #   kubectl rollout status   -> get/list/watch on deployments AND replicasets
  #   kubectl logs / get pods -> get/list on pods, get on pods/log

  # -- What `kubectl apply -k deploy/k8s/base` creates, by API group ---------
  # Core group is spelled "" (empty string). Namespace is deliberately absent:
  # it is cluster-scoped and cannot be granted by a Role, so the pipeline must
  # not try to apply namespace.yaml (ts already exists).
  - apiGroups: [""]
    resources: ["services", "persistentvolumeclaims", "secrets", "configmaps"]
    verbs: ["get", "list", "create", "patch", "update"]

  - apiGroups: ["apps"]
    resources: ["deployments", "statefulsets"]
    verbs: ["get", "list", "watch", "create", "patch", "update"]

  - apiGroups: ["networking.k8s.io"]
    resources: ["ingresses"]
    verbs: ["get", "list", "create", "patch", "update"]

  # Strimzi custom resources (milestone 6). Namespaced like everything
  # else here -- the pipeline may declare Kafka clusters and topics in ts,
  # but installing the operator/CRDs stays an admin act (install-strimzi.sh).
  - apiGroups: ["kafka.strimzi.io"]
    resources: ["kafkas", "kafkanodepools", "kafkatopics"]
    verbs: ["get", "list", "create", "patch", "update"]

  # -- Read-only extras for `rollout status` and post-failure debugging -------
  # A Deployment rolls out via ReplicaSets it owns; rollout status watches them.
  - apiGroups: ["apps"]
    resources: ["replicasets"]
    verbs: ["get", "list", "watch"]

  - apiGroups: [""]
    resources: ["pods", "pods/log", "events"]
    verbs: ["get", "list", "watch"]

  # NOT granted, on purpose:
  #   delete            -- CI should roll forward, never wipe; a human deletes.
  #   nodes, namespaces -- cluster-scoped, out of a Role's reach anyway.
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-deployer
  namespace: ts
subjects:
  - kind: ServiceAccount
    name: jenkins
    namespace: ts
roleRef:
  kind: Role
  name: jenkins-deployer
```

```
apiGroup: rbac.authorization.k8s.io
```

deploy/k8s/create-secrets.sh

```
#!/usr/bin/env bash
#
# Create the Secrets the ts namespace needs. Deliberately NOT a manifest:
# secrets do not belong in git. Re-runnable -- it replaces what is there.
#
#   ./deploy/k8s/create-secrets.sh
#
# Values match docker-compose.yml so behaviour is identical to local dev.
# Override any of them from the environment before running.

set -euo pipefail

NS="${NS:-ts}"
PG_USER="${PG_USER:-postgres}"
PG_PASSWORD="${PG_PASSWORD:-root}"
MINIO_USER="${MINIO_USER:-minioadmin}"
MINIO_PASSWORD="${MINIO_PASSWORD:-minioadmin}"
# Default matches the fallback baked into config-server's api-gateway.yml /
# auth-service.yml so dev behaviour is unchanged. Override in anything public.
JWT_SECRET="${JWT_SECRET:-
    ↪ mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}"

kubectl get namespace "$NS" >/dev/null 2>&1 || kubectl create namespace "$NS"

# Four secrets, not two: the same credential is spelled differently by the
# server that owns it and the client that consumes it. Postgres wants
# POSTGRES_USER; Spring wants SPRING_DATASOURCE_USERNAME. Keeping them as
# separate secrets means each pod's envFrom pulls in only names it understands,
# rather than a grab-bag it has to ignore half of.
#
# --dry-run=client | apply is the idempotent-create idiom: `kubectl create secret`
# alone fails if the secret exists, and there is no `kubectl create --force`.

# --- postgres, server side ---
kubectl -n "$NS" create secret generic postgres-course \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_course \
  --dry-run=client -o yaml | kubectl apply -f -

# --- postgres, client side (course-service) ---
kubectl -n "$NS" create secret generic postgres-course-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- minio, server side ---
kubectl -n "$NS" create secret generic minio \
```

```
  --from-literal=MINIO_ROOT_USER="$MINIO_USER" \
  --from-literal=MINIO_ROOT_PASSWORD="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- minio, client side (course-service) ---
kubectl -n "$NS" create secret generic minio-app \
  --from-literal=APP_S3_ACCESS_KEY="$MINIO_USER" \
  --from-literal=APP_S3_SECRET_KEY="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- postgres-auth, server side ---
kubectl -n "$NS" create secret generic postgres-auth \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_auth \
  --dry-run=client -o yaml | kubectl apply -f -

# --- postgres-auth, client side (auth-service) ---
kubectl -n "$NS" create secret generic postgres-auth-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- google oauth (auth-service) ---
# Real values come from .env / the environment; the placeholders keep the pod
# bootable when they are absent -- Google login then fails at click time, not
# the whole service at startup. Mirrors the ${GOOGLE_CLIENT_ID:placeholder}
# defaults in config-server's auth-service.yml.
kubectl -n "$NS" create secret generic google-oauth \
  --from-literal=GOOGLE_CLIENT_ID="${GOOGLE_CLIENT_ID:-placeholder}" \
  --from-literal=GOOGLE_CLIENT_SECRET="${GOOGLE_CLIENT_SECRET:-placeholder}" \
  --dry-run=client -o yaml | kubectl apply -f -

# --- jwt, shared by api-gateway (validates) and auth-service (signs) ---
kubectl -n "$NS" create secret generic jwt \
  --from-literal=JWT_SECRET="$JWT_SECRET" \
  --dry-run=client -o yaml | kubectl apply -f -

echo
kubectl -n "$NS" get secrets
```

deploy/k8s/install-strimzi.sh

```
#!/usr/bin/env bash
#
# One-time ADMIN step: install the Strimzi operator + CRDs into the ts
# namespace, watching that namespace only. Run before the first
# `kubectl apply -k deploy/k8s/base` that includes kafka.yaml.
#
# Same category as namespace creation: CRDs and the operator's RBAC are
# cluster-scoped, so the Jenkins ServiceAccount cannot (and should not)
# install them. Jenkins only applies the namespaced Kafka/KafkaTopic
```

```
# resources afterwards.
#
#   ./deploy/k8s/install-strimzi.sh

set -euo pipefail

NS="${NS:-ts}"

kubectl get namespace "$NS" >/dev/null 2>&1 || kubectl create namespace "$NS"

# Server-side apply, because the Strimzi bundle's CRDs are too large for
# the client-side last-applied-configuration annotation. Idempotent: safe
# to re-run, also how you take operator upgrades later.
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS"

kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator --timeout=300s

echo
echo "Operator ready. Kafka clusters in '${NS}' will now be reconciled."
```

deploy/k8s/base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: ts

# Applied in dependency order for readability only -- Kubernetes starts pods in
# parallel regardless. Ordering here is documentation, not orchestration.
resources:
  # namespace.yaml is intentionally NOT listed. Namespaces are cluster-scoped,
  # and the Jenkins ServiceAccount only holds a namespaced Role, so `apply`
  # would be Forbidden. The namespace is created once by create-secrets.sh
  # (or `kubectl apply -f namespace.yaml` by an admin).
  - postgres-course.yaml
  - postgres-auth.yaml
  - mongodb.yaml
  - minio.yaml
  # kafka.yaml requires the Strimzi operator (one-time admin step:
  # ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
  - kafka.yaml
  - kafka-topics.yaml
  - maildev.yaml
  - config-server.yaml
  - discovery-server.yaml
  - course-service.yaml
  - auth-service.yaml
  - dictionary-service.yaml
  - notification-service.yaml
  - api-gateway.yaml
```

```
# Stamped onto every object, so `kubectl delete -l part-of=teacher-supporter`
# cleans up the whole slice without touching kube-system.
commonLabels:
  app.kubernetes.io/part-of: teacher-supporter

# Secrets are NOT here on purpose -- see ../create-secrets.sh.
```

deploy/k8s/base/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: ts
```

deploy/k8s/base/postgres-course.yaml

```
# StatefulSet rather than Deployment: a database needs a stable identity and a
# volume that is never handed to a second replica. volumeClaimTemplates gives
# each pod its own PVC, provisioned by k3s' local-path on the node's SSD.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-course
spec:
  serviceName: postgres-course
  replicas: 1
  selector:
    matchLabels: { app: postgres-course }
  template:
    metadata:
      labels: { app: postgres-course }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
          envFrom:
            - secretRef: { name: postgres-course }
          env:
            # The official image refuses to initialise into a non-empty directory,
            # and local-path volumes arrive with a lost+found. A subdirectory
            # sidesteps it -- the standard fix, not a workaround.
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
```

```
            command: ["pg_isready", "-U", "postgres", "-d", "ts_course"]
          initialDelaySeconds: 10
          periodSeconds: 5
          failureThreshold: 6
        resources:
          requests: { memory: 256Mi, cpu: 100m }
          limits: { memory: 512Mi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests:
            storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-course
spec:
  selector: { app: postgres-course }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }
```

deploy/k8s/base/postgres-auth.yaml

```
# Second Postgres, same shape as postgres-course.yaml -- see the comments
# there for why StatefulSet, PGDATA subdirectory, and volumeClaimTemplates.
#
# Deliberately a SEPARATE server rather than a second database in the same
# one: database-per-service is the microservice contract. auth and course
# must not be able to join each other's tables, today or by accident later.
# On this box it costs ~256Mi -- cheap insurance.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-auth
spec:
  serviceName: postgres-auth
  replicas: 1
  selector:
    matchLabels: { app: postgres-auth }
  template:
    metadata:
      labels: { app: postgres-auth }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
```

```
          envFrom:
            - secretRef: { name: postgres-auth }
          env:
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "ts_auth"]
            initialDelaySeconds: 10
            periodSeconds: 5
            failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests:
            storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-auth
spec:
  selector: { app: postgres-auth }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }
```

deploy/k8s/base/mongodb.yaml

```
# Same reasoning as postgres-course.yaml: databases get a StatefulSet for the
# stable identity + a volumeClaimTemplate PVC that is never shared.
#
# No credentials, matching docker-compose.yml -- the official mongo image only
# enables auth when MONGO_INITDB_ROOT_USERNAME/PASSWORD are set. Acceptable
# here because the Service is ClusterIP: only pods inside the cluster can
# reach it, and there is no Ingress to it. If this ever went multi-tenant or
# network policies were the goal, auth would come first.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongodb
spec:
  serviceName: mongodb
  replicas: 1
  selector:
    matchLabels: { app: mongodb }
```

```
  template:
    metadata:
      labels: { app: mongodb }
    spec:
      containers:
        - name: mongodb
          image: mongo:8
          ports:
            - { name: mongo, containerPort: 27017 }
          volumeMounts:
            # Unlike postgres, mongod is happy with a lost+found in its data
            # dir, so no PGDATA-style subdirectory trick is needed.
            - { name: data, mountPath: /data/db }
          readinessProbe:
            # mongosh ping: same idea as pg_isready -- "is the server
            # accepting commands", not just "is the port open".
            exec:
              command: ["mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
            initialDelaySeconds: 10
            periodSeconds: 5
            failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests:
            storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  selector: { app: mongodb }
  ports:
    - { name: mongo, port: 27017, targetPort: 27017 }
```

deploy/k8s/base/minio.yaml

```
# MinIO is a hard startup dependency for course-service: S3StorageService's
# @PostConstruct calls headBucket and only catches NoSuchBucketException, so an
# unreachable MinIO fails bean creation. Kubernetes has no depends_on -- course
# service will CrashLoopBackOff until this pod is serving, then recover.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minio
```

```
spec:
  replicas: 1
  strategy: { type: Recreate }   # RWO volume: never two pods holding it at once
  selector:
    matchLabels: { app: minio }
  template:
    metadata:
      labels: { app: minio }
    spec:
      containers:
        - name: minio
          image: minio/minio:latest
          args: ["server", "/data", "--console-address", ":9001"]
          ports:
            - { name: s3, containerPort: 9000 }
            - { name: console, containerPort: 9001 }
          envFrom:
            - secretRef: { name: minio }
          volumeMounts:
            - { name: data, mountPath: /data }
          readinessProbe:
            httpGet: { path: /minio/health/ready, port: s3 }
            initialDelaySeconds: 10
            periodSeconds: 5
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumes:
        - name: data
          persistentVolumeClaim: { claimName: minio-data }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: minio-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: minio
spec:
  selector: { app: minio }
  ports:
    - { name: s3, port: 9000, targetPort: 9000 }
    - { name: console, port: 9001, targetPort: 9001 }
```

deploy/k8s/base/kafka.yaml

```
# Kafka via the Strimzi operator (KRaft mode -- no ZooKeeper).
#
# PREREQUISITE (one-time, admin): the operator + CRDs must be installed
# first -- ../install-strimzi.sh. Without it, applying this file fails with
# "no matches for kind Kafka". Same class of admin step as namespace.yaml.
#
# We do not create StatefulSets or Services for Kafka ourselves. We declare
# a Kafka cluster as data; the operator (Chapter 14's reconciliation idea,
# extended to custom kinds) computes the pods, Services, listener config,
# and rolling restarts from it. What we hand-tuned in docker-compose.yml --
# the advertised-listeners maze -- the operator now derives.
#
# Client address inside the cluster:  ts-kafka-kafka-bootstrap:9092
#   (<cluster-name>-kafka-bootstrap -- Strimzi's naming convention)
---
# Node pool: WHERE Kafka runs and with what storage/resources.
# One node holding both KRaft roles -- the k8s equivalent of the compose
# broker's process.roles=broker,controller. RF=1 everywhere: a homelab
# accepts zero fault tolerance; production starts at 3 nodes / RF=3.
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: dual-role
  labels:
    # Joins this pool to the Kafka resource below. The operator matches on
    # this label -- it must equal the Kafka resource's metadata.name.
    strimzi.io/cluster: ts-kafka
spec:
  replicas: 1
  roles: [controller, broker]
  storage:
    type: persistent-claim      # local-path PVC, like the databases
    size: 5Gi
    deleteClaim: false          # deleting the Kafka CR keeps the log data
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
  annotations:
    strimzi.io/kraft: enabled
    strimzi.io/node-pools: enabled
spec:
  kafka:
    # No version pinned: the operator's default (its newest supported
    # Kafka) applies, and operator upgrades roll the broker for us.
    listeners:
      # One internal plaintext listener. No TLS/auth for the same reason
      # Mongo has none: ClusterIP-only, nothing routes here from outside.
      - name: plain
        port: 9092
        type: internal
        tls: false
```

```
    config:
      # Single broker: every replication setting must be 1, or topic
      # creation fails asking for brokers that do not exist.
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168          # matches docker-compose.yml
    jvmOptions:
      # Kafka's default heap assumes a server; half a GiB serves a homelab
      # event trickle comfortably and leaves RAM for the Spring services.
      -Xms: 256m
      -Xmx: 512m
  entityOperator:
    # Reconciles KafkaTopic resources (kafka-topics.yaml) into real topics.
    # userOperator is omitted: no authentication, so no users to manage.
    topicOperator: {}
```

deploy/k8s/base/kafka-topics.yaml

```
# Topics as declared objects, reconciled by Strimzi's topic operator.
#
# Kafka would auto-create these on first use (auto.create.topics.enable
# defaults to true), but declared topics beat auto-created ones: partition
# count and retention are decisions, not defaults; they exist before the
# first producer runs (no race on day one); and `kubectl get kafkatopics`
# becomes the inventory of every event contract in the system.
#
# partitions: 1 -- total order per topic, which trivially preserves the
# per-user ordering the publishers key for. Raise partitions later only
# with a consumer that scales past one instance; note partitions can be
# added but never removed.
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.registered
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.activated
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
```

```
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.admin-provisioned
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.assignment.created
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
```

deploy/k8s/base/maildev.yaml

```
# Maildev: an SMTP server that delivers nothing -- it catches every mail
# and shows it in a web UI. The dev/homelab stand-in for a real relay;
# swapping to real mail later is only MAIL_HOST/credentials env changes
# on notification-service.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: maildev
spec:
  replicas: 1
  selector:
    matchLabels: { app: maildev }
  template:
    metadata:
      labels: { app: maildev }
    spec:
      containers:
        - name: maildev
          image: maildev/maildev:2.2.1
          ports:
            - { name: web, containerPort: 1080 }
            - { name: smtp, containerPort: 1025 }
          readinessProbe:
            tcpSocket: { port: smtp }
            periodSeconds: 5
          resources:
            requests: { memory: 64Mi, cpu: 25m }
            limits: { memory: 128Mi }
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: maildev
spec:
  selector: { app: maildev }
  ports:
    - { name: web, port: 1080, targetPort: 1080 }
    - { name: smtp, port: 1025, targetPort: 1025 }
# No Ingress on purpose: the UI is an operator tool, not part of the app.
# To read caught mail:  kubectl -n ts port-forward svc/maildev 1080:1080
# then open http://localhost:1080 on Windows.
```

deploy/k8s/base/config-server.yaml

```
# Config Server holds no state: its YAML is baked into the jar under
# classpath:/configurations (spring.profiles.active=native). No PVC, no DB.
# Every other pod blocks on this one, so it starts first in practice by virtue
# of being the only thing with no dependencies of its own.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-server
spec:
  replicas: 1
  selector:
    matchLabels: { app: config-server }
  template:
    metadata:
      labels: { app: config-server }
    spec:
      containers:
        - name: config-server
          image: localhost:5000/config-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8888 }
          env:
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          # startupProbe buys a slow JVM time to boot without forcing a long
          # liveness initialDelay. Until it passes, liveness is not evaluated.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30      # up to 150s to start
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
          resources:
```

```
          requests: { memory: 384Mi, cpu: 100m }
          limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: config-server
spec:
  selector: { app: config-server }
  ports:
    - { name: http, port: 8888, targetPort: 8888 }
```

deploy/k8s/base/discovery-server.yaml

```
# Eureka. Kept deliberately -- this project is the classic Spring Cloud
# showcase, and running it on Kubernetes alongside Service DNS is the point.
#
# Note the two service-discovery systems now coexist: Kubernetes resolves
# `discovery-server` to a ClusterIP, and Eureka resolves `course-service` to a
# pod IP that the clients registered themselves. Only the first survives a pod
# reschedule instantly; Eureka takes up to 90s to evict a dead instance.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: discovery-server
spec:
  replicas: 1
  selector:
    matchLabels: { app: discovery-server }
  template:
    metadata:
      labels: { app: discovery-server }
    spec:
      containers:
        - name: discovery-server
          image: localhost:5000/discovery-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8761 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
```

```
          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: discovery-server
spec:
  selector: { app: discovery-server }
  ports:
    - { name: http, port: 8761, targetPort: 8761 }
```

deploy/k8s/base/course-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: course-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: course-service }
  template:
    metadata:
      labels: { app: course-service }
    spec:
      containers:
        - name: course-service
          image: localhost:5000/course-service:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8082 }
          env:
            # Same values as docker-compose.yml, with compose service names
            # swapped for Kubernetes Service names. They happen to be identical,
            # because both resolve a name to an address on a private network.
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
      ↪ server:8761/eureka/" }
            - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-course:5432/
      ↪ ts_course" }
            - { name: APP_S3_ENDPOINT, value: "http://minio:9000" }

            # Eureka on Kubernetes needs this or nothing can call anything.
            # By default a client registers its *hostname*, which inside a pod is
            # the pod name -- a string no other pod can resolve. Pod IPs are
            # routable cluster-wide, so register the IP instead.
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }
```

```
        # Strimzi's bootstrap Service (milestone 6). This service both
        # produces (ts.assignment.created) and consumes (ts.user.activated).
        - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap
    ↪ :9092" }

        # Zipkin is not in the cluster yet (observability is milestone 8).
        - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

        - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
      envFrom:
        - secretRef: { name: postgres-course-app }
        - secretRef: { name: minio-app }

      # ---- Health probes ------------------------------------------------
      # No actuator in this service, so /actuator/health is a 404. The one
      # unauthenticated HTTP path that exists is springdoc's /v3/api-docs
      # (permitAll in SecurityConfig). It only answers 200 once the whole
      # Spring context is up -- which is AFTER Flyway has migrated and the
      # MinIO bucket check has run. That makes it a true "ready" signal,
      # unlike a tcpSocket check that passes as soon as Tomcat binds.

      # startupProbe: "give the JVM time to boot, then hand over."
      # While this probe is failing, readiness/liveness are NOT evaluated.
      # 12 x 10s = up to 120s of grace for a cold start on two cores.
      # If it still isn't up after that, kubelet restarts the container.
      startupProbe:
        httpGet: { path: /v3/api-docs, port: http }
        periodSeconds: 10
        failureThreshold: 12

      # readinessProbe: "should the Service send traffic here right now?"
      # Failing readiness pulls the pod out of the Service endpoints; it
      # does NOT restart anything. Cheap and frequent.
      readinessProbe:
        httpGet: { path: /v3/api-docs, port: http }
        periodSeconds: 5
        failureThreshold: 3

      # No livenessProbe on purpose. Liveness restarts the container, and
      # the only thing it would catch beyond readiness is a wedged JVM --
      # rare, and a probe firing during a long GC pause on a 2-core box
      # would kill a healthy pod. Add one later if we ever see a real hang.
      # -------------------------------------------------------------------

      resources:
        requests: { memory: 512Mi, cpu: 200m }
        limits: { memory: 768Mi }
        # No CPU limit on purpose. A limit throttles the JVM exactly during
        # class loading, when it wants every core it can get -- turning a
        # 60s start into a 200s one and failing the startupProbe.
---
apiVersion: v1
kind: Service
metadata:
  name: course-service
```

```
spec:
  selector: { app: course-service }
  ports:
    - { name: http, port: 8082, targetPort: 8082 }
# NOTE: the direct Ingress that used to live here (ts.local/courses ->
# course-service) is gone. It was milestone-3 scaffolding to prove the chain
# end-to-end, but it bypassed the api-gateway's JWT filter. All external
# traffic now enters through the gateway's Ingress (api-gateway.yaml).
# In-cluster callers still reach this Service directly -- that is what the
# Service object above is for.
```

deploy/k8s/base/auth-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: auth-service }
  template:
    metadata:
      labels: { app: auth-service }
    spec:
      containers:
        - name: auth-service
          image: localhost:5000/auth-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8081 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
    ↪ server:8761/eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }
            - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-auth:5432/
    ↪ ts_auth" }

            # Strimzi's bootstrap Service (milestone 6). Producer-only here:
            # registration/activation/admin-provisioned events.
            - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap
    ↪ :9092" }

            # Google's OAuth callback must round-trip through the browser, so
            # it uses the EXTERNAL name (ts.local via Traefik -> gateway ->
            # here), not a cluster-internal one. This exact URL must also be
            # registered under "Authorized redirect URIs" in the Google Cloud
            # console, or Google refuses with redirect_uri_mismatch.
            - { name:
    ↪ SPRING_SECURITY_OAUTH2_CLIENT_REGISTRATION_GOOGLE_REDIRECT_URI
    ↪ ,
```

```
            value: "http://ts.local/login/oauth2/code/google" }

          # Zipkin arrives with observability (milestone 8).
          - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

          - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
        envFrom:
          - secretRef: { name: postgres-auth-app }
          # Signs JWTs with the same key the gateway validates with.
          - secretRef: { name: jwt }
          - secretRef: { name: google-oauth }

        # /actuator/health is NOT usable here: this service's SecurityConfig
        # permitAll list covers /register, /login, ... and /v3/api-docs, but
        # not /actuator/** -- so health answers 401 and the kubelet would
        # restart a perfectly healthy pod forever. Same fallback as
        # course-service: /v3/api-docs only turns 200 after the full context
        # (Flyway migration included) is up, which is an honest ready signal.
        startupProbe:
          httpGet: { path: /v3/api-docs, port: http }
          periodSeconds: 10
          failureThreshold: 12
        readinessProbe:
          httpGet: { path: /v3/api-docs, port: http }
          periodSeconds: 5
          failureThreshold: 3

        resources:
          requests: { memory: 512Mi, cpu: 200m }
          limits: { memory: 768Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
spec:
  selector: { app: auth-service }
  ports:
    - { name: http, port: 8081, targetPort: 8081 }
```

deploy/k8s/base/dictionary-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dictionary-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: dictionary-service }
  template:
    metadata:
```

```
    labels: { app: dictionary-service }
  spec:
    containers:
      - name: dictionary-service
        image: localhost:5000/dictionary-service:1.0.0-SNAPSHOT
        imagePullPolicy: Always
        ports:
          - { name: http, containerPort: 8083 }
        env:
          - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
          - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
    ↪ server:8761/eureka/" }
          - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

          # Config-server's dictionary-service.yml says localhost:27017;
          # in the cluster mongo lives behind the `mongodb` Service.
          - { name: SPRING_DATA_MONGODB_URI, value: "mongodb://mongodb:27017/
    ↪ ts_dictionary" }

          # Zipkin arrives with observability (milestone 8).
          - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

          - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

        # This service's SecurityConfig permits /actuator/** -- so unlike
        # course-service (which had to use /v3/api-docs), the probe can hit
        # the purpose-built health endpoint. It aggregates a Mongo ping, so
        # "ready" here really means "can serve dictionary lookups".
        startupProbe:
          httpGet: { path: /actuator/health, port: http }
          periodSeconds: 10
          failureThreshold: 12
        readinessProbe:
          httpGet: { path: /actuator/health, port: http }
          periodSeconds: 5
          failureThreshold: 3

        resources:
          requests: { memory: 512Mi, cpu: 200m }
          limits: { memory: 768Mi }
          # No CPU limit -- same class-loading argument as course-service.
---
apiVersion: v1
kind: Service
metadata:
  name: dictionary-service
spec:
  selector: { app: dictionary-service }
  ports:
    - { name: http, port: 8083, targetPort: 8083 }
```

deploy/k8s/base/notification-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notification-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: notification-service }
  template:
    metadata:
      labels: { app: notification-service }
    spec:
      containers:
        - name: notification-service
          image: localhost:5000/notification-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8084 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
    ↪ server:8761/eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Strimzi's bootstrap Service: <cluster>-kafka-bootstrap.
            - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap
    ↪ :9092" }

            # The config's ${MAIL_HOST:localhost} default, repointed at the
            # maildev Service. Overriding the spring.mail.* keys directly
            # (relaxed binding) rather than the custom placeholder names --
            # same result, and consistent with how compose does it.
            - { name: SPRING_MAIL_HOST, value: "maildev" }
            - { name: SPRING_MAIL_PORT, value: "1025" }

            # Zipkin arrives with observability (milestone 8).
            - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

          # This service has no Spring Security at all -- no SecurityConfig,
          # no security starter -- so /actuator/health is open and the probes
          # can use the real endpoint (compare auth-service's workaround).
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
            failureThreshold: 12
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 3

          resources:
            requests: { memory: 512Mi, cpu: 100m }
            limits: { memory: 768Mi }
---
```

```
# Even a pure consumer gets a Service: Eureka registration aside, the
# actuator endpoints (and later Prometheus scraping) need an address.
apiVersion: v1
kind: Service
metadata:
  name: notification-service
spec:
  selector: { app: notification-service }
  ports:
    - { name: http, port: 8084, targetPort: 8084 }
```

deploy/k8s/base/api-gateway.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
spec:
  replicas: 1
  selector:
    matchLabels: { app: api-gateway }
  template:
    metadata:
      labels: { app: api-gateway }
    spec:
      containers:
        - name: api-gateway
          image: localhost:5000/api-gateway:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8080 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-
    ↪ server:8761/eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Zipkin is not in the cluster yet (observability is milestone 8).
            - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          envFrom:
            # JWT_SECRET -- shared with auth-service, which signs the tokens the
            # gateway validates. One secret, two consumers, or logins break in a
            # way that looks like a gateway bug.
            - secretRef: { name: jwt }

          # Unlike course-service, the gateway exposes actuator, so probes can
          # use the real health endpoint. Reactive stack (Netty) boots faster
          # than the MVC services: 6 x 10s of startup grace instead of 12.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
```

```
            periodSeconds: 10
            failureThreshold: 6
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 3

          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 512Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: api-gateway
spec:
  selector: { app: api-gateway }
  ports:
    - { name: http, port: 8080, targetPort: 8080 }
---
# Edge routing decision: with the gateway in the cluster there are two routers.
# Traefik (this Ingress) is the front door on ts.local; Spring Cloud Gateway
# knows Eureka and validates JWTs. Decision made here:
#
#   /api          -> gateway   (all REST traffic, JWT-checked)
#   /oauth2       -> gateway   (Google login redirect start)
#   /login/oauth2 -> gateway   (Google login callback)
#   /             -> free      (reserved for the frontend, milestone 7)
#
# The old direct `/courses` Ingress on course-service is deleted with this
# change: it reached the service without passing the gateway's JWT filter,
# i.e. an unauthenticated side door. From now on the ONLY way in is through
# the gateway -- which is the entire point of having one.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-gateway
spec:
  rules:
    - host: ts.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /oauth2
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /login/oauth2
            pathType: Prefix
```

```
          backend:
            service:
              name: api-gateway
              port: { number: 8080 }
```

docker-compose.yml

```
services:
  # ===== INFRASTRUCTURE =====
  postgres-auth:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_auth
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5433:5432"
    volumes:
      - postgres-auth-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5

  postgres-course:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_course
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5434:5432"
    volumes:
      - postgres-course-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5

  mongodb:
    image: mongo:8
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

  kafka:
    image: apache/kafka:3.9.0
    hostname: kafka
    ports:
      - "9092:9092"
    environment:
```

```
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://localhost
     ↪ :9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,
     ↪ PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_LOG_RETENTION_HOURS: 168
    CLUSTER_ID: MkU3OEVBNTcwNTJENDM2Qk

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  ports:
    - "9090:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:19092
  depends_on:
    - kafka

zipkin:
  image: openzipkin/zipkin:latest
  ports:
    - "9411:9411"

maildev:
  image: maildev/maildev:2.2.1
  ports:
    - "1080:1080"
    - "1025:1025"

minio:
  image: minio/minio:latest
  command: server /data --console-address ":9001"
  ports:
    - "9000:9000"   # S3 API
    - "9001:9001"   # Web console
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin
  volumes:
    - minio-data:/data
  healthcheck:
    test: ["CMD", "mc", "ready", "local"]
    interval: 10s
    retries: 5

# ===== SPRING CLOUD =====
config-server:
  build:
    context: ./config-server
    dockerfile: Dockerfile
```

```
    ports:
      - "8888:8888"
      - "5888:5888"
    environment:
      JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
        ↪ =*:5888"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"]
      interval: 15s
      timeout: 5s
      retries: 10
      start_period: 30s

  discovery-server:
    build:
      context: ./discovery-server
      dockerfile: Dockerfile
    ports:
      - "8761:8761"
      - "5761:5761"
    depends_on:
      config-server:
        condition: service_healthy
    environment:
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
      JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
        ↪ =*:5761"

  api-gateway:
    build:
      context: ./api-gateway
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
      - "5080:5080"
    depends_on:
      - discovery-server
    environment:
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
      MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
      JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
        ↪ =*:5080"

  # ===== APPLICATION SERVICES =====
  auth-service:
    build:
      context: ./auth-service
      dockerfile: Dockerfile
    ports:
      - "8081:8081"
      - "5081:5081"
    depends_on:
      config-server:
        condition: service_healthy
      postgres-auth:
```

```
      condition: service_healthy
    kafka:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-auth:5432/ts_auth
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    GOOGLE_CLIENT_ID: ${GOOGLE_CLIENT_ID}
    GOOGLE_CLIENT_SECRET: ${GOOGLE_CLIENT_SECRET}
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
     ↪ =*:5081"

course-service:
  build:
    context: ./course-service
    dockerfile: Dockerfile
  ports:
    - "8082:8082"
    - "5082:5082"
  depends_on:
    config-server:
      condition: service_healthy
    postgres-course:
      condition: service_healthy
    kafka:
      condition: service_started
    minio:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-course:5432/ts_course
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    APP_S3_ENDPOINT: http://minio:9000
    APP_S3_PUBLIC_ENDPOINT: http://localhost:9000
    APP_S3_ACCESS_KEY: minioadmin
    APP_S3_SECRET_KEY: minioadmin
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
     ↪ =*:5082"

dictionary-service:
  build:
    context: ./dictionary-service
    dockerfile: Dockerfile
  ports:
    - "8083:8083"
    - "5083:5083"
  depends_on:
    config-server:
      condition: service_healthy
    mongodb:
      condition: service_started
  environment:
```

```
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
      SPRING_DATA_MONGODB_URI: mongodb://mongodb:27017/ts_dictionary
      MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
      JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
       ↪ =*:5083"

  notification-service:
    build:
      context: ./notification-service
      dockerfile: Dockerfile
    ports:
      - "8084:8084"
      - "5084:5084"
    depends_on:
      config-server:
        condition: service_healthy
      kafka:
        condition: service_started
    environment:
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
      SPRING_MAIL_HOST: maildev
      SPRING_MAIL_PORT: 1025
      MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
      JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address
       ↪ =*:5084"

volumes:
  postgres-auth-data:
  postgres-course-data:
  mongo-data:
  minio-data:
```

config-server: configurations/api-gateway.yml

```
server:
  port: 8080
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://auth-service
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2
        - id: auth-oauth2
          uri: lb://auth-service
          predicates:
```

```
            - Path=/oauth2/**
        - id: auth-oauth2-callback
          uri: lb://auth-service
          predicates:
            - Path=/login/oauth2/**
        - id: course-service-courses
          uri: lb://course-service
          predicates:
            - Path=/api/courses/**
          filters:
            - StripPrefix=2
        - id: course-service-students
          uri: lb://course-service
          predicates:
            - Path=/api/students/**
          filters:
            - StripPrefix=2
        - id: course-service-assignments
          uri: lb://course-service
          predicates:
            - Path=/api/assignments/**
          filters:
            - StripPrefix=2
        - id: course-service-enrollments
          uri: lb://course-service
          predicates:
            - Path=/api/enrollments/**
          filters:
            - StripPrefix=2
        - id: course-service-submissions
          uri: lb://course-service
          predicates:
            - Path=/api/submissions/**
          filters:
            - StripPrefix=2
        - id: dictionary-service
          uri: lb://dictionary-service
          predicates:
            - Path=/api/dictionary/**
          filters:
            - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
```

```
  pattern:
    level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/auth-service.yml

```
server:
  port: 8081
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_auth
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID:placeholder}
            client-secret: ${GOOGLE_CLIENT_SECRET:placeholder}
            scope: openid, profile, email
            redirect-uri: http://localhost:8080/login/oauth2/code/google
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
    access-token-expiry-ms: 900000
    refresh-token-expiry-ms: 604800000
  totp:
    issuer: TeacherSupporter
  activation:
    base-url: http://localhost:3000/activate
  oauth2:
    success-redirect-uri: http://localhost:3000/oauth/callback
management:
  tracing:
```

```
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/course-service.yml

```
server:
  port: 8082
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
  servlet:
    multipart:
      max-file-size: 20MB
      max-request-size: 25MB
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: course-group
      auto-offset-reset: earliest
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
```

```
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
      ↪ JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000}
    # Browser-reachable endpoint used to sign download URLs. When the service runs
    # inside Docker the internal endpoint is minio:9000, which a browser cannot resolve.
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000}
    region: ${APP_S3_REGION:us-east-1}
    access-key: ${APP_S3_ACCESS_KEY:minioadmin}
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PRESIGNED_EXPIRY_MIN:15}
resilience4j:
  circuitbreaker:
    instances:
      authService:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/dictionary-service.yml

```
server:
  port: 8083
  error:
    include-message: always
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/ts_dictionary
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
```

```
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/notification-service.yml

```
server:
  port: 8084
  error:
    include-message: always
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: notification-group
      auto-offset-reset: earliest
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
        ↪ JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto
  mail:
    host: ${MAIL_HOST:localhost}
    port: ${MAIL_PORT:1025}
    username: ${MAIL_USERNAME:}
    password: ${MAIL_PASSWORD:}
    properties:
      mail.smtp.auth: false
      mail.smtp.starttls.enable: false
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```